

Image classification

Image classification - the problem

- Training data is a set of high-dimensional points



Image classification

- Training data is a set of high-dimensional points
- We could separate the space into regions for different classes

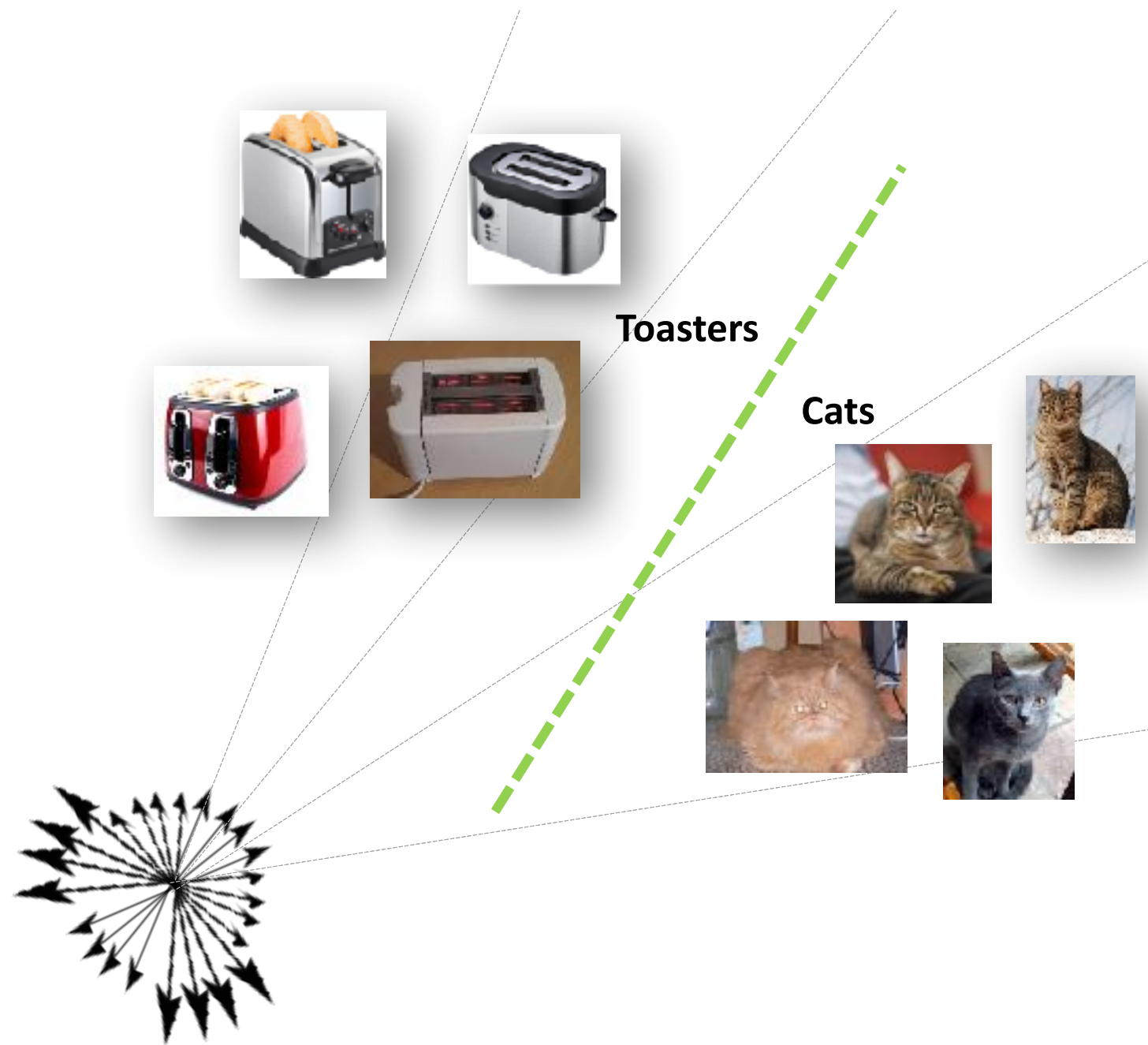


Image classification

- Training data is a set of high-dimensional points
- We could separate the space into regions for different classes



Image classification

- Training data is a set of high-dimensional points
- Approach 1: separate the space into regions for different classes
- Approach 2: Estimate the distribution of points in each class

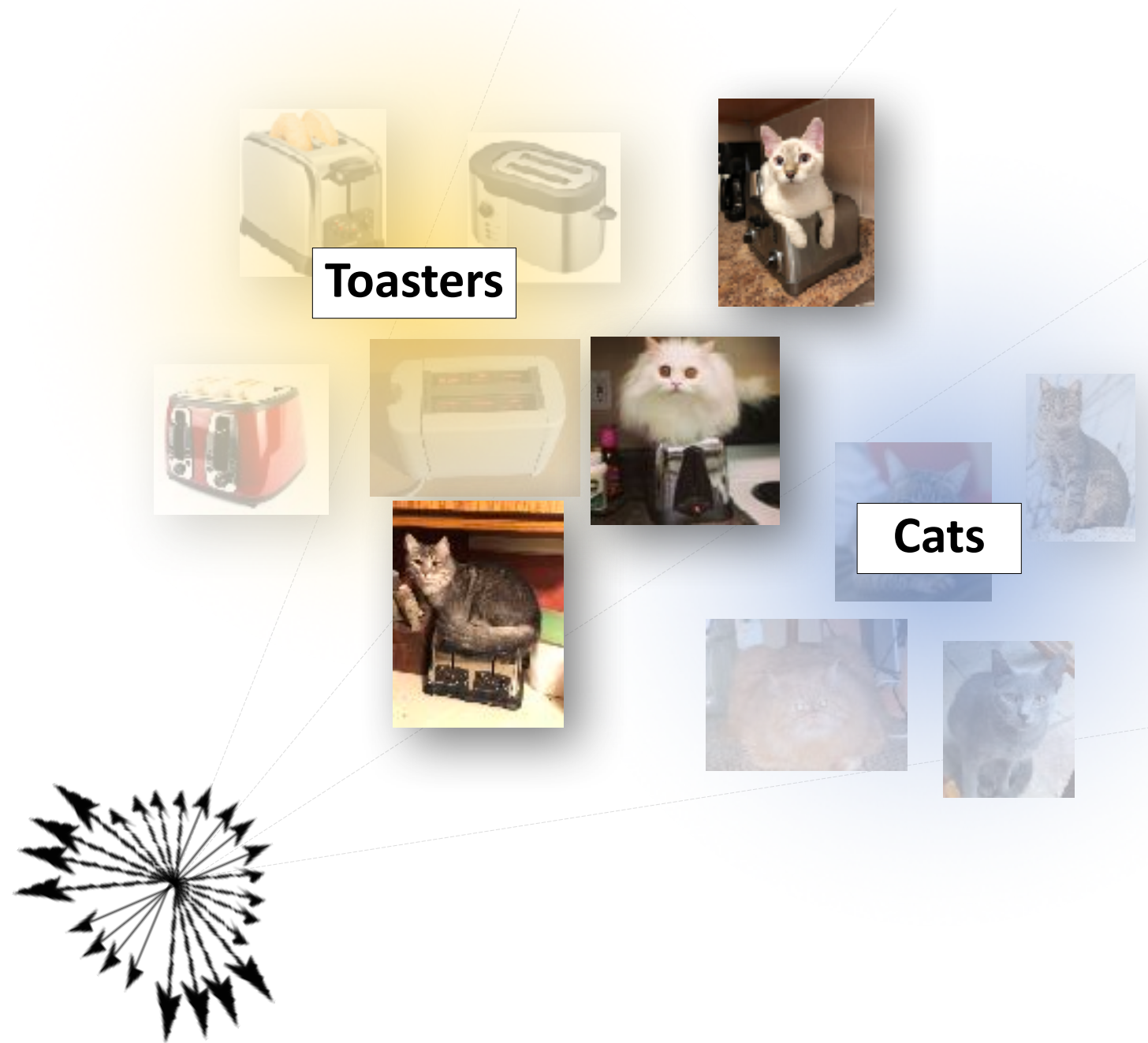


Image Classifiers

- Input: an image
- Output: the class label for that image
- Label is generally one or more of the discrete labels used in training

$$f\left(\text{img_cat}\right) = \text{"Cat"}$$

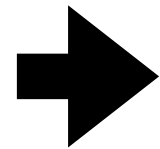
$$f\left(\text{img_dog}\right) = \text{"Dog"}$$

$$f\left(\text{img_toaster}\right) = \text{"Toaster"}$$

```
def classifier(image):  
    //Do some stuff  
    return class_label;
```

Training & Testing a Classifier

```
def classify_image(image):  
    # Some magic here?  
    return class_label
```



```
def train(images, labels):  
    # Machine learning!  
    return model
```

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```

Rule based / engineered

Machine Learning

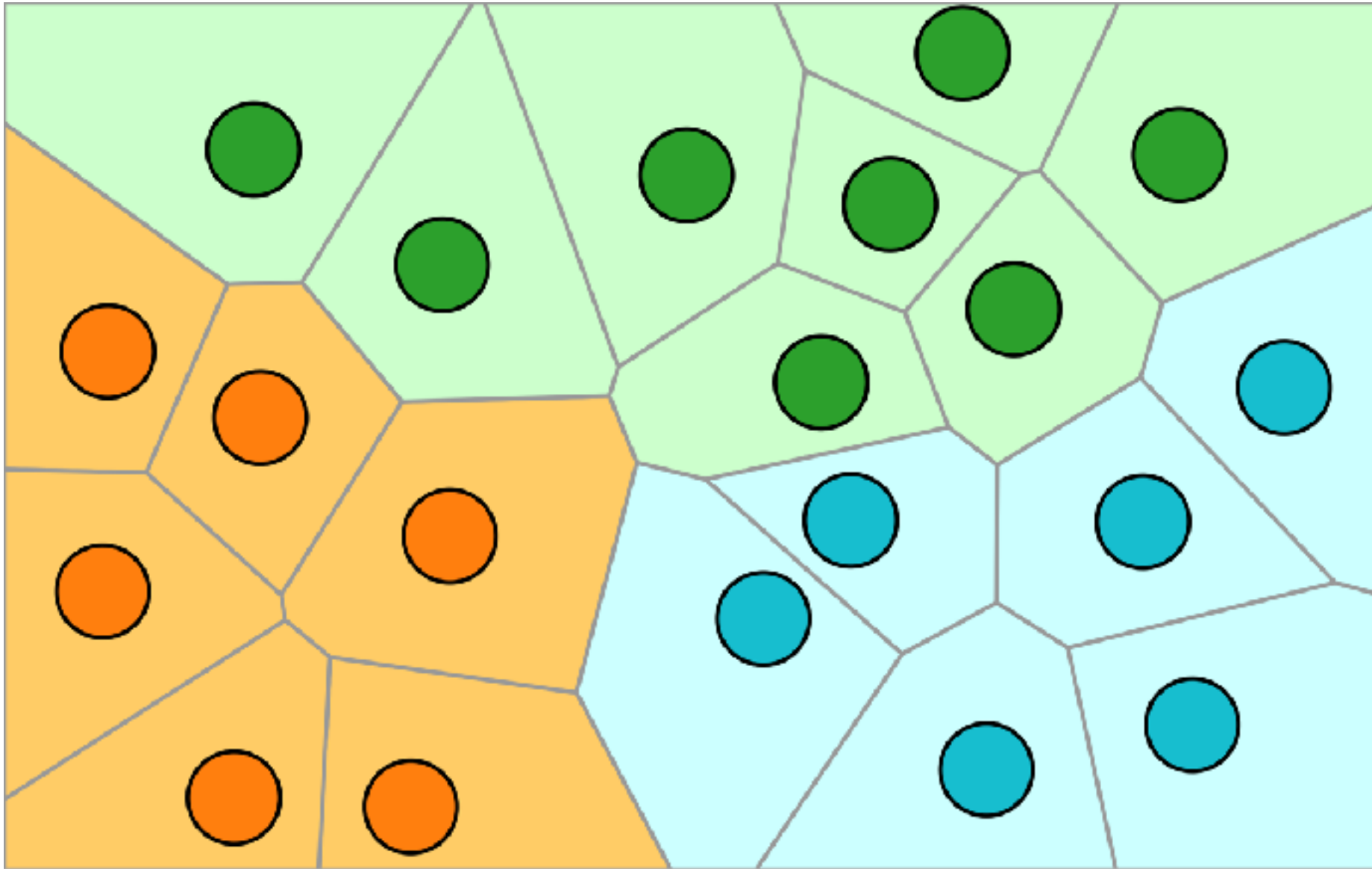
Training & Testing a Classifier

- Collect a database of images with labels
- Use ML to train an image classifier
- Evaluate the classifier on test images



Nearest neighbor classifier

Nearest neighbor classifier



First classifier: Nearest Neighbor

```
def train(images, labels):  
    # Machine learning!  
    return model
```



Memorize all data
and labels

```
def predict(model, test_images):  
    # Use model to predict labels  
    return test_labels
```



Predict label of the
most similar
training image

Example Dataset: CIFAR10

10 classes

50,000 training images

10,000 testing images

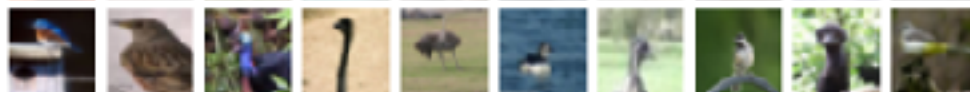
airplane



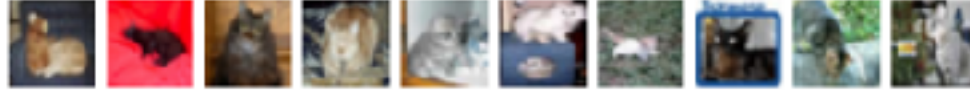
automobile



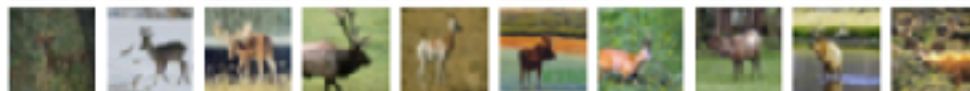
bird



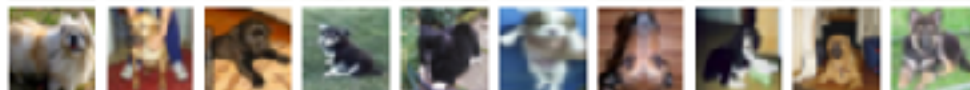
cat



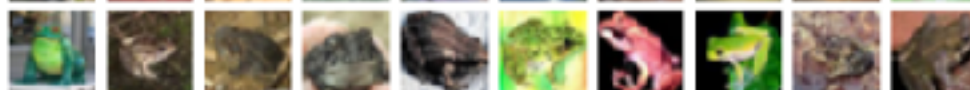
deer



dog



frog



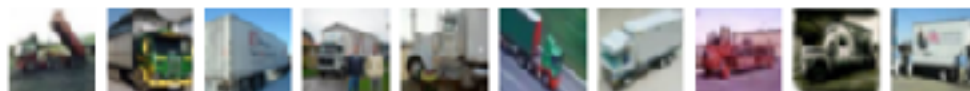
horse



ship



truck

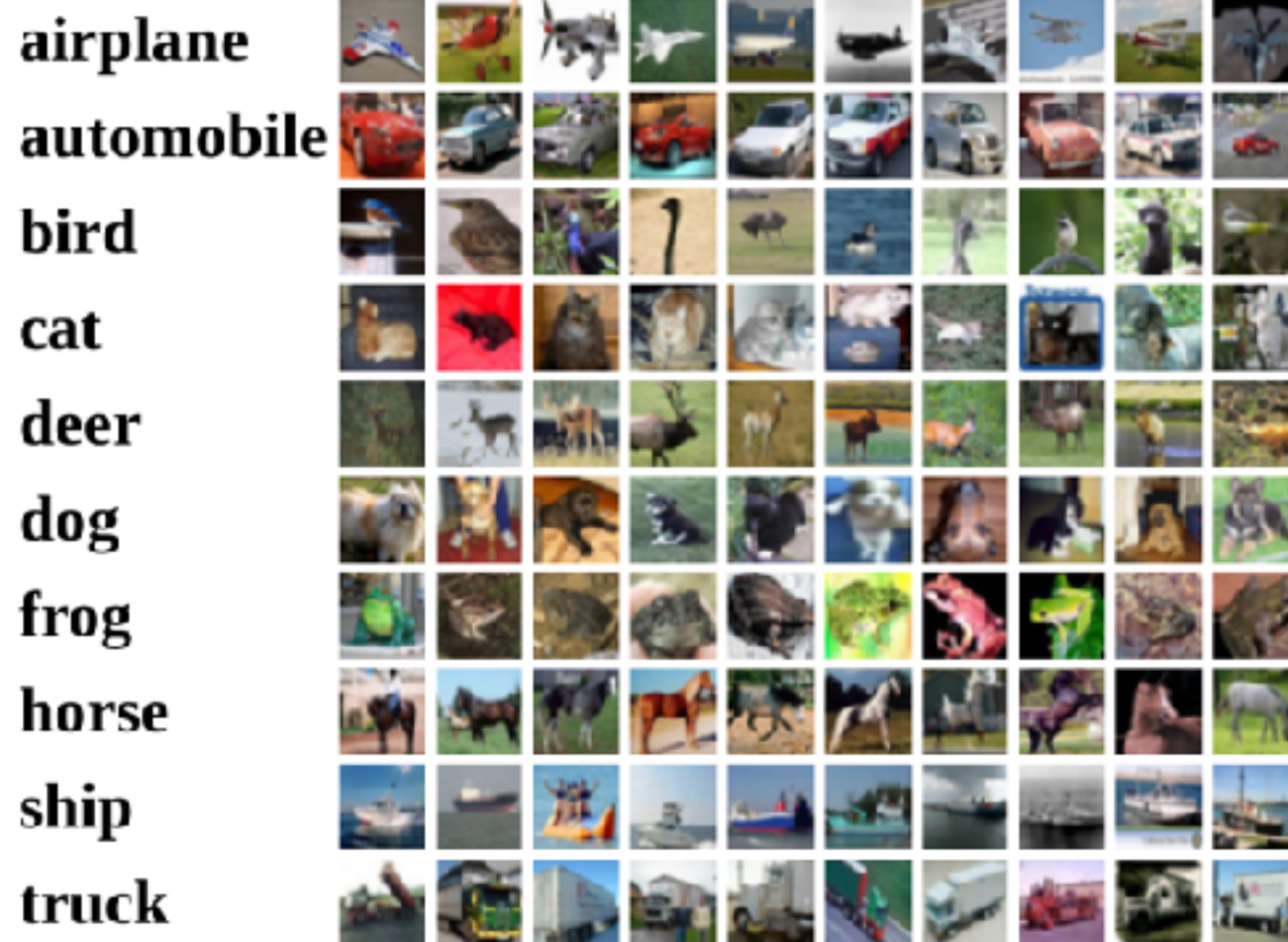


Example Dataset: CIFAR10

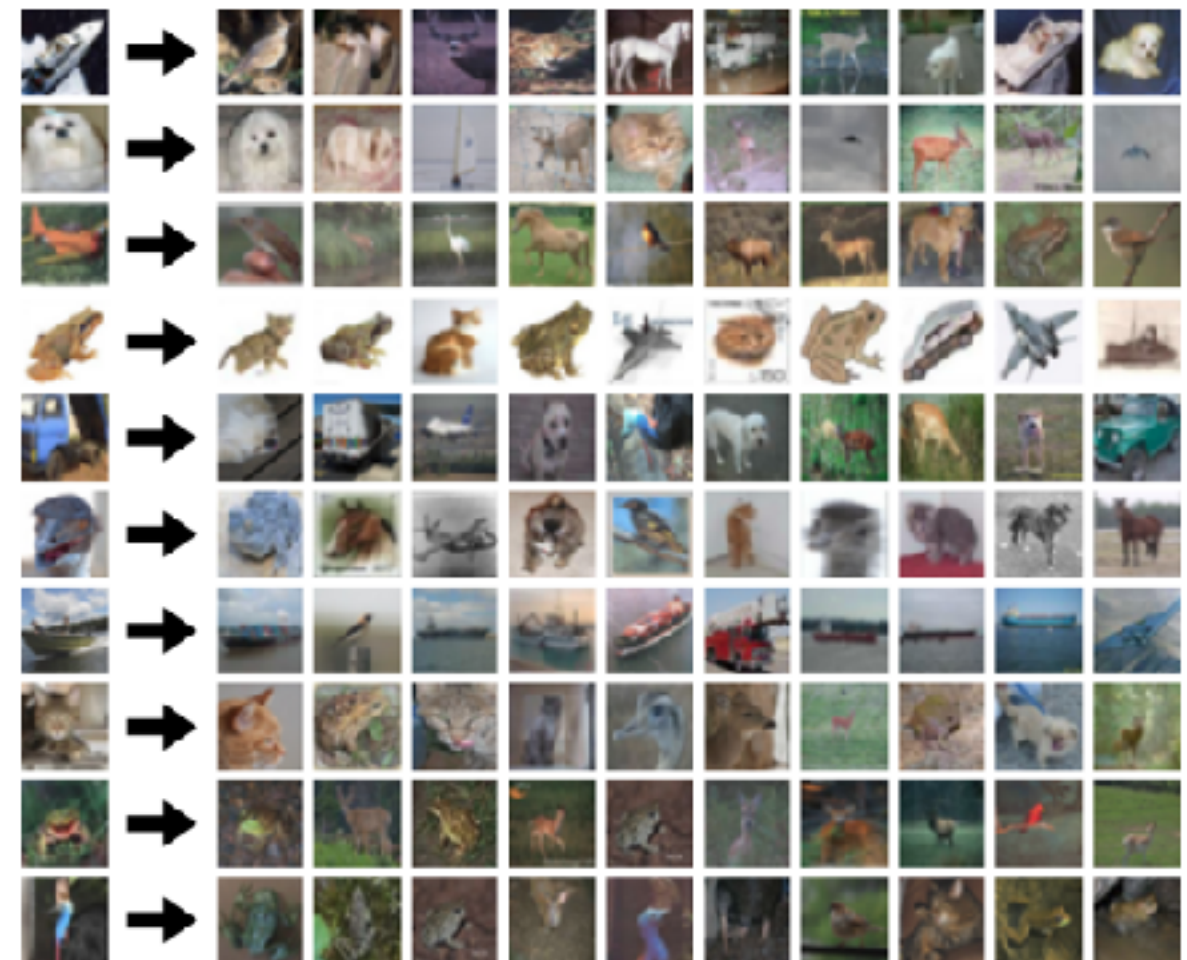
10 classes

50,000 training images

10,000 testing images



Test images & nearest neighbors

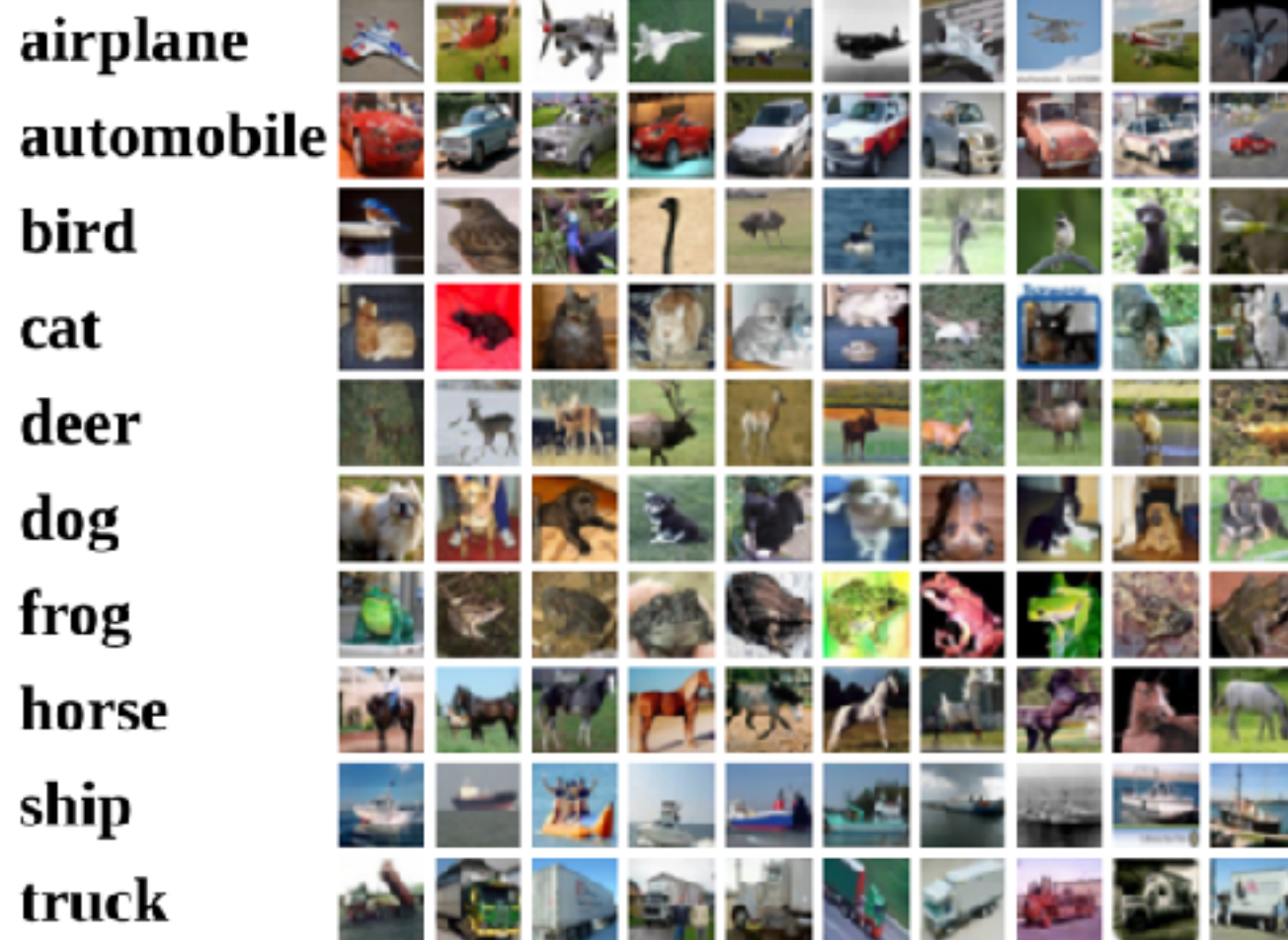


Example Dataset: CIFAR10

10 classes

50,000 training images

10,000 testing images

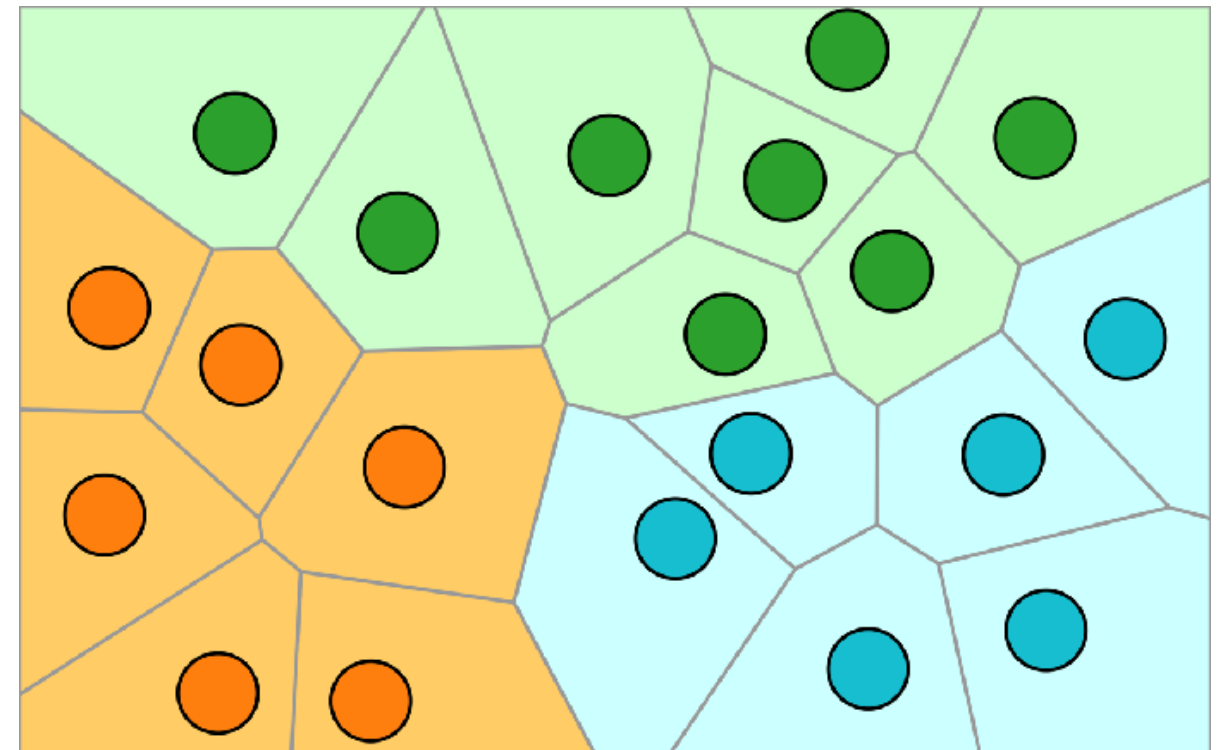


Test images & nearest neighbors



Complexity and Storage

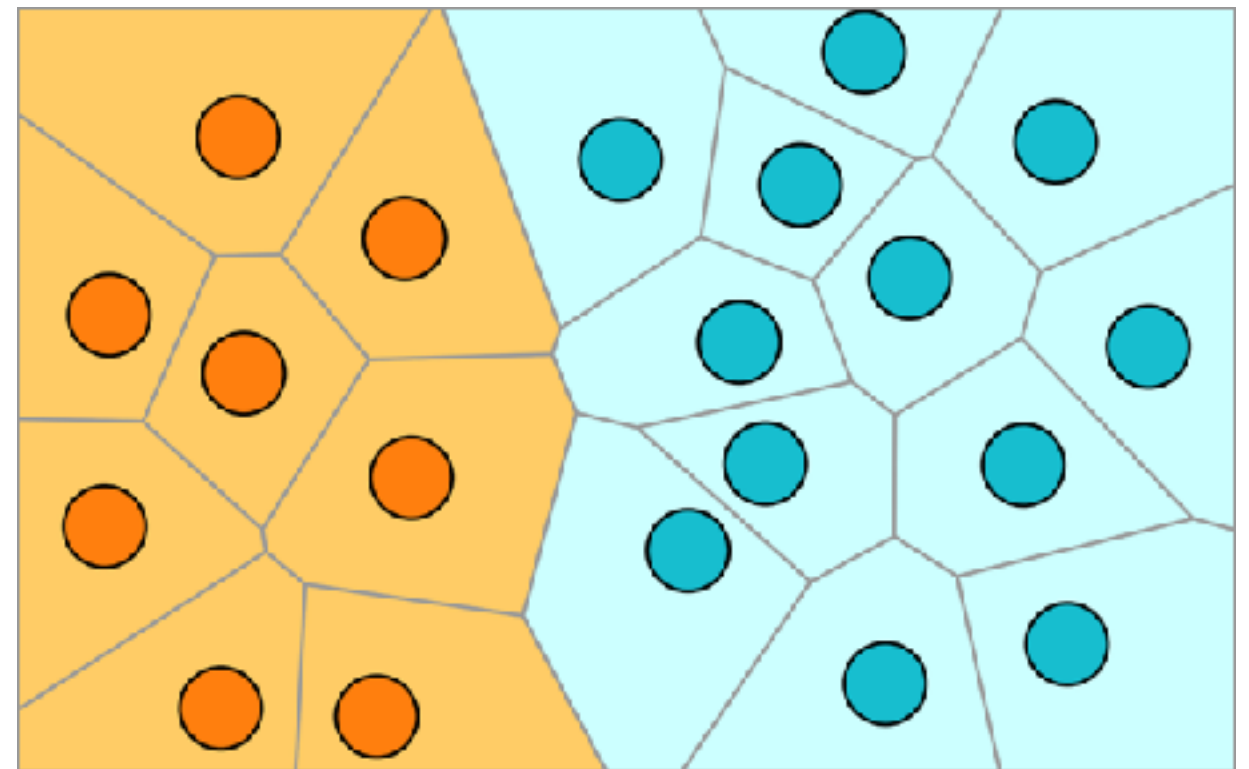
- N training images, M test images
 - Training: $O(1)$
 - Testing: $O(MN)$
- As the number of dimensions increases, the same amount of data becomes more sparse.
- Amount of data we need ends up being exponential in the number of dimensions
- We often need the opposite:
 - Slow training is ok
 - Fast testing is necessary



Linear classifier

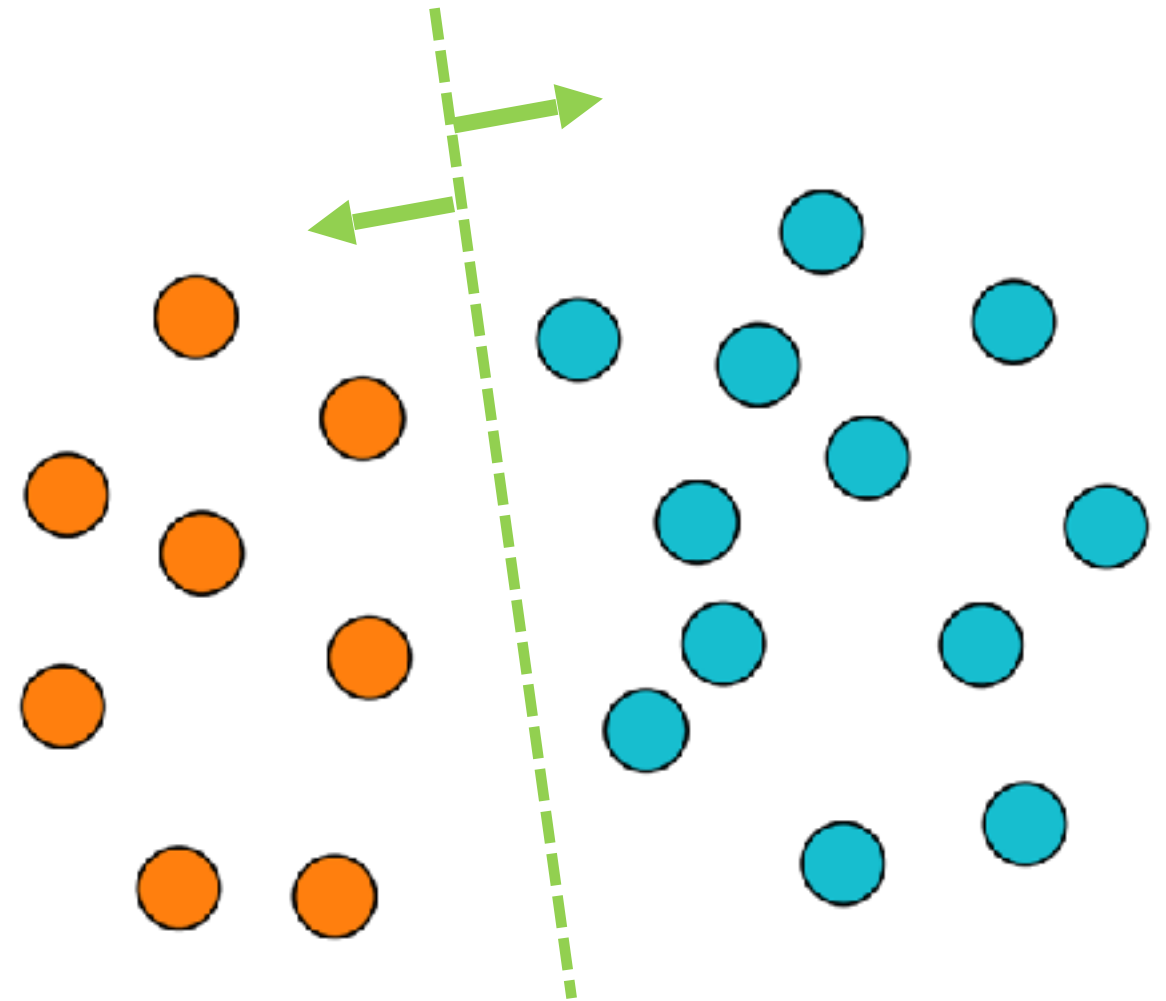
Linear Classification vs. Nearest Neighbors

- Nearest Neighbors
 - Store every image
 - Find nearest neighbors at test time, and assign same class



Linear Classification vs. Nearest Neighbors

- Nearest Neighbors
 - Store every image
 - Find nearest neighbors at test time, and assign same class
- Linear Classifier
 - Store hyperplanes that best separate different classes
 - We can compute continuous class score by calculating (signed) distance from hyperplane



Parametric Approach



image parameters

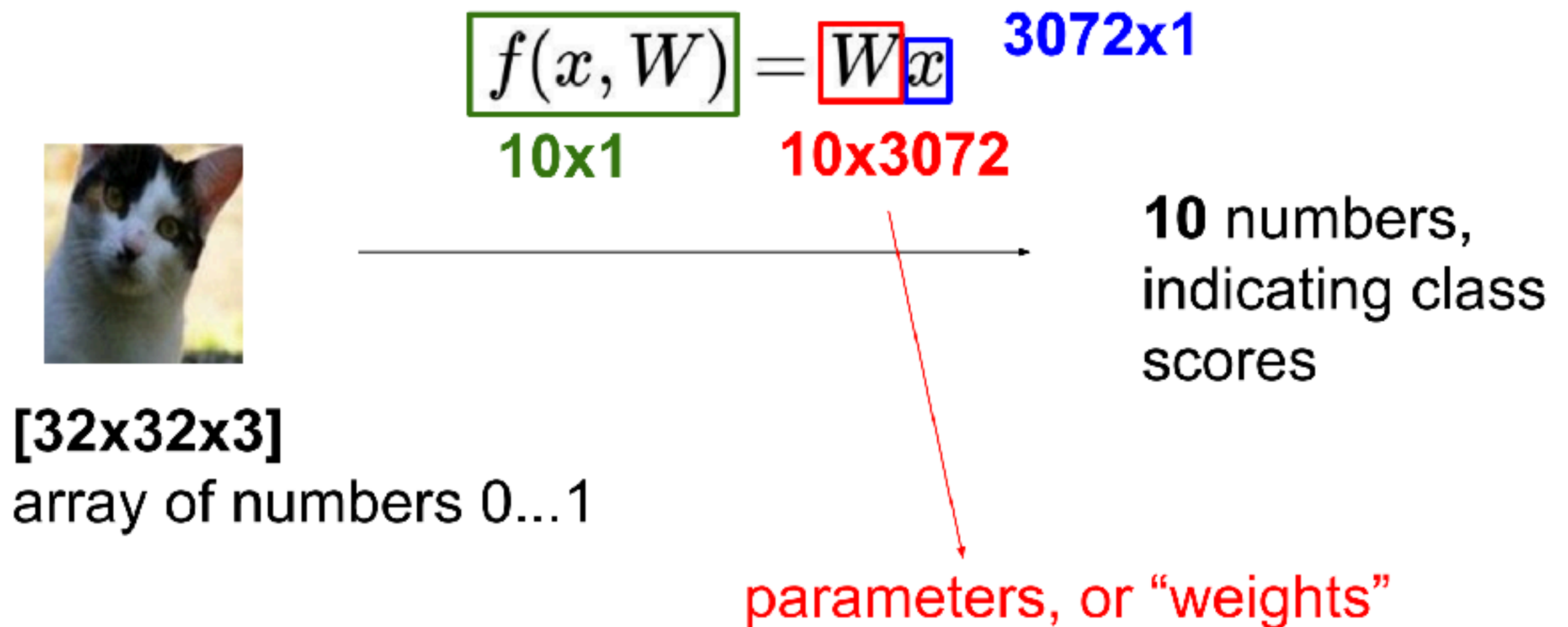
$f(\mathbf{x}, \mathbf{W})$

10 numbers,
indicating class
scores

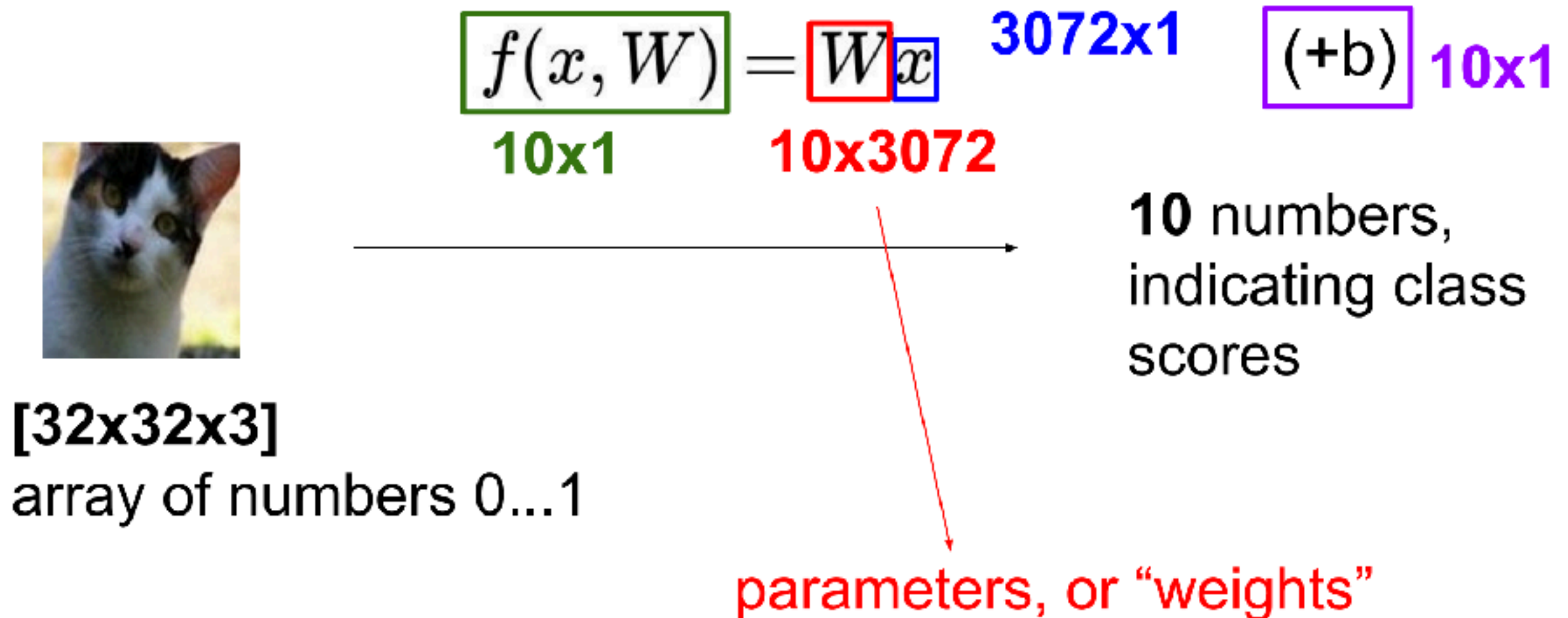
[32x32x3]

array of numbers 0...1
(3072 numbers total)

Parametric Approach: Linear

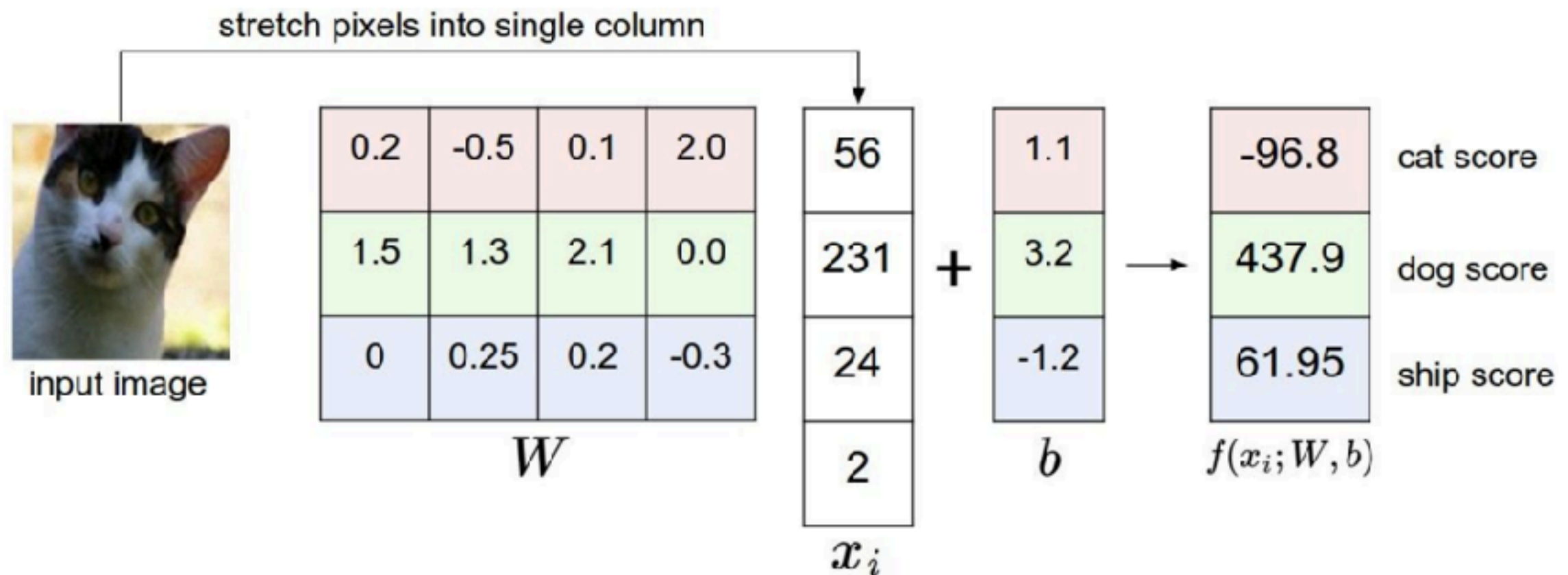


Parametric Approach: Linear



Algebraic interpretation

Example with an image with 4 pixels, and 3 classes (**cat**/**dog**/**ship**)



Geometric Interpretation

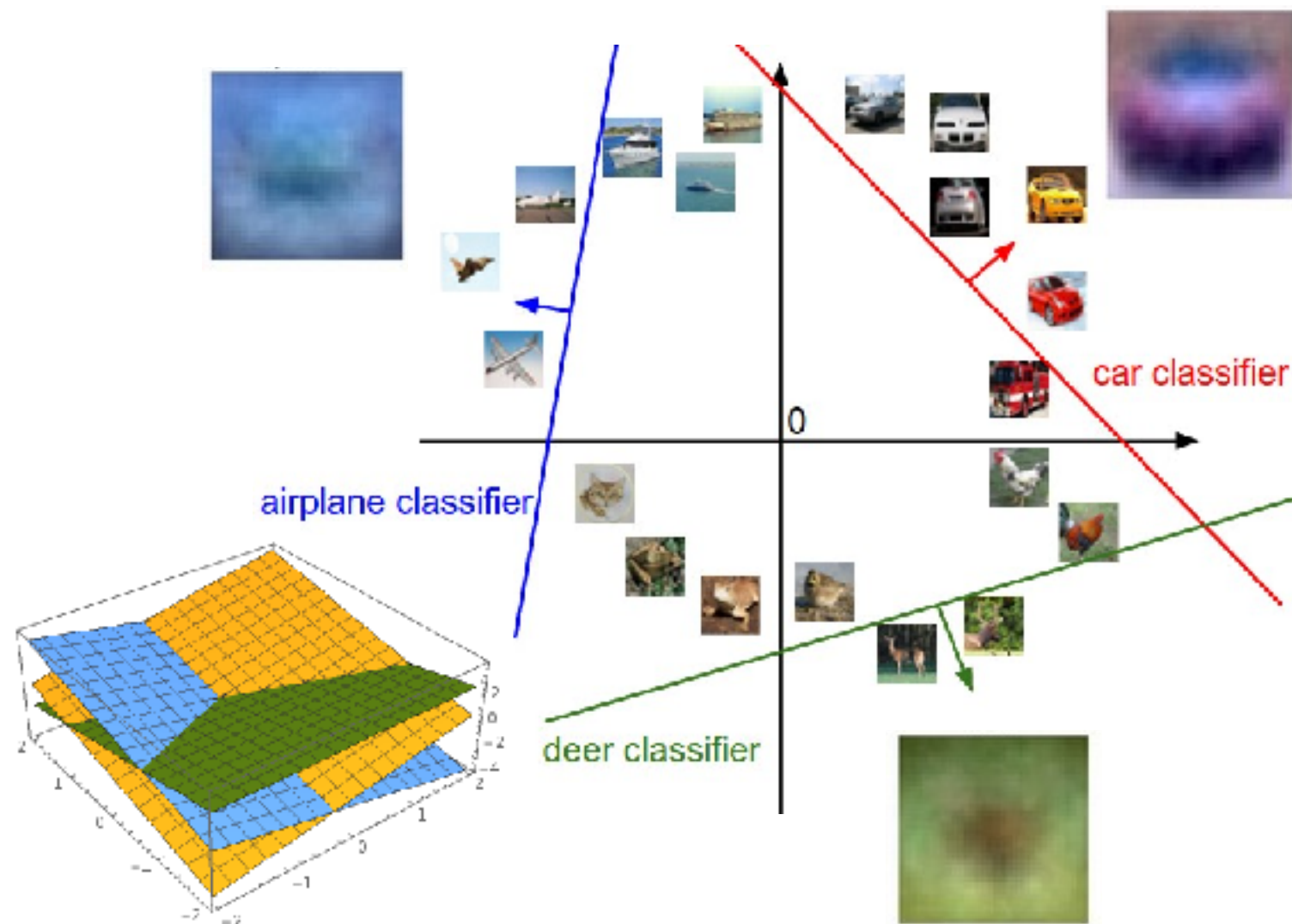
- Parameters define a hyperplane for each class

$$f(\mathbf{x}_i, \mathbf{W}, \mathbf{b}) = \mathbf{W}\mathbf{x}_i + \mathbf{b}$$

- We can think of each class score as defining a distribution that is proportional to distance from the corresponding hyperplane



Geometric interpretation



Cat image by [Nikita](#) is licensed under [CC-BY 2.0](#)

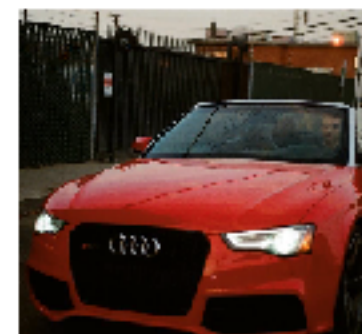
Visual interpretation

We can think of the rows in W as templates for each class
(Remember PCA's principal components - a similar thing)



So far: Defined a (linear) score function $f(x, W) = Wx + b$

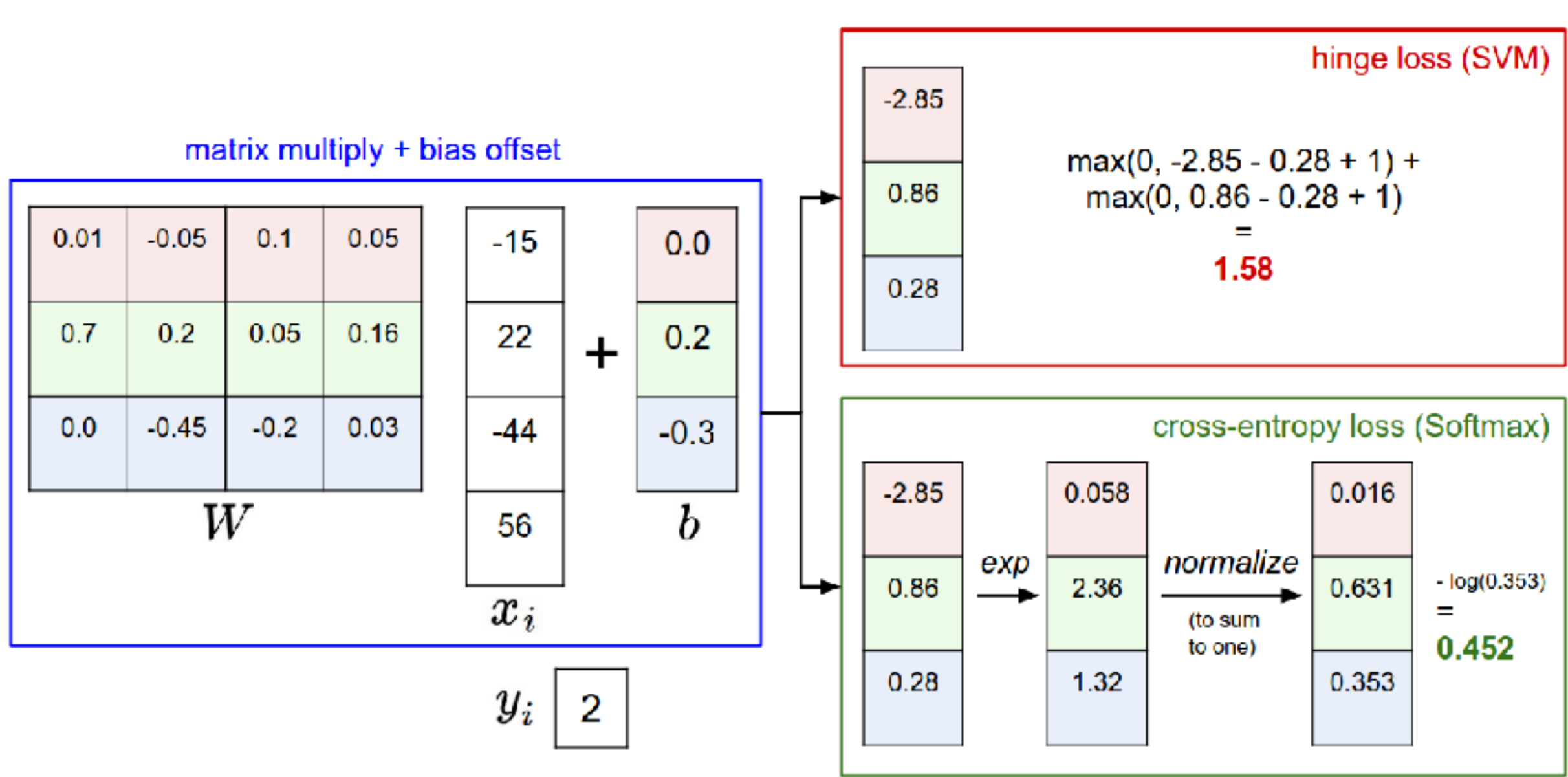
Example class
scores for 3
images for
some W :



How can we tell
whether this W
is good or bad?

airplane	-3.45	-0.51	3.42
automobile	-8.87	6.04	4.64
bird	0.09	5.37	2.65
cat	2.9	-4.22	5.1
deer	4.40	-4.79	2.64
dog	0.02	3.50	5.55
frog	3.70	4.49	-4.34
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.24

Max-Margin and Softmax losses



Loss functions

$$s = f(x; W) \stackrel{\text{e.g.}}{=} Wx$$

$$L_i = -\log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

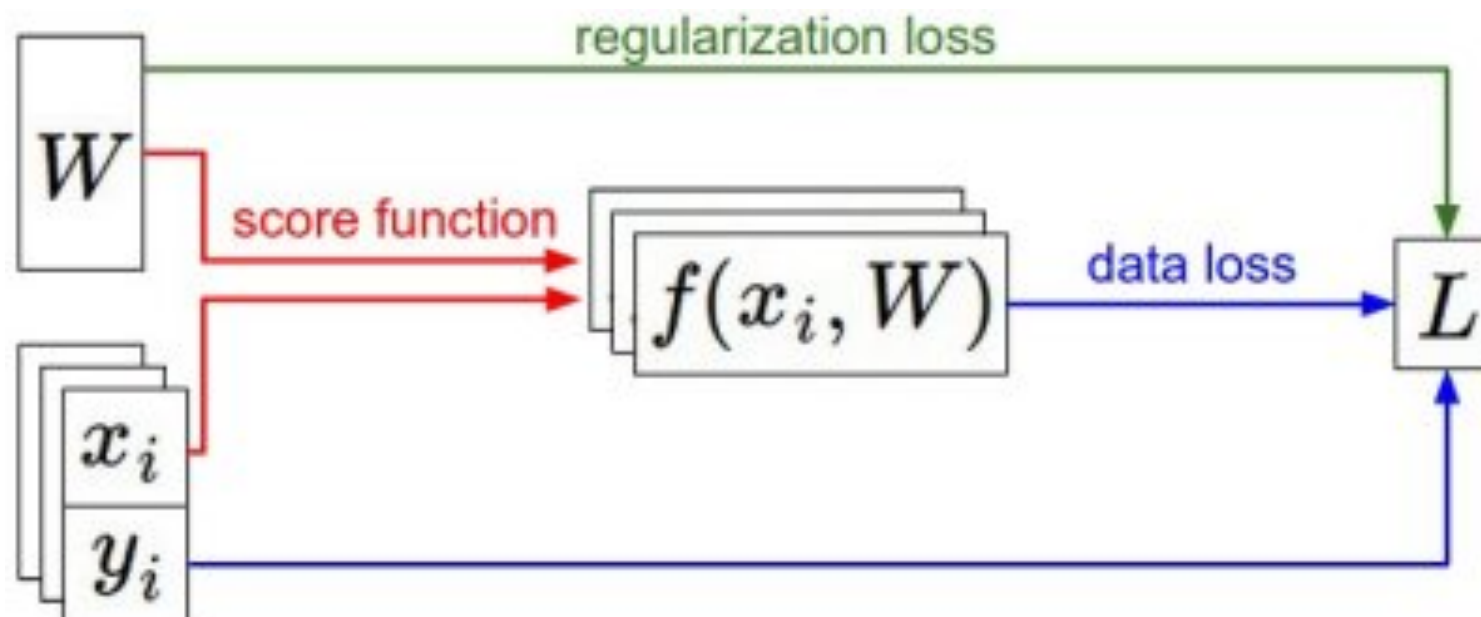
Softmax

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

SVM

$$L = \frac{1}{N} \sum_{i=1}^N L_i + R(W)$$

Full loss



Stochastic Gradient Descent (SGD)

$$L(W) = \frac{1}{N} \sum_{i=1}^N L_i(x_i, y_i, W) + \lambda R(W)$$

Full sum expensive when N is large!

$$\nabla_W L(W) = \frac{1}{N} \sum_{i=1}^N \nabla_W L_i(x_i, y_i, W) + \lambda \nabla_W R(W)$$

Approximate sum using a **minibatch** of examples
32 / 64 / 128 common

```
# Vanilla Minibatch Gradient Descent
```

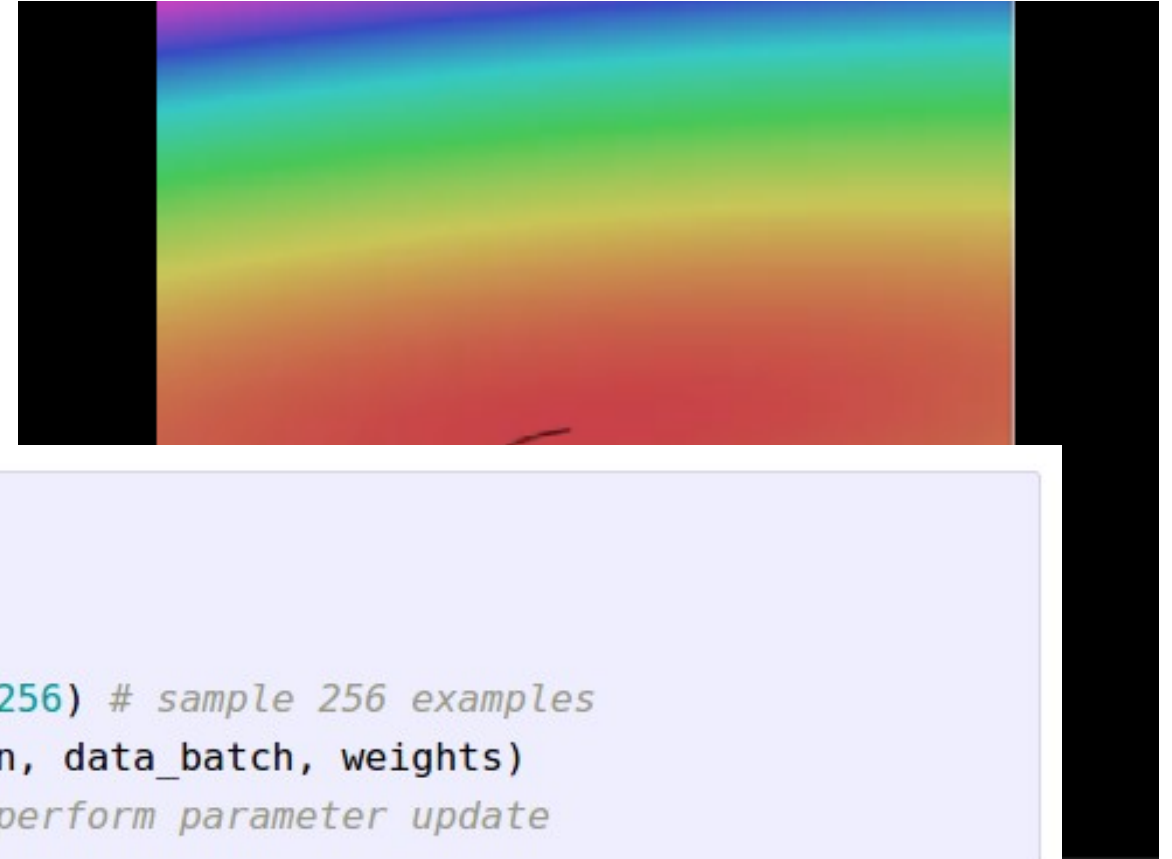
```
while True:
```

```
    data_batch = sample_training_data(data, 256) # sample 256 examples
```

```
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```

Optimization



```
# Vanilla Minibatch Gradient Descent
```

```
while True:
```

```
    data_batch = sample_training_data(data, 256) # sample 256 examples
```

```
    weights_grad = evaluate_gradient(loss_fun, data_batch, weights)
```

```
    weights += - step_size * weights_grad # perform parameter update
```

```
# Vanilla Gradient Descent
```

```
while True:
```

```
    weights_grad = evaluate_gradient(loss_fun, data, weights)
```

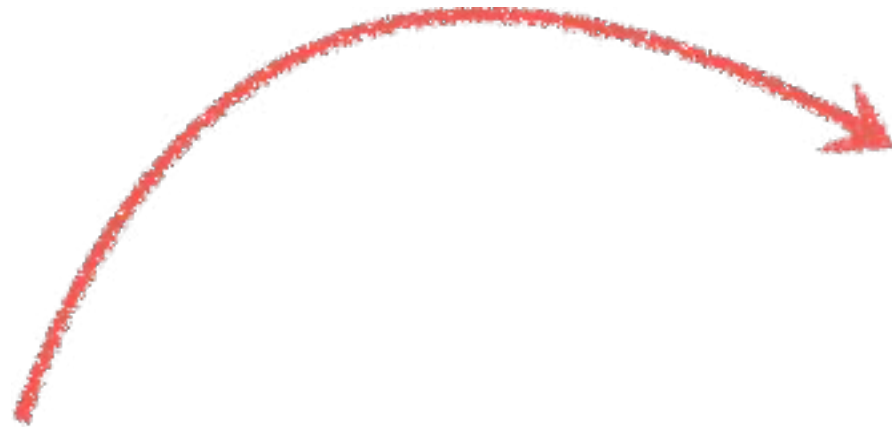
```
    weights += - step_size * weights_grad # perform parameter update
```

Bag-of-words

What object do these parts belong to?



Some local feature are
very informative



An object as



a collection of local features
(bag-of-features)

- deals well with occlusion
- scale invariant
- rotation invariant

Using image features for classification

- Use image features!
- Problem: Each image gives a set of features with variable size. How to classify?
- Idea: Compute fixed-size histograms of features matching a „feature dictionary“

Dictionary Learning:

Learn Visual Words using clustering

Encode:

build Bags-of-Words (BOW) vectors
for each image

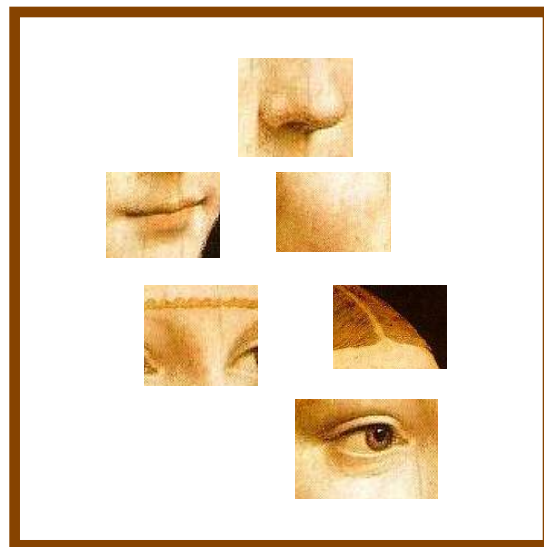
Classify:

Train and test data using BOWs

Dictionary Learning:

Learn Visual Words using clustering

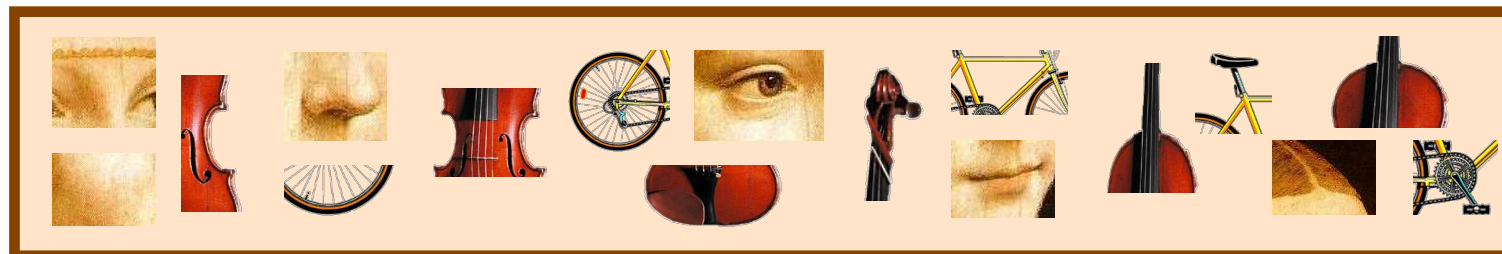
1. extract features (e.g., SIFT) from images

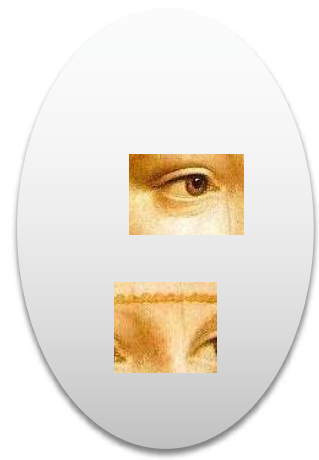


Dictionary Learning:

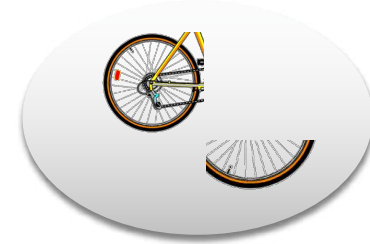
Learn Visual Words using clustering

2. Learn visual dictionary (e.g., K-means clustering)



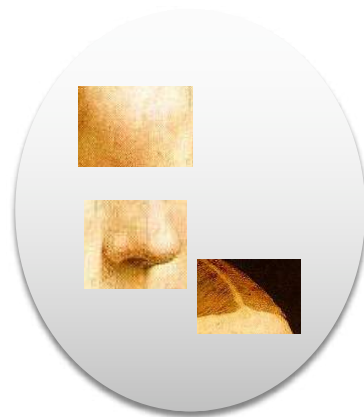


1. Quantization: image features gets associated to a visual word (nearest cluster center)



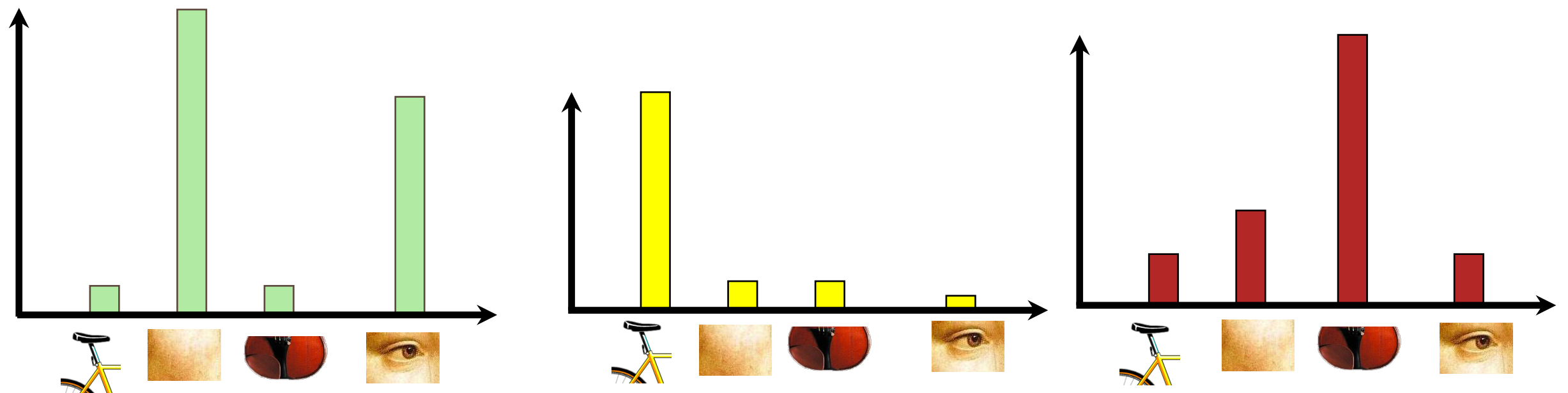
Encode:

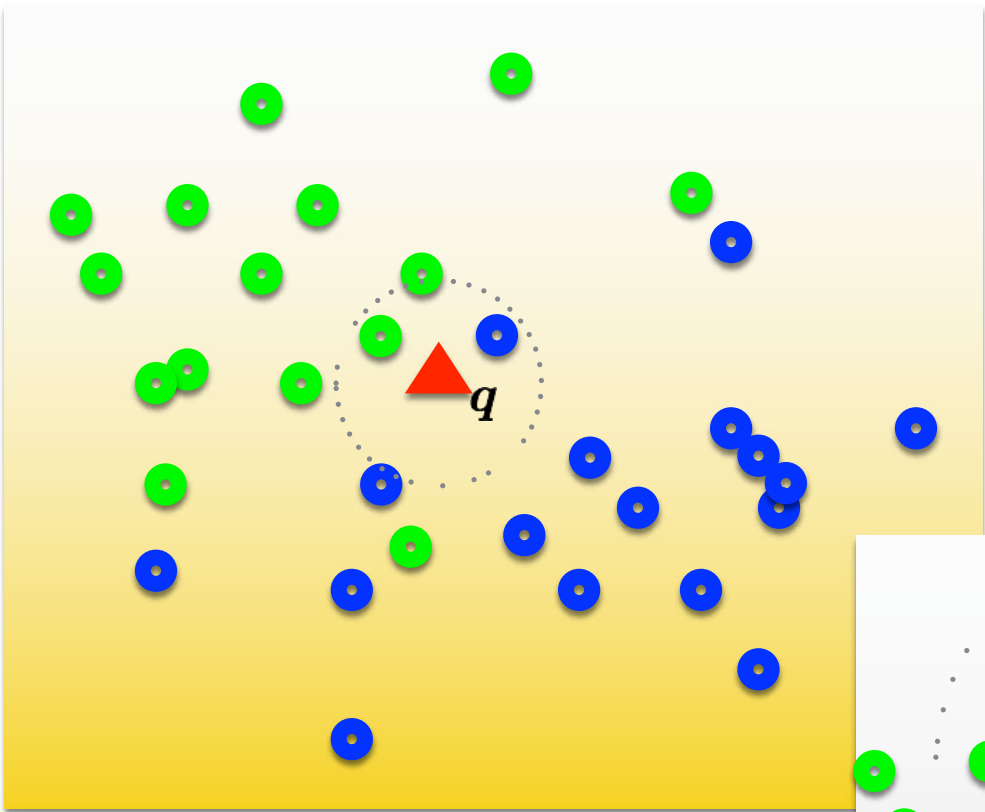
build Bags-of-Words (BOW) vectors
for each image



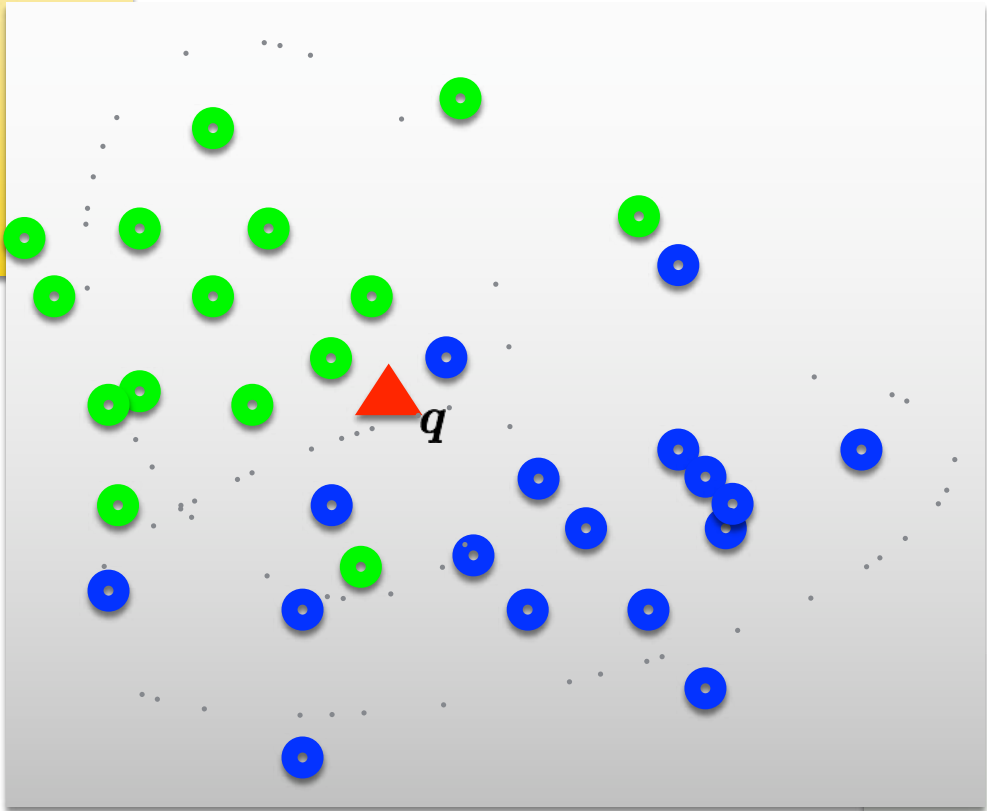
Encode:
build Bags-of-Words (BOW) vectors
for each image

2. Histogram: count the
number of visual word
occurrences

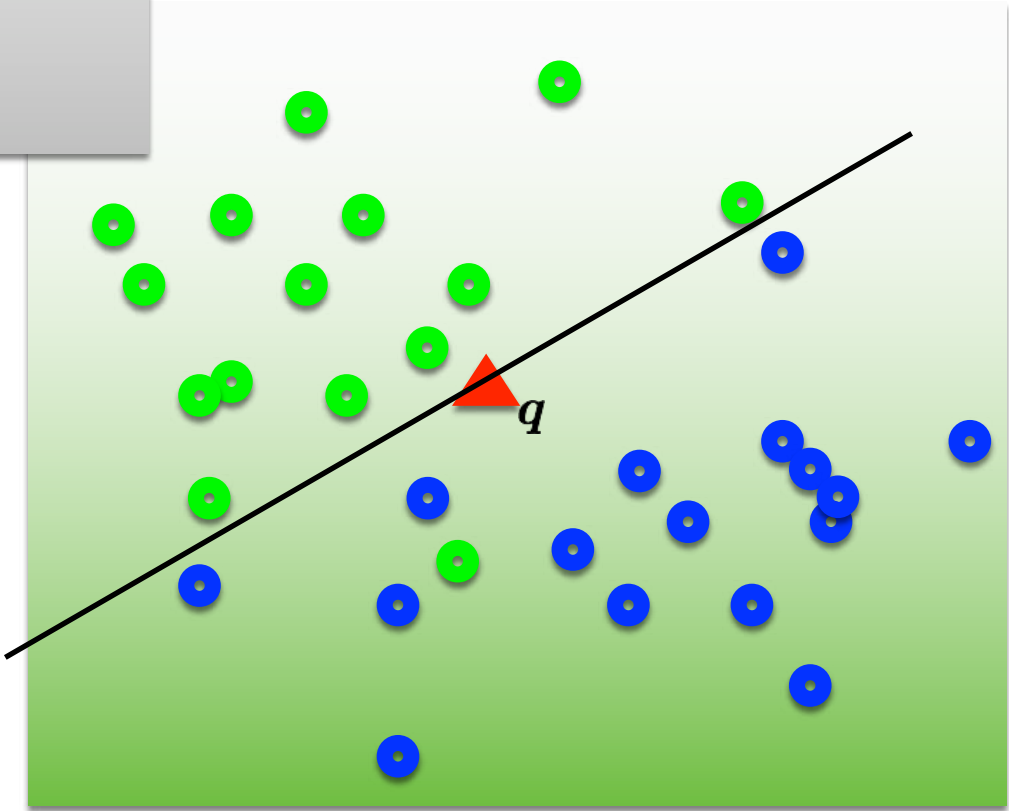




K nearest neighbors



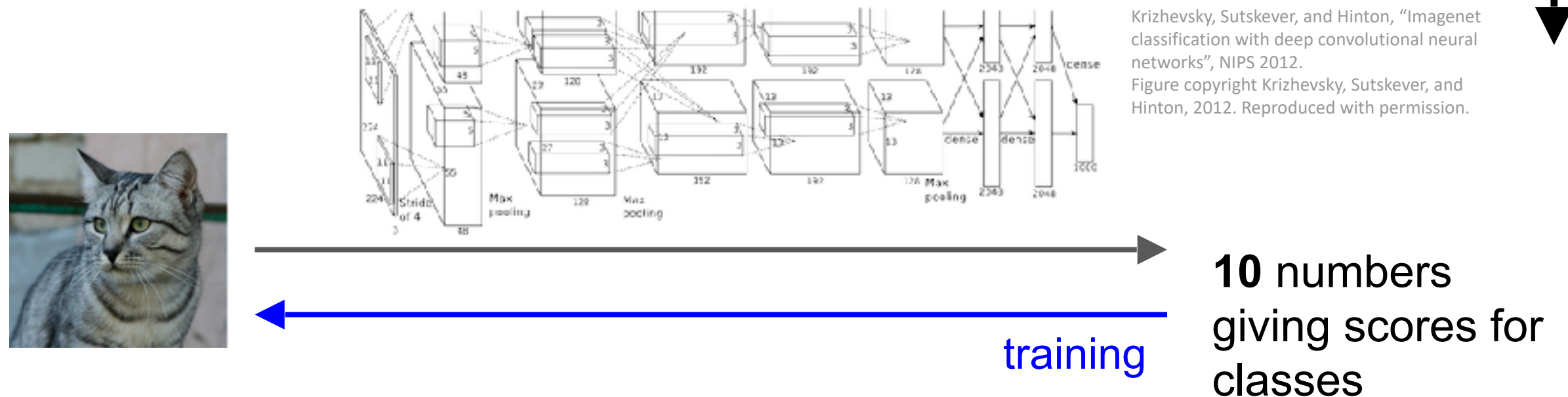
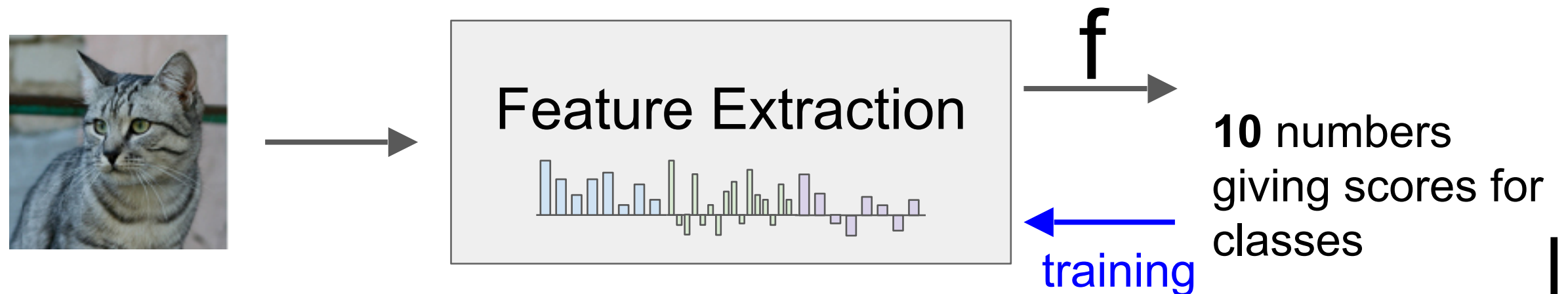
Naïve Bayes



Support Vector Machine

Deep classifier

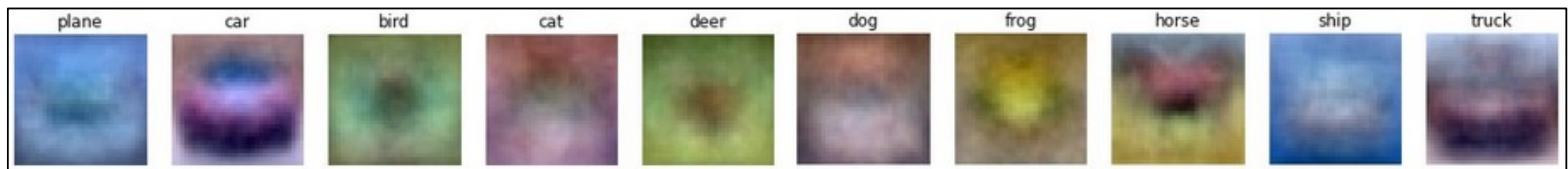
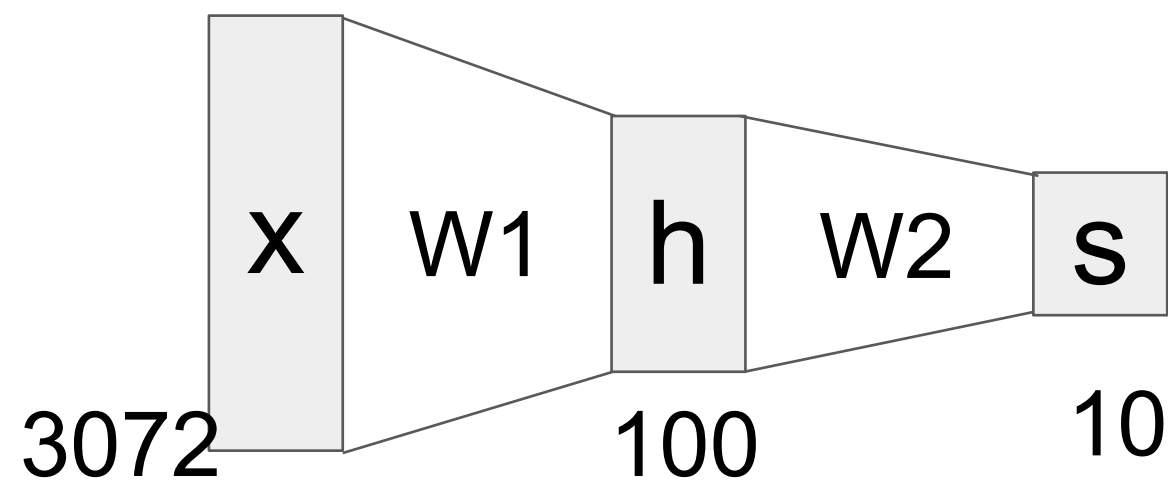
Image features vs ConvNets



Multi-layered networks

(**Before**) Linear score function: $f = Wx$

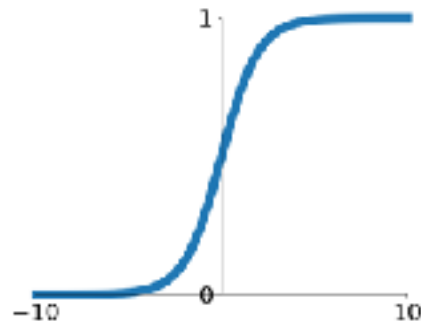
(**Now**) 2-layer Neural Network $f = W_2 \max(0, W_1 x)$



Activation functions

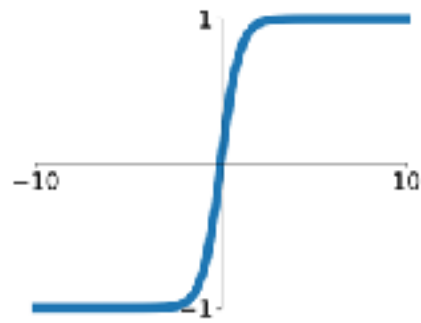
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



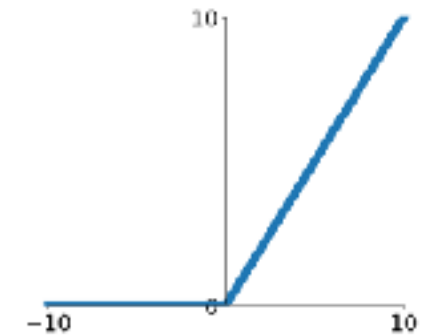
tanh

$$\tanh(x)$$



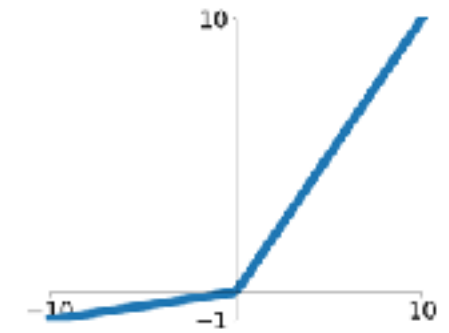
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

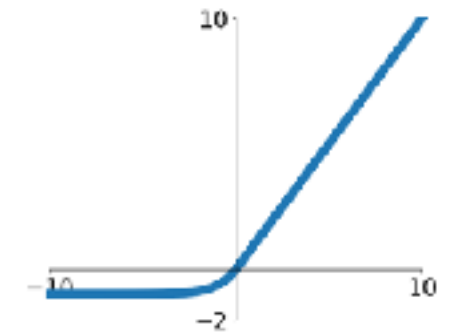


Maxout

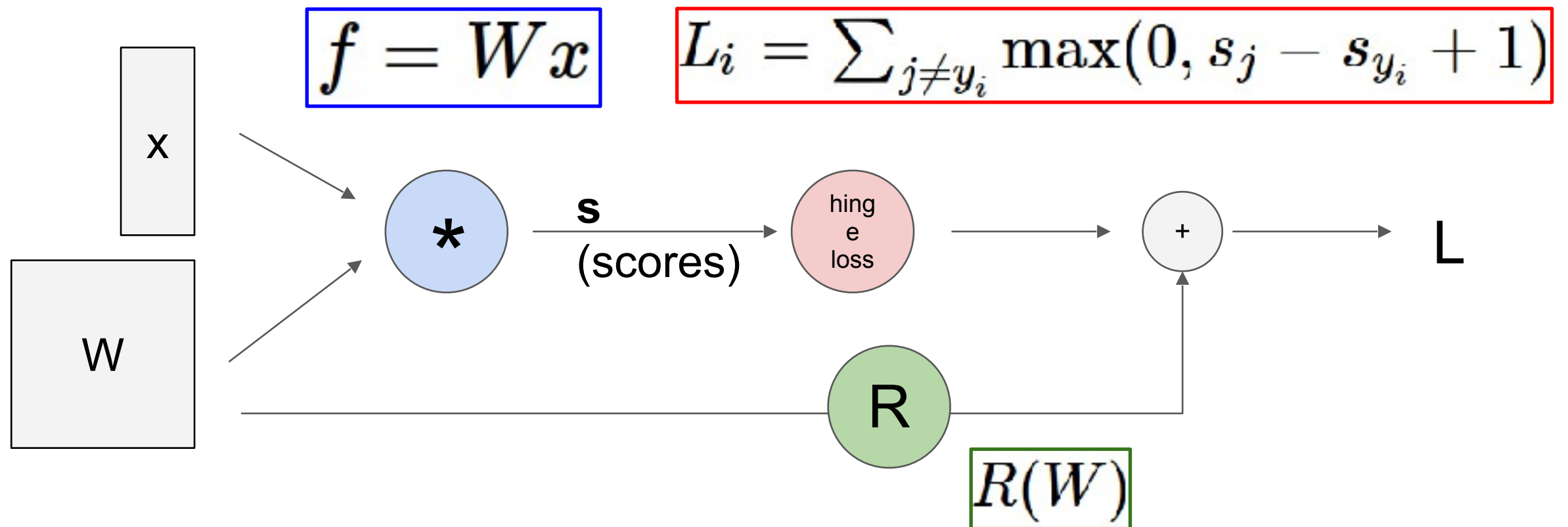
$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

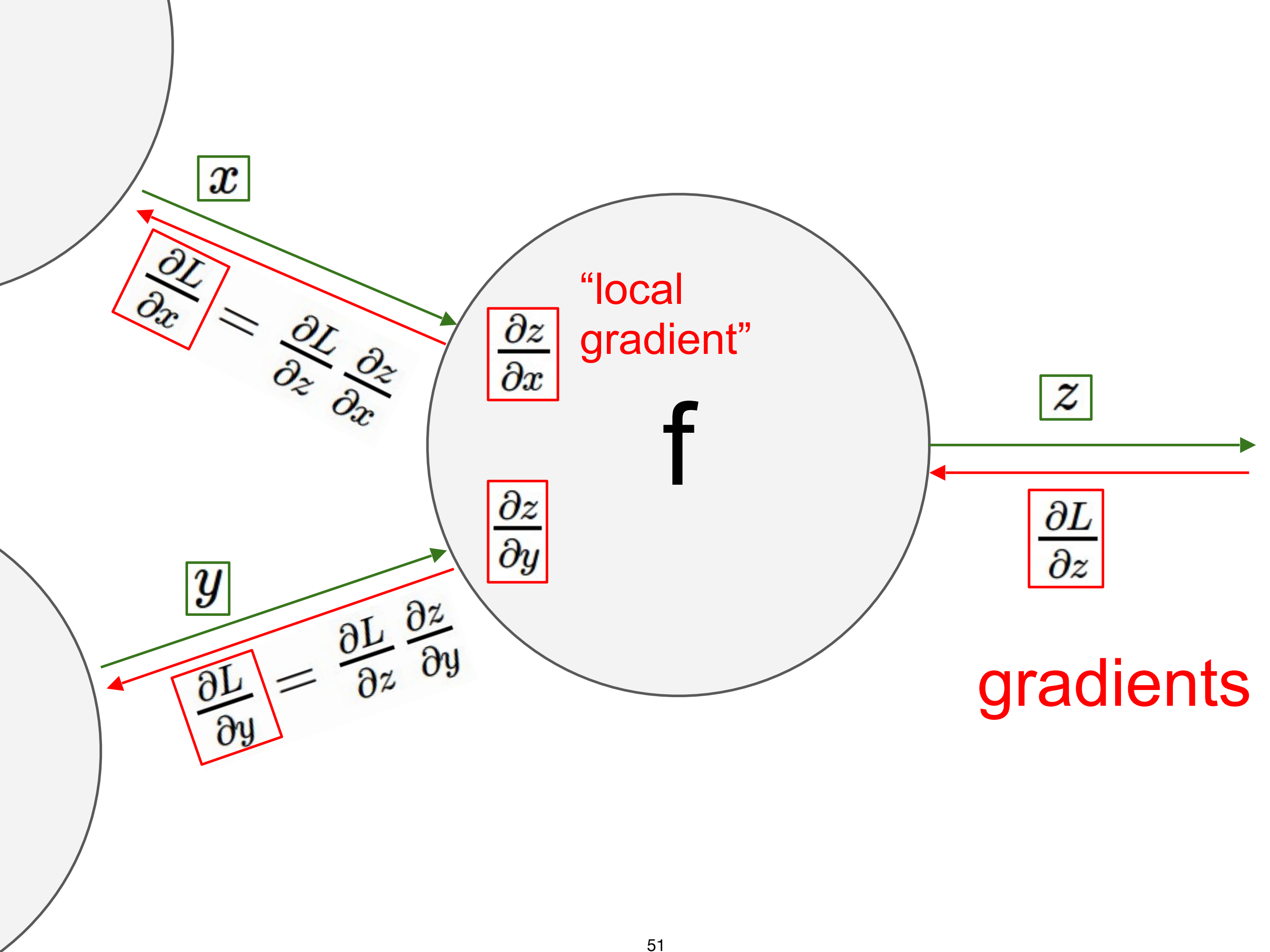
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



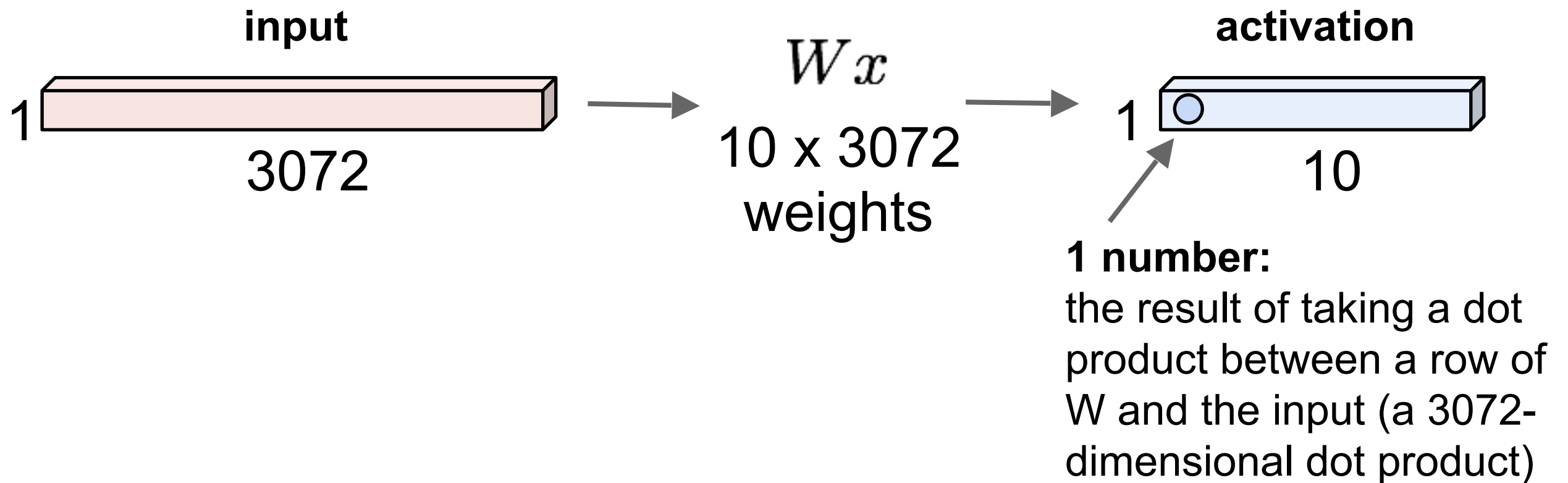
Computational graphs



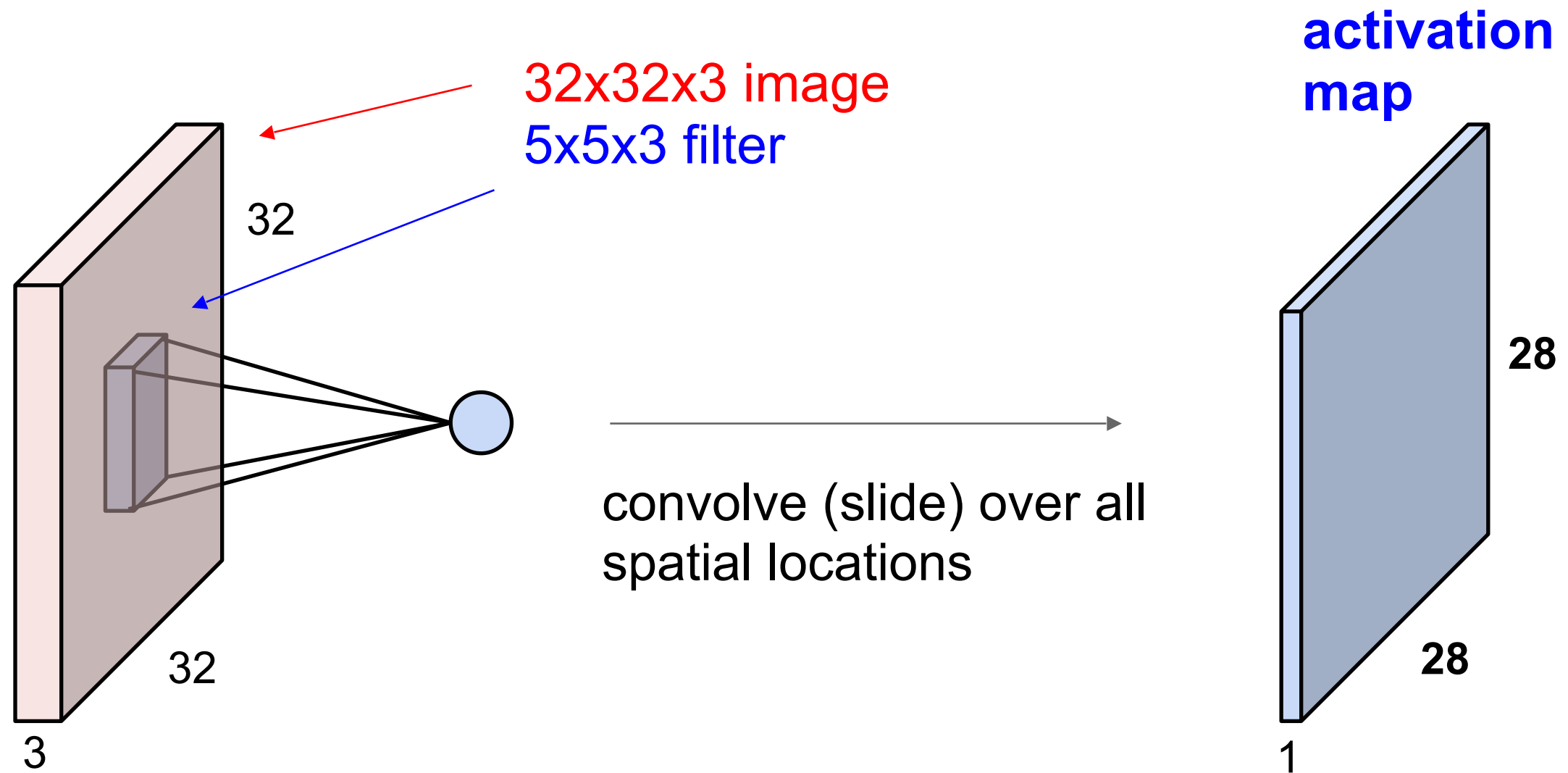


Fully connected layer

32x32x3 image -> stretch to 3072 x 1

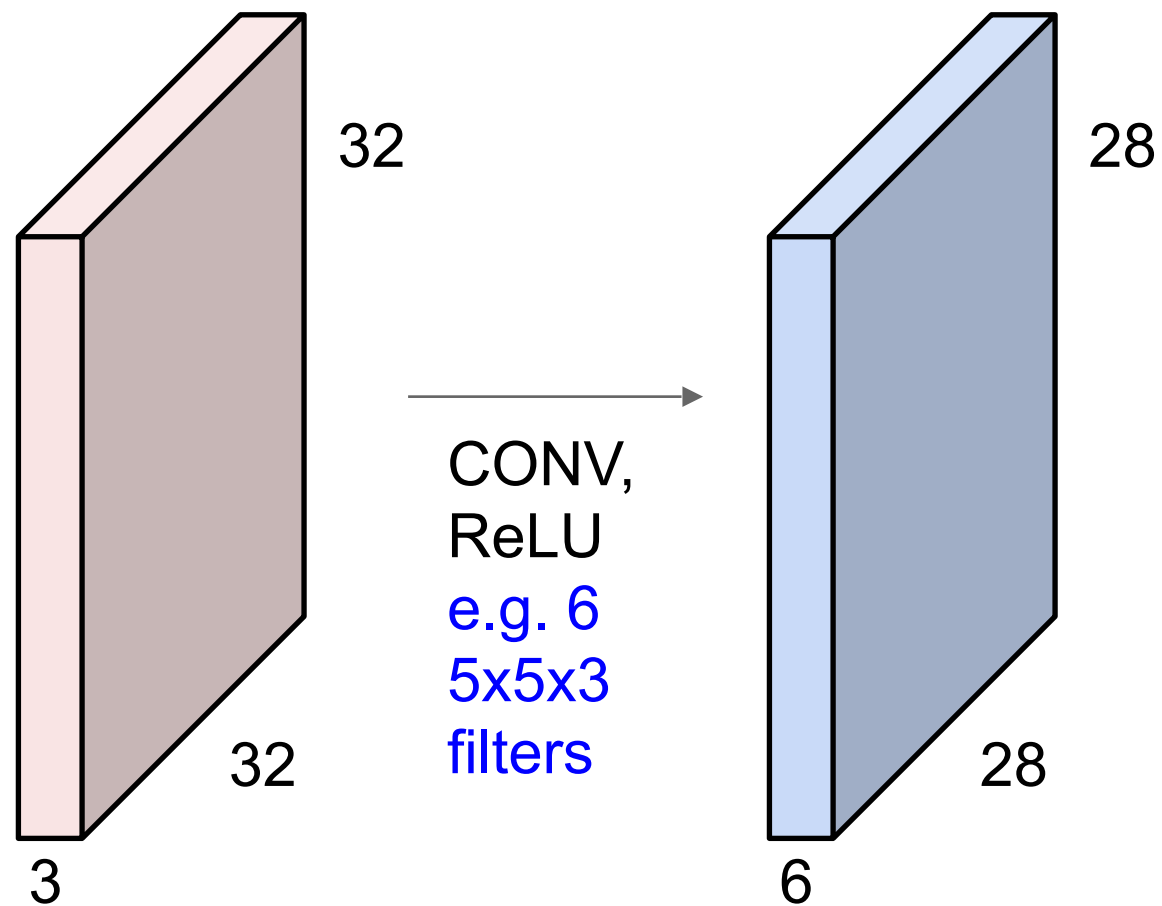


Convolutional network



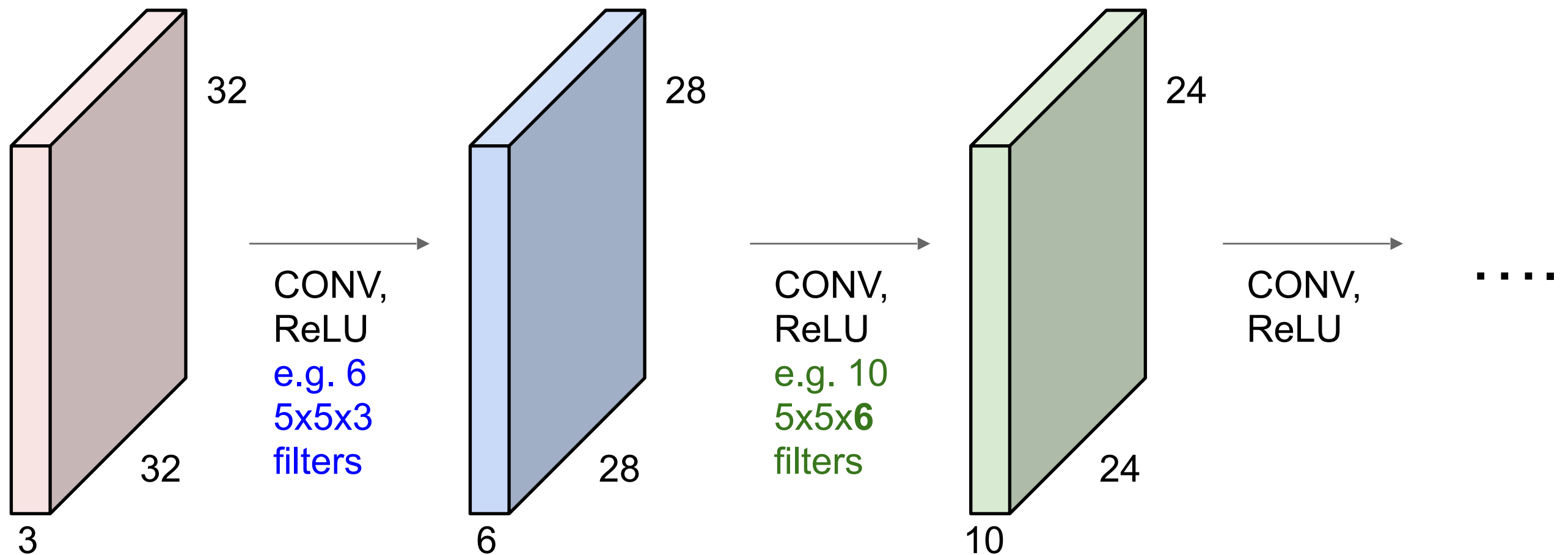
Convolutional network

sequence of Convolutional Layers, interspersed with activation functions



Convolutional network

sequence of Convolutional Layers, interspersed with activation functions

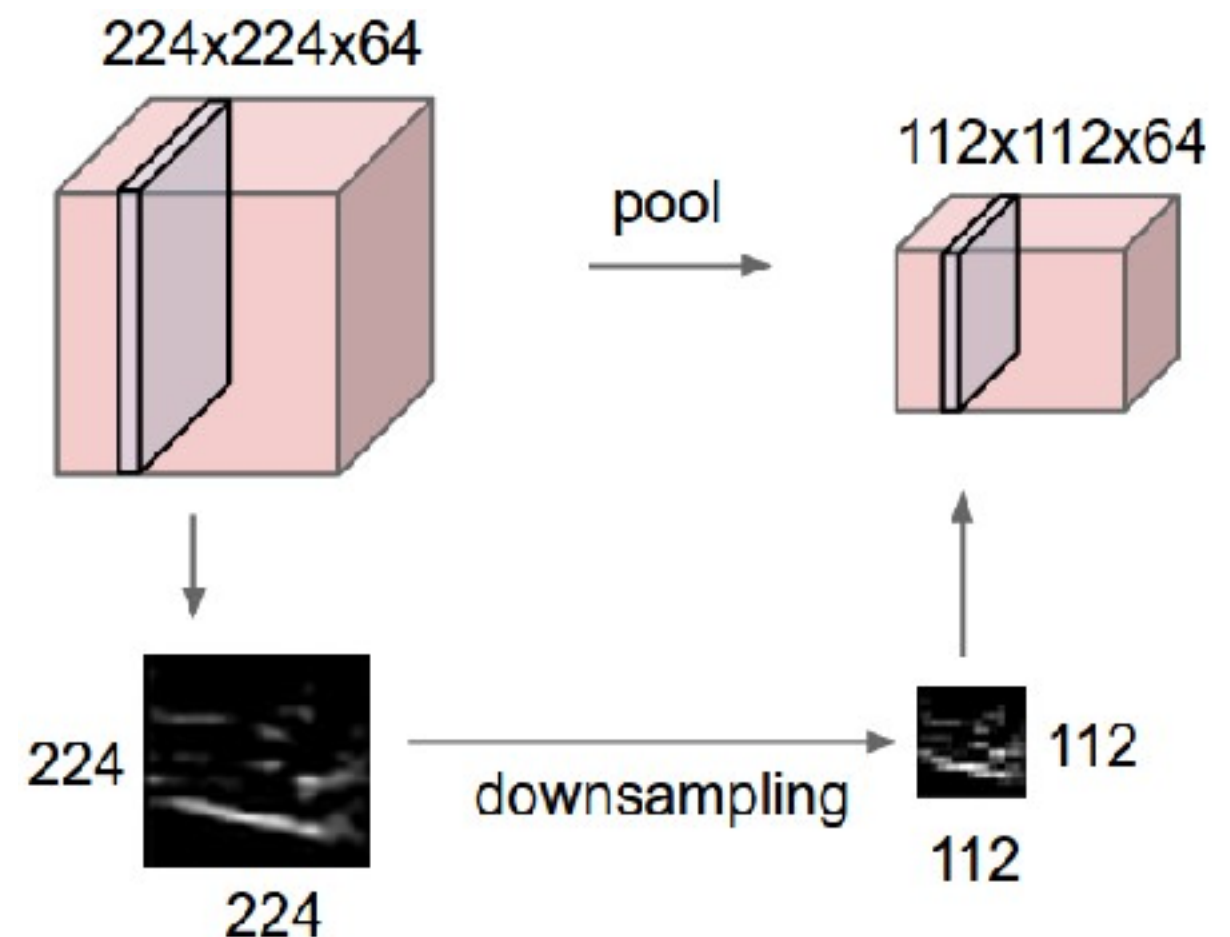


Summary. To summarize, the Conv Layer:

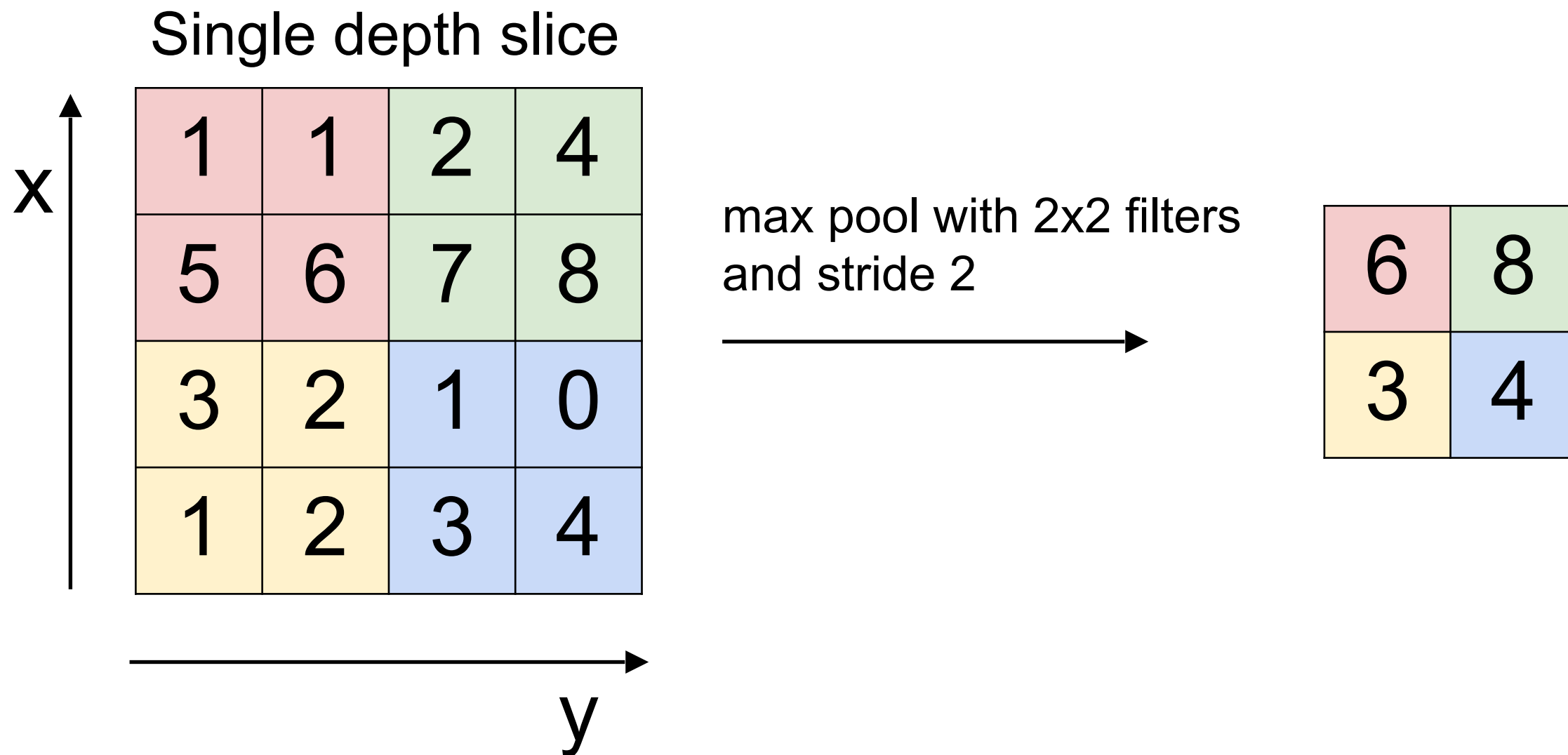
- Accepts a volume of size $W_1 \times H_1 \times D_1$
- Requires four hyperparameters:
 - Number of filters K ,
 - their spatial extent F ,
 - the stride S ,
 - the amount of zero padding P .
- Produces a volume of size $W_2 \times H_2 \times D_2$ where:
 - $W_2 = (W_1 - F + 2P)/S + 1$
 - $H_2 = (H_1 - F + 2P)/S + 1$ (i.e. width and height are computed equally by symmetry)
 - $D_2 = K$
- With parameter sharing, it introduces $F \cdot F \cdot D_1$ weights per filter, for a total of $(F \cdot F \cdot D_1) \cdot K$ weights and K biases.
- In the output volume, the d -th depth slice (of size $W_2 \times H_2$) is the result of performing a valid convolution of the d -th filter over the input volume with a stride of S , and then offset by d -th bias.

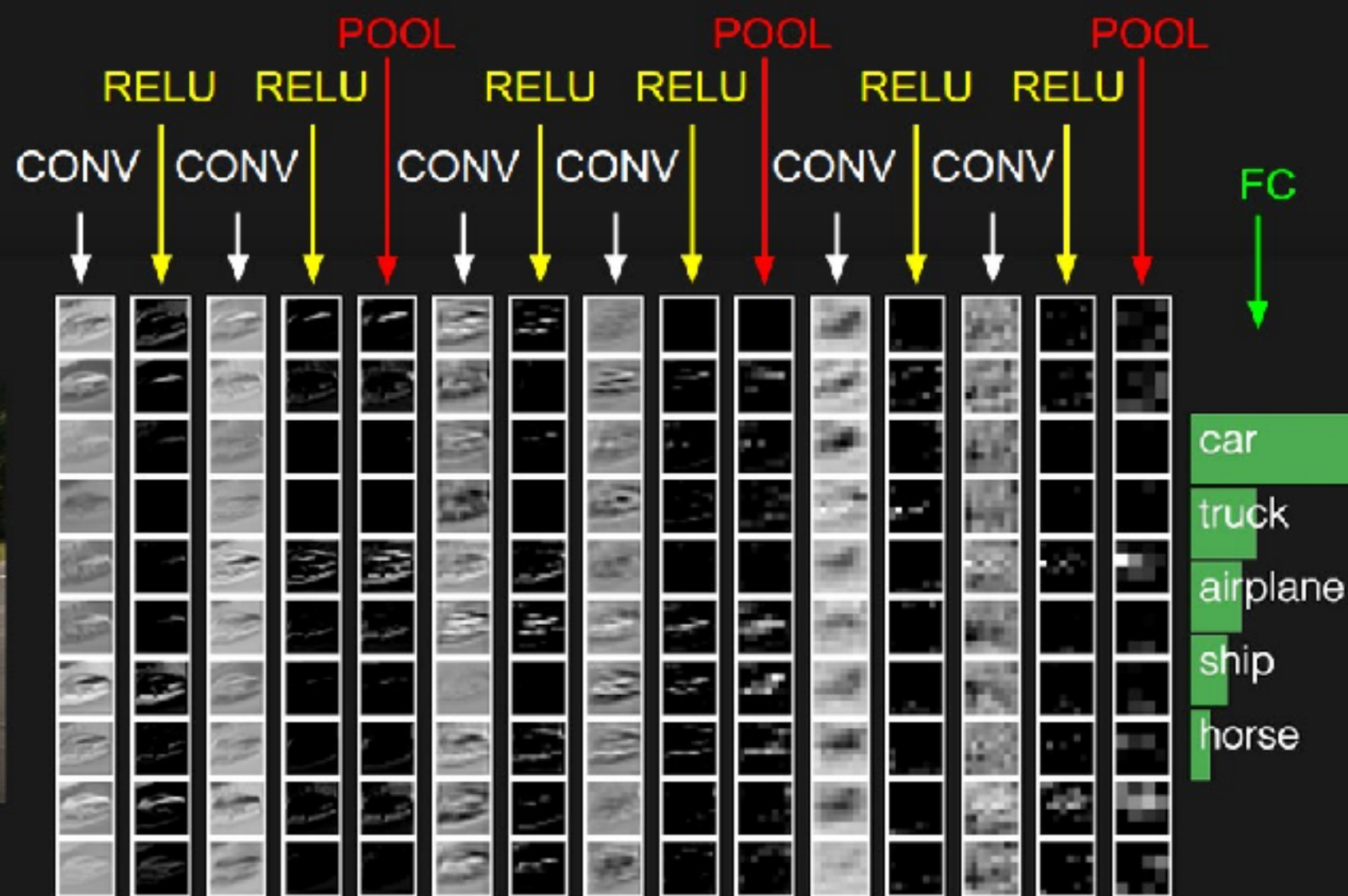
Pooling layers

- Reduce spatial resolution
-



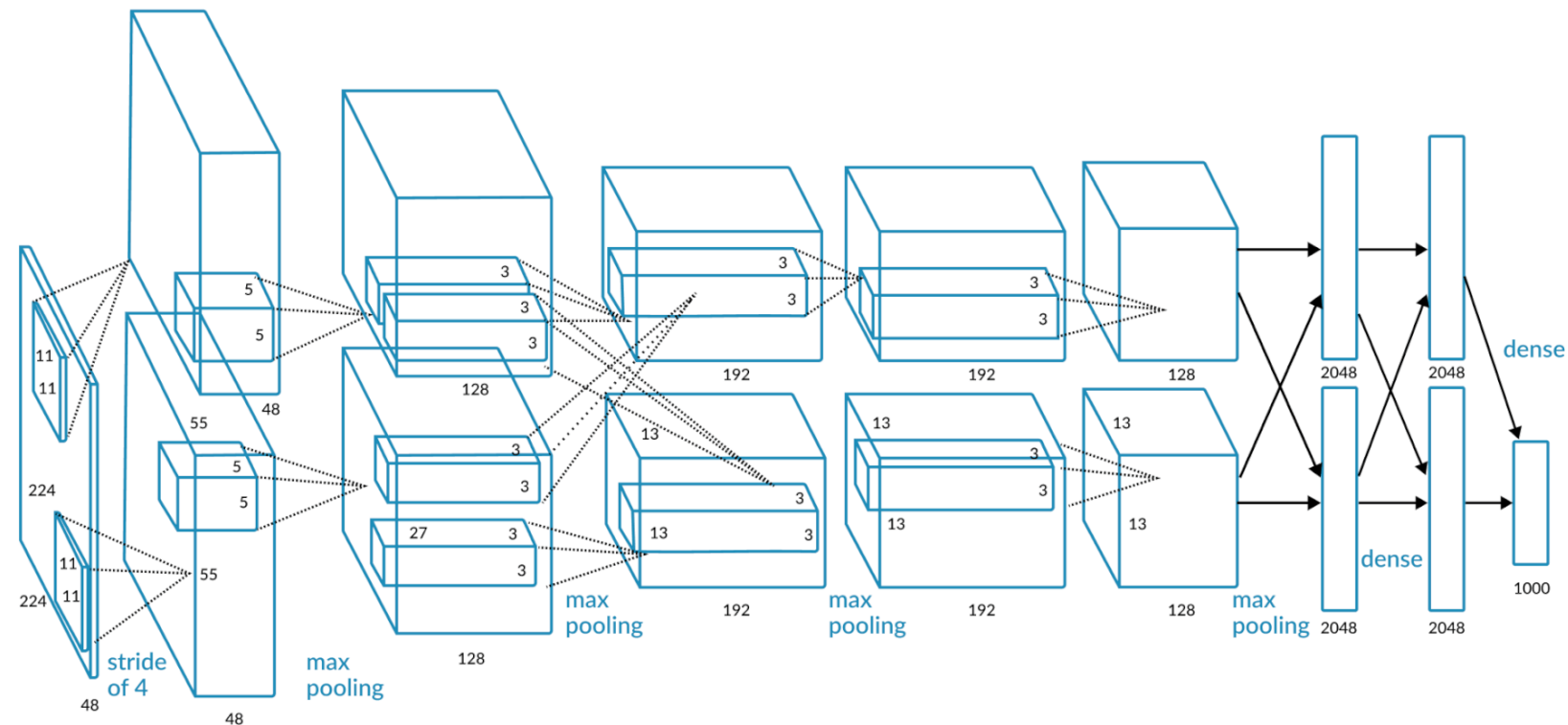
Example: Max pooling





Quick recap: Famous CNN architectures

AlexNet (Krizhevsky et al., 2012)

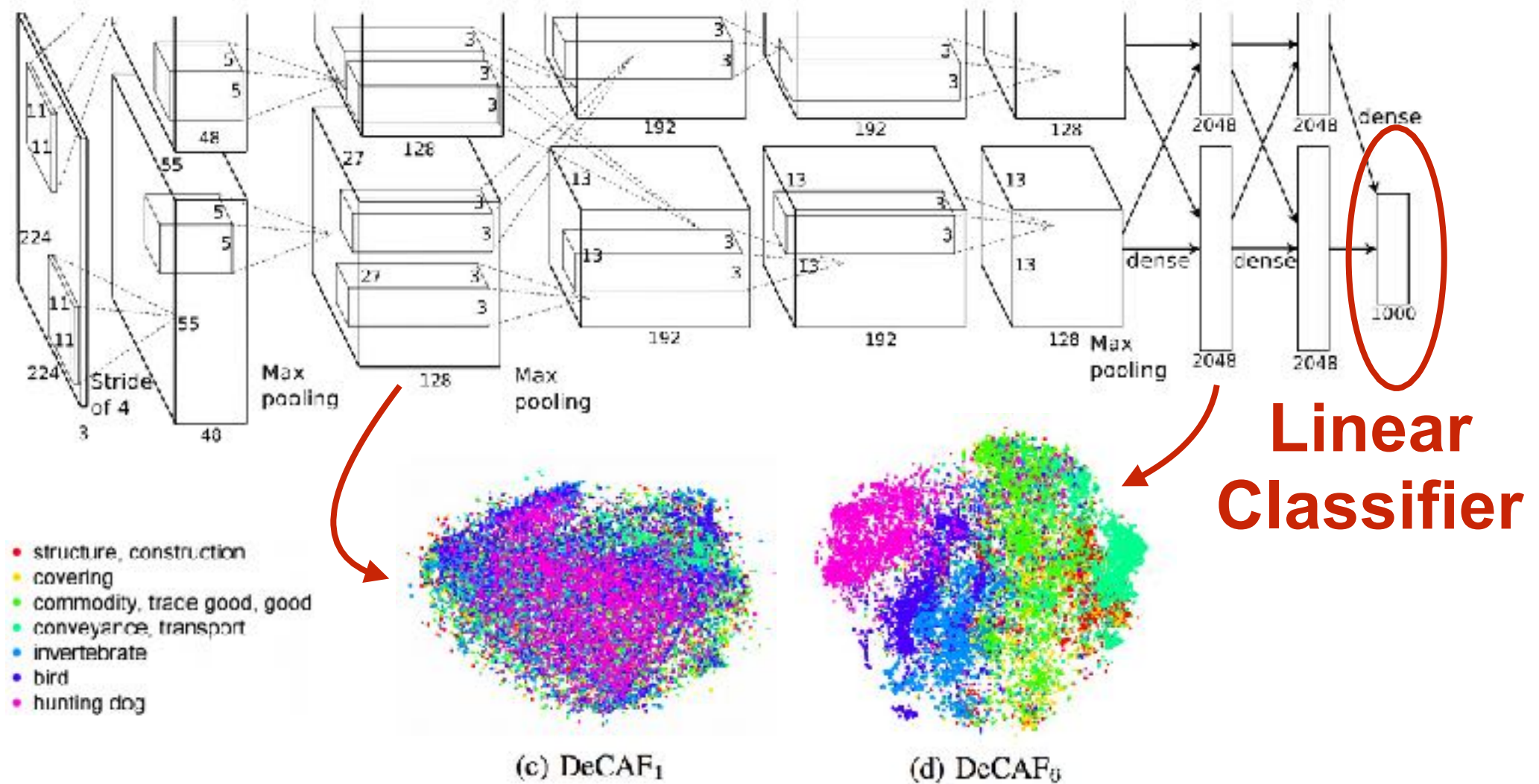


AlexNet:

- ▶ 8 layers, ReLUs, dropout, data augmentation, trained on 2 GTX 580 GPUs
- ▶ Number of feature channels increase with depth, spatial resolution decreases
- ▶ Triggered deep learning revolution, showed that CNNs work well in practice

Krizhevsky, Sutskever and Hinton: ImageNet Classification with Deep Convolutional Neural Networks. NIPS, 2012.

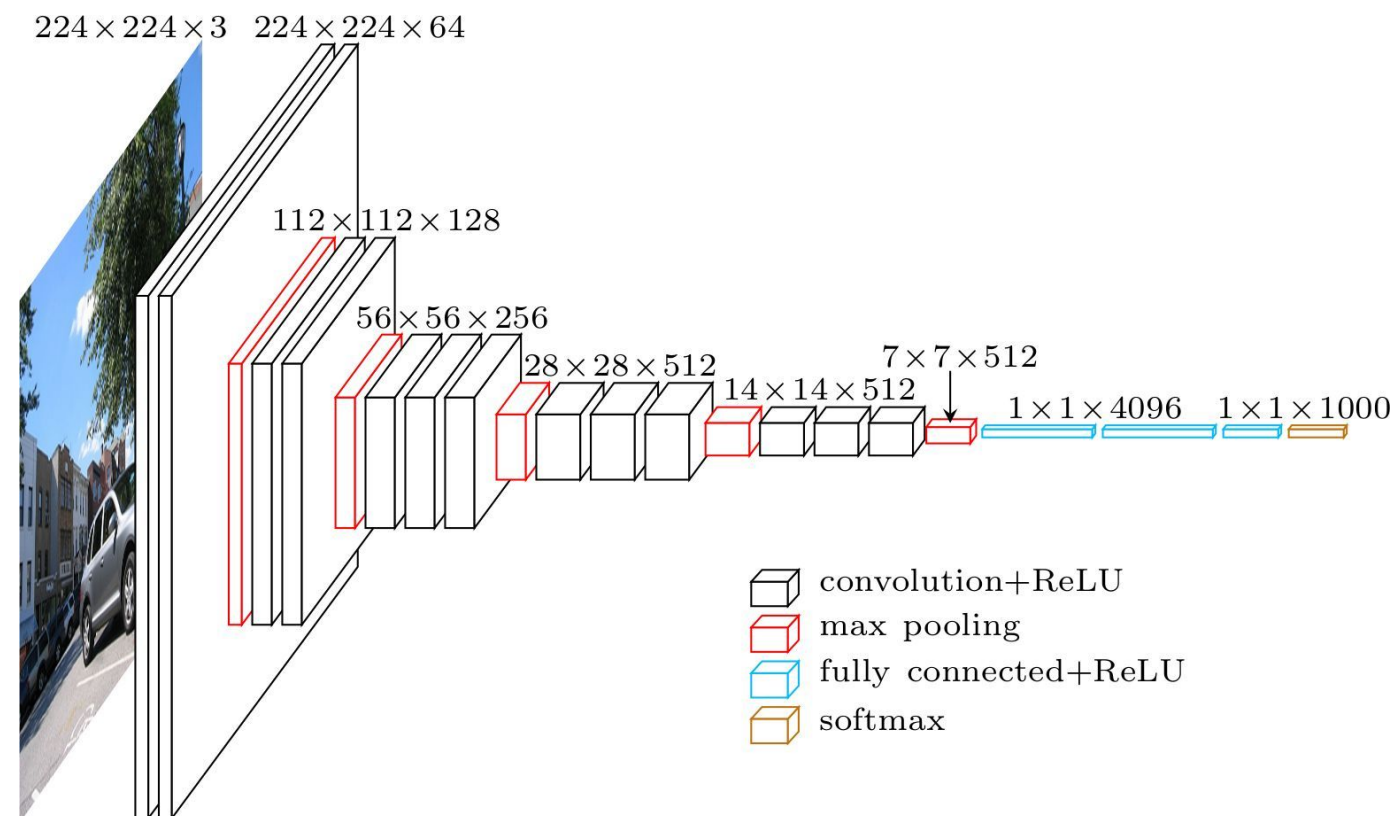
AlexNet (Krizhevsky et al., 2012)



(2D visualization
using t-SNE)

[Donahue, "DeCAF: DeCAF: A Deep Convolutional
...", arXiv 2013]

VGG Net (Simonyan et al, 2015)

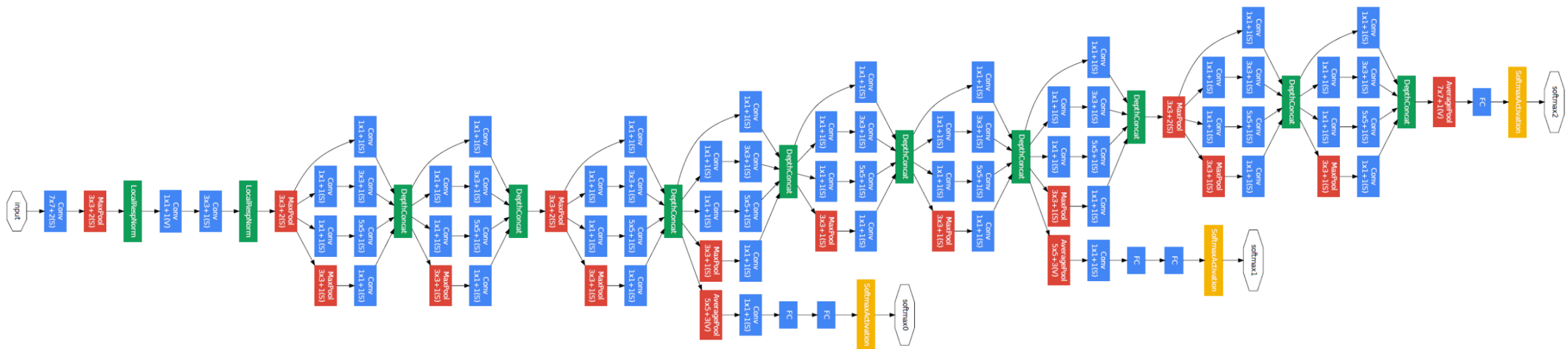


VGG:

- Uses 3×3 convolutions everywhere (same expressiveness, fewer parameters)
- Three 3×3 layers: same receptive field as one 7×7 layer, but less parameters
- 2 Variants: 16 and 19 Layers. Showed that deeper networks are better.

Simonyan and Zisserman: Very Deep Convolutional Networks for Large-Scale Image Recognition. ICLR, 2015.

GoogLeNet (Szegedy et al., 2015)

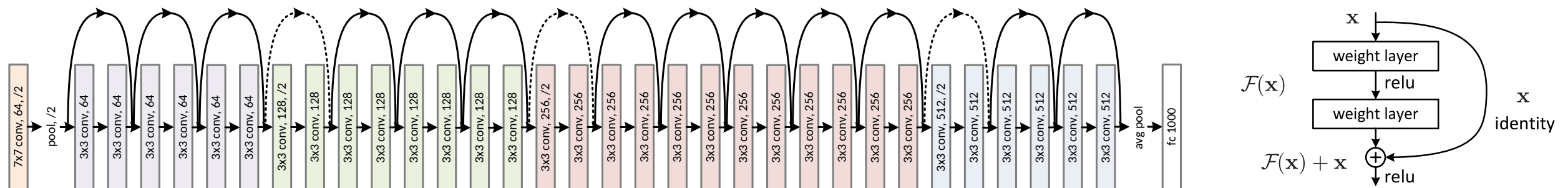


Inception:

- ▶ 22 layers. Inception modules utilize conv/pool operations with varying filter size
- ▶ Multiple intermediate classification heads to improve gradient flow
- ▶ Global average pooling (no FC layers), 27x less parameters (5 million) than VGG-16

Szegedy et al.: Going deeper with convolutions. CVPR, 2015.

ResNet (He et al., 2016)



ResNet:

- ▶ Residual connections allow for training deeper networks (up to 152 layers)
- ▶ Very simple and regular network structure with 3×3 convolutions
- ▶ Uses strided convolutions for downsampling
- ▶ ResNet and ResNet-like architectures are dominating today

He, Zhang, Ren and Sun: Deep Residual Learning for Image Recognition. CVPR, 2016.

Overview: Recognition tasks

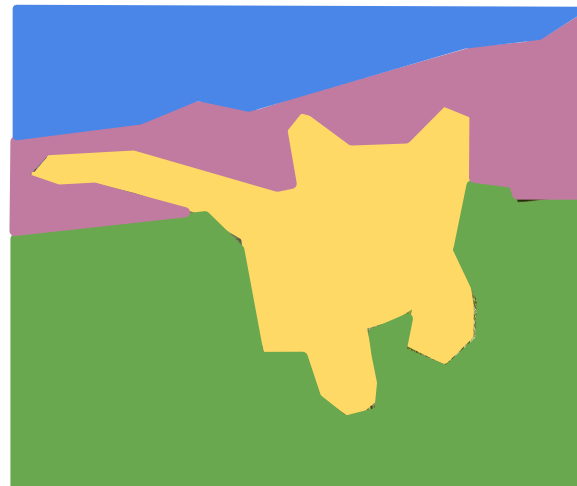
Classification



CAT

No spatial extent

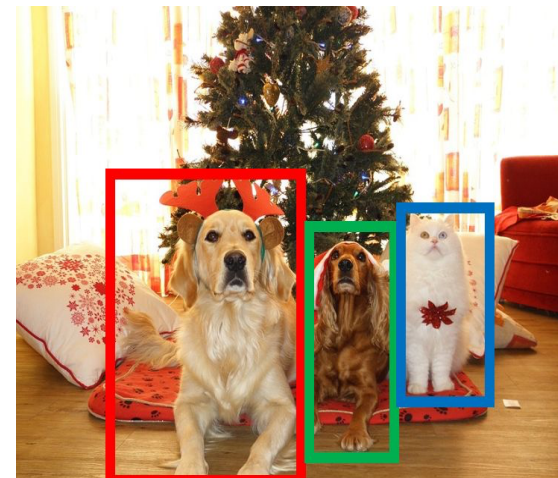
Semantic Segmentation



GRASS, CAT, TREE, SKY

No objects, just pixels

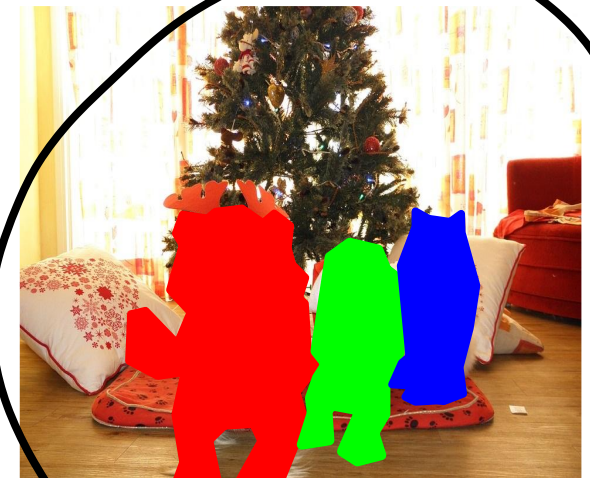
Object Detection



DOG, DOG, CAT

Multiple Object

Instance Segmentation

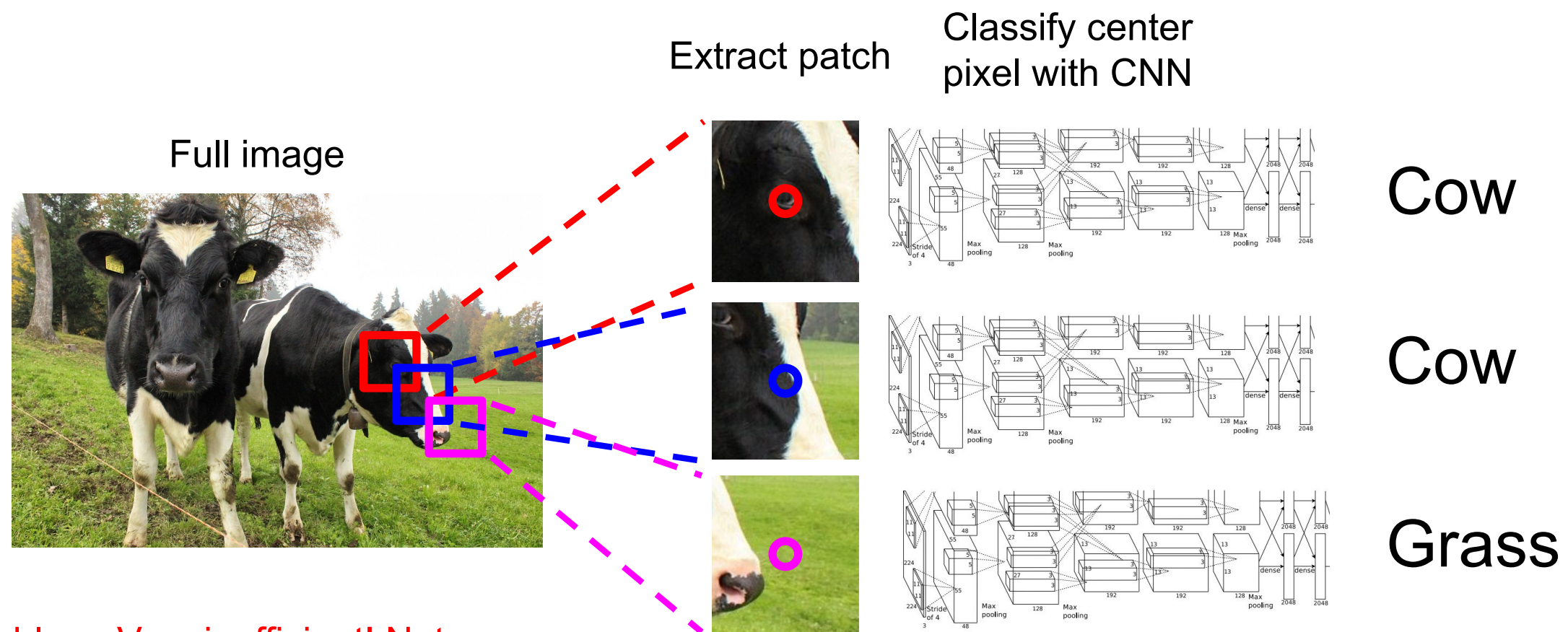


DOG, DOG, CAT

[This image is CC0 public domain](#)

Semantic segmentation

Pixel-wise classification with CNN

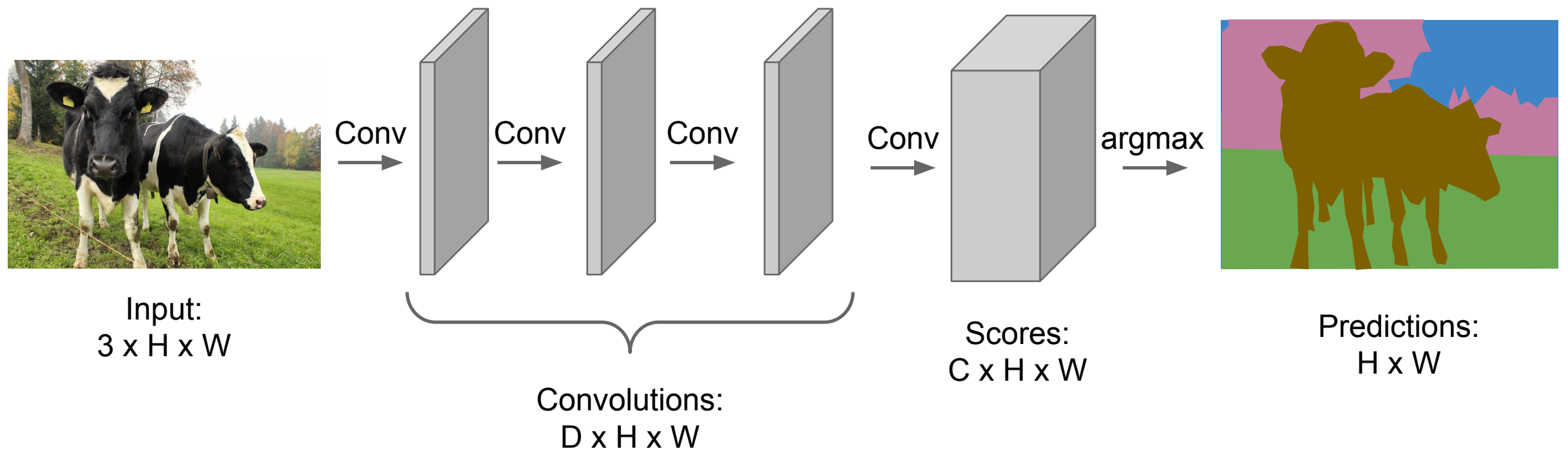


Problem: Very inefficient! Not reusing shared features between overlapping patches

Farabet et al, "Learning Hierarchical Features for Scene Labeling," TPAMI 2013
Pinheiro and Collobert, "Recurrent Convolutional Neural Networks for Scene Labeling", ICML 2014

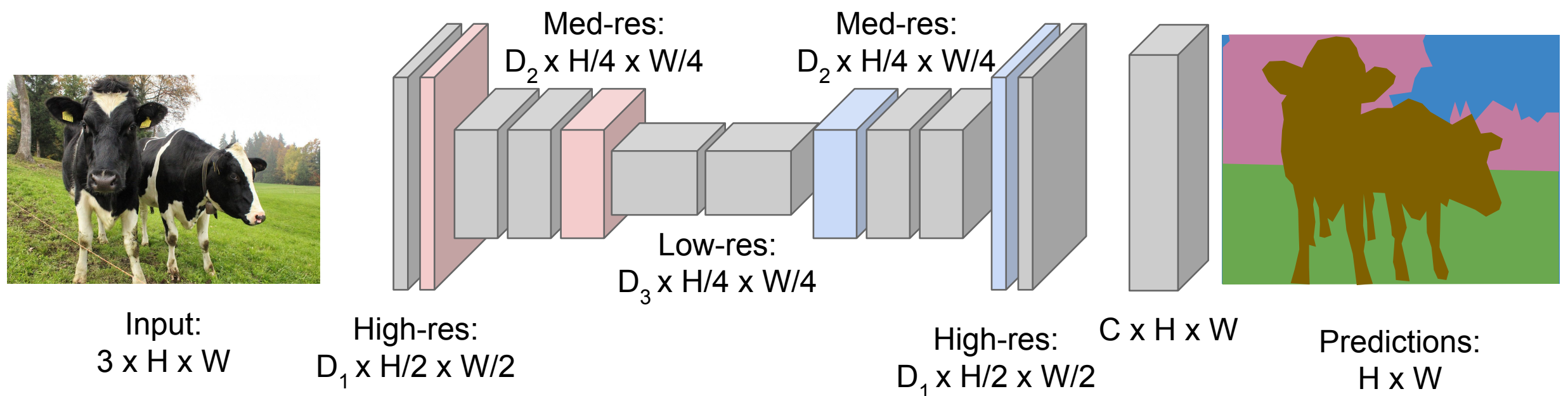
Fully convolutional

Design a network with only convolutional layers without downsampling operators to make predictions for pixels all at once!



Full convolutional with lower resolution feature maps

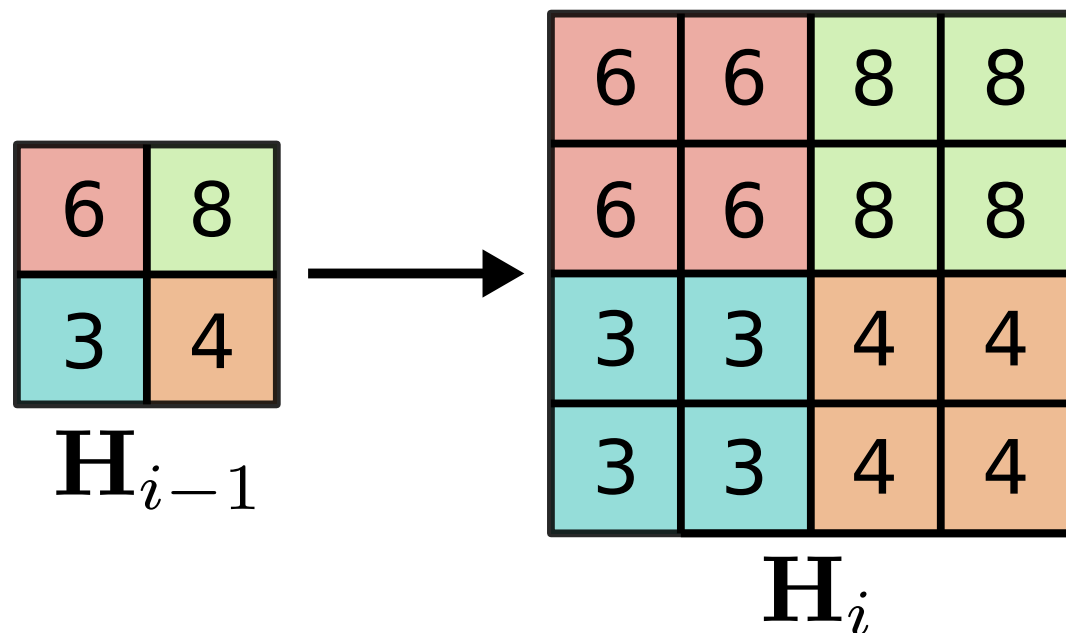
Design network as a bunch of convolutional layers, with **downsampling** and **upsampling** inside the network!



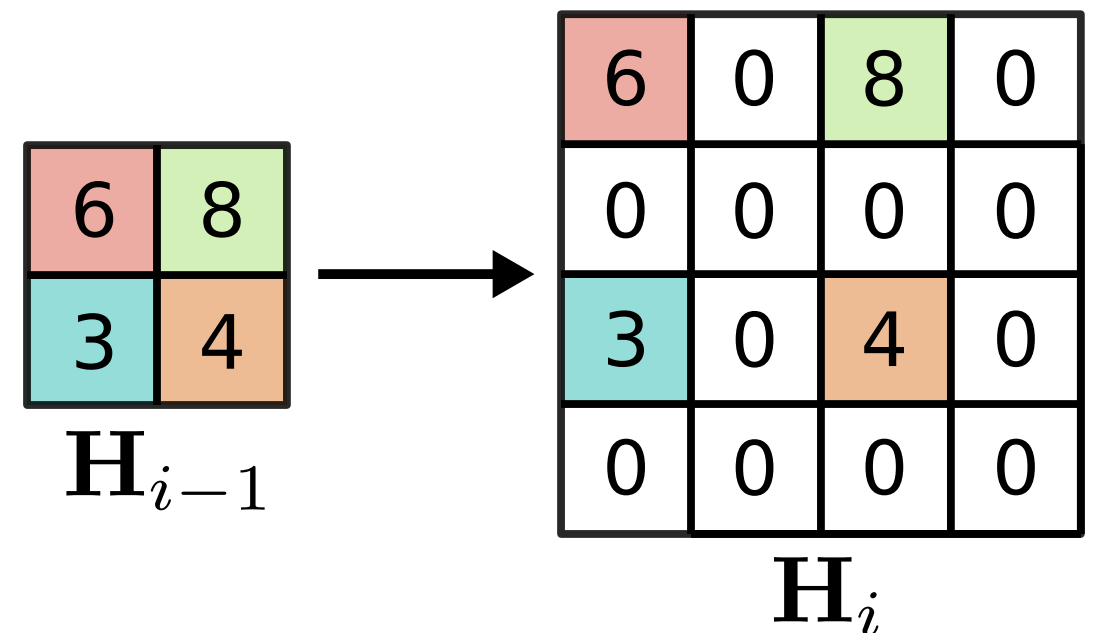
Long, Shelhamer, and Darrell, "Fully Convolutional Networks for Semantic Segmentation", CVPR 2015
Noh et al, "Learning Deconvolution Network for Semantic Segmentation", ICCV 2015

Upsampling

Nearest Neighbor Upsampling



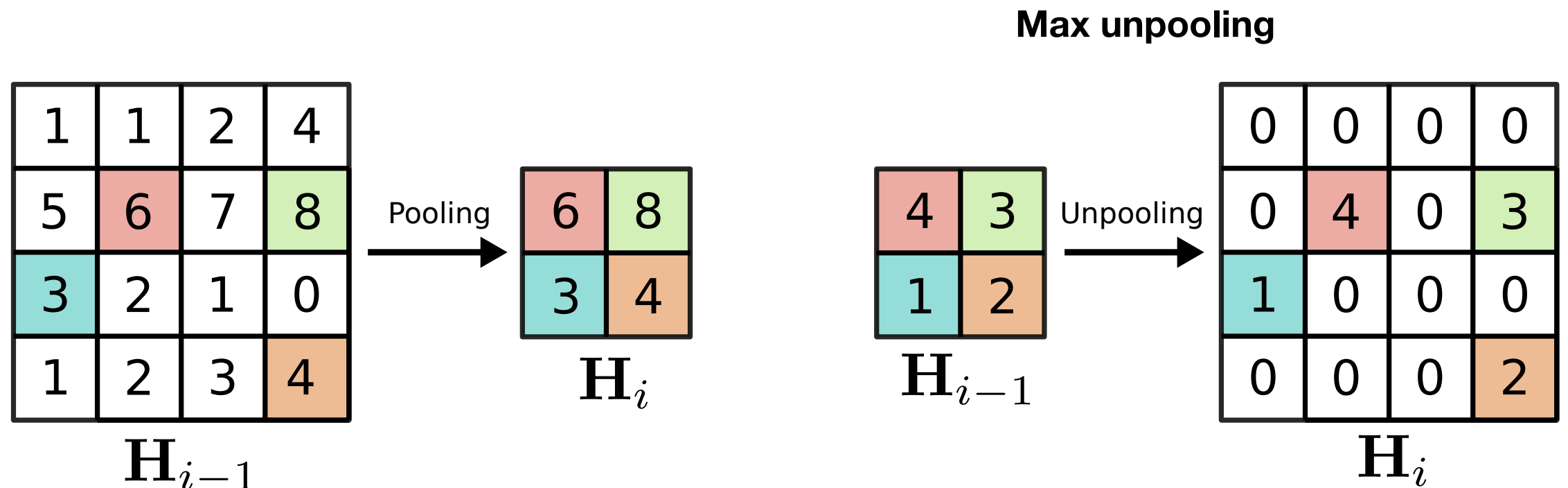
Bed of Nails Upsampling



- **Nearest neighbor:** scale each channel using nearest neighbor interpolation
- **Bilinear:** scale each channel using bilinear neighbor interpolation
- **Bed of nails:** insert elements at sparse location (followed by convolution)

Badrinarayanan, Kendall and Cipolla: SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation. TPAMI, 2017.

Upsampling

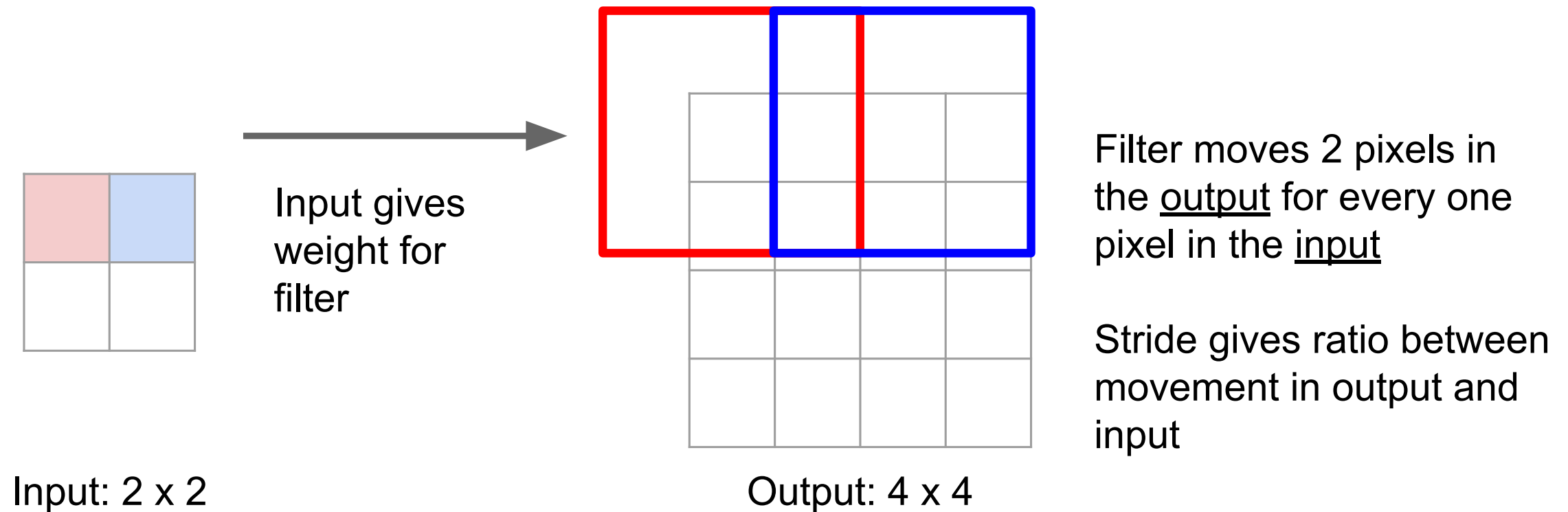


- For unpooling, **remember** which element was **maximum** during pooling
- Requires corresponding pairs of downsampling and upsampling layers
- This approach has been used in SegNet

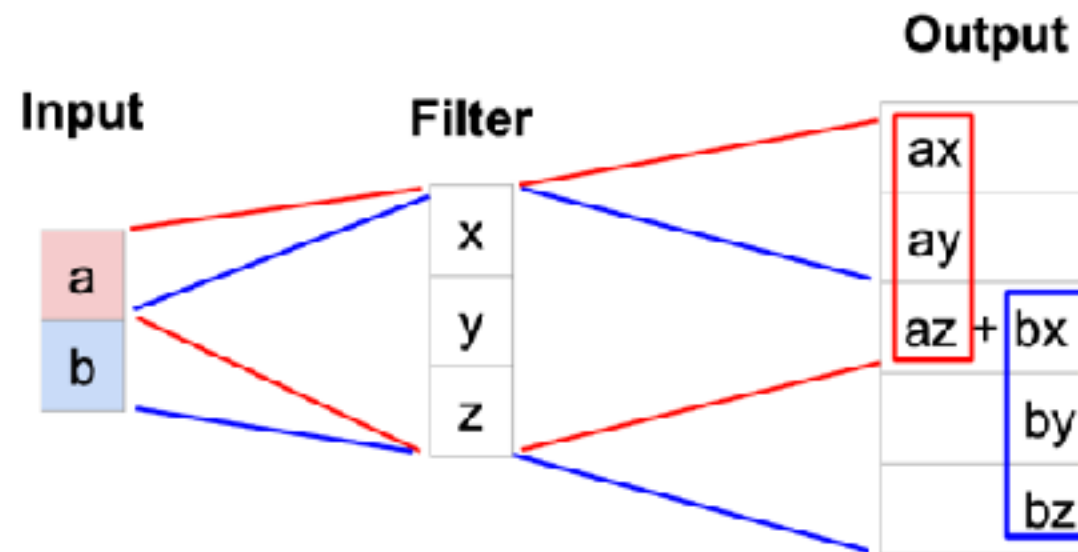
Badrinarayanan, Kendall and Cipolla: SegNet: A Deep Convolutional Encoder-Decoder Architecture for Image Segmentation. TPAMI, 2017.

Upsampling

3 x 3 **transposed** convolution, stride 2 pad 1



Upsampling: 1D example



We can express convolution in terms of a matrix multiplication

$$\vec{x} * \vec{a} = X \vec{a}$$

$$\begin{bmatrix} x & y & z & 0 & 0 & 0 \\ 0 & 0 & x & y & z & 0 \end{bmatrix} \begin{bmatrix} 0 \\ a \\ b \\ c \\ d \\ 0 \end{bmatrix} = \begin{bmatrix} ay + bz \\ bx + cy + dz \end{bmatrix}$$

Transposed convolution multiplies by the transpose of the same matrix:

$$\vec{x} *^T \vec{a} = X^T \vec{a}$$

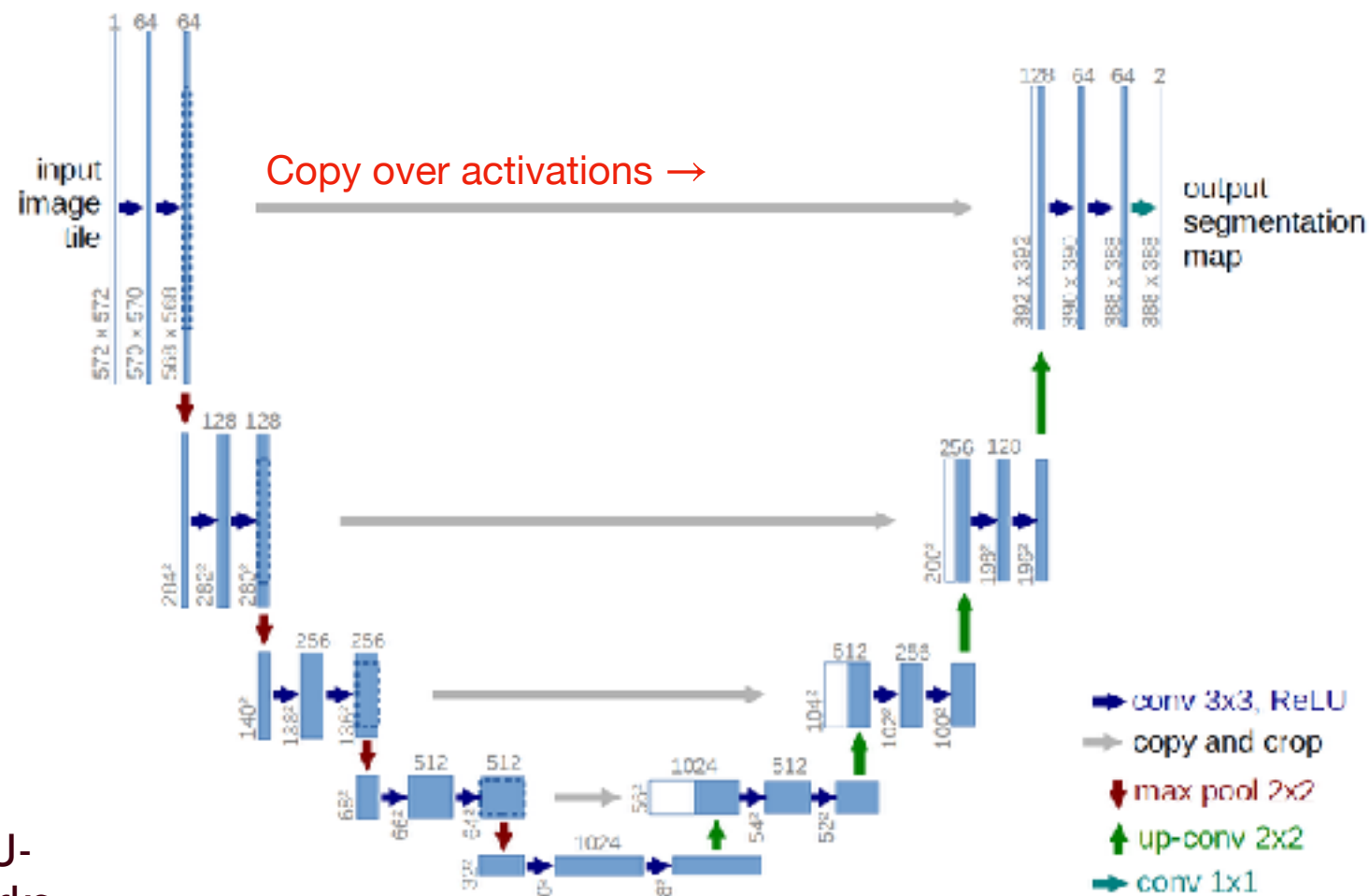
$$\begin{bmatrix} x & 0 \\ y & 0 \\ z & x \\ 0 & y \\ 0 & z \\ 0 & 0 \end{bmatrix} \begin{bmatrix} a \\ b \end{bmatrix} = \begin{bmatrix} ax \\ ay \\ az + bx \\ by \\ bz \\ 0 \end{bmatrix}$$

Skip connections for improved spatial detail

Basic scheme: max-pooling, up convolutions + skip connections



Input:
 $3 \times H \times W$



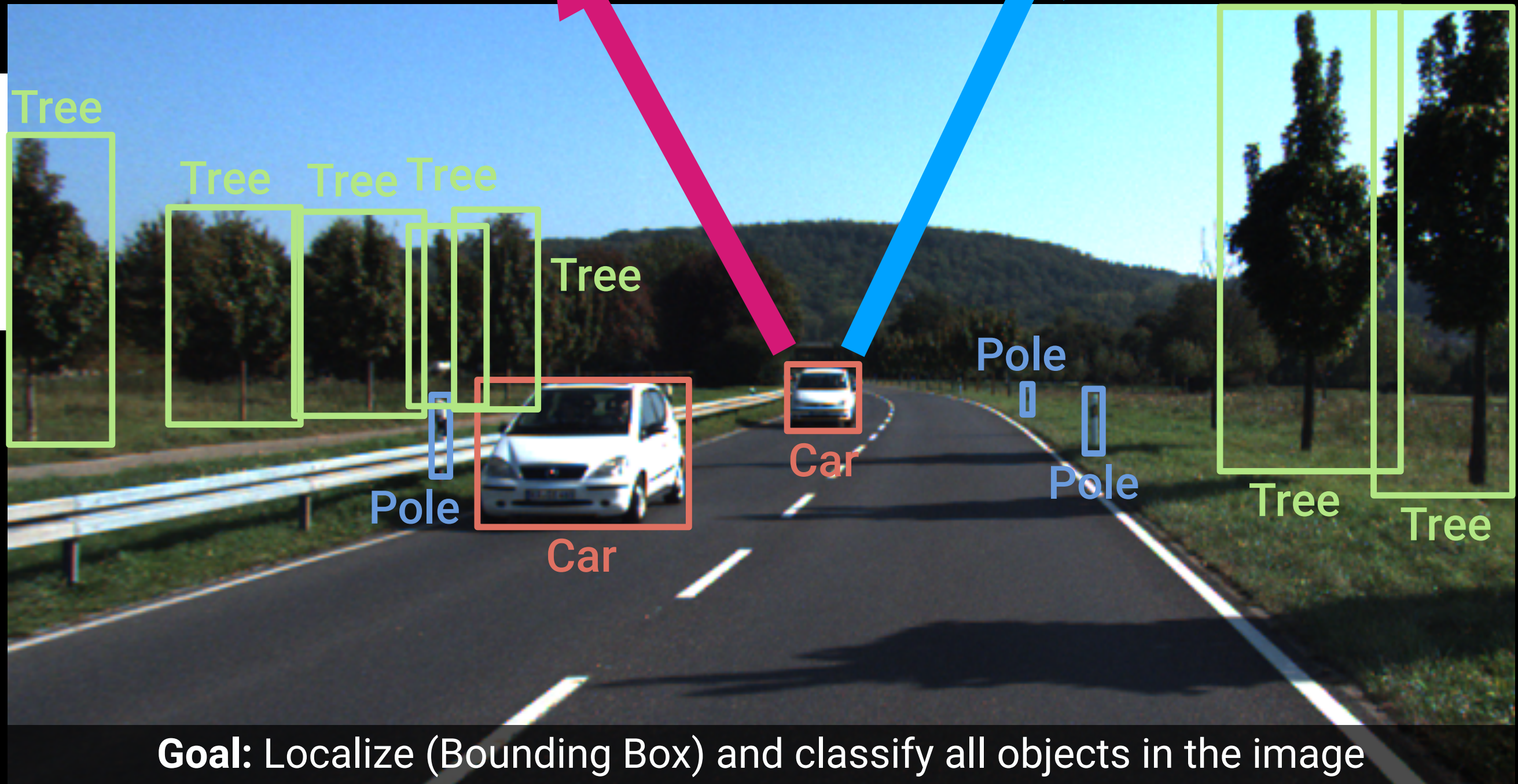
Predictions:
H x W

Ronneberger et al. 2015, U-Net: Convolutional Networks for Biomedical Image Segmentation

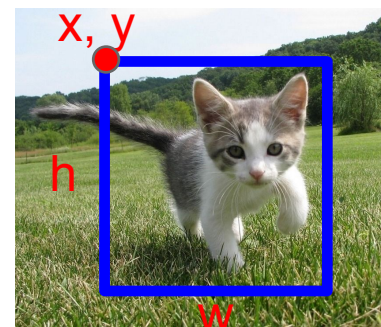
Object detection

Where?
[x, y, h, w]

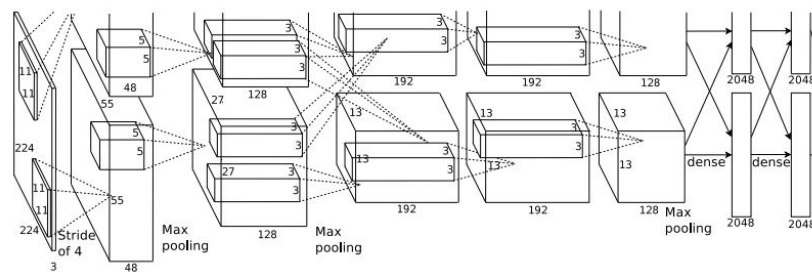
What?
„Car“



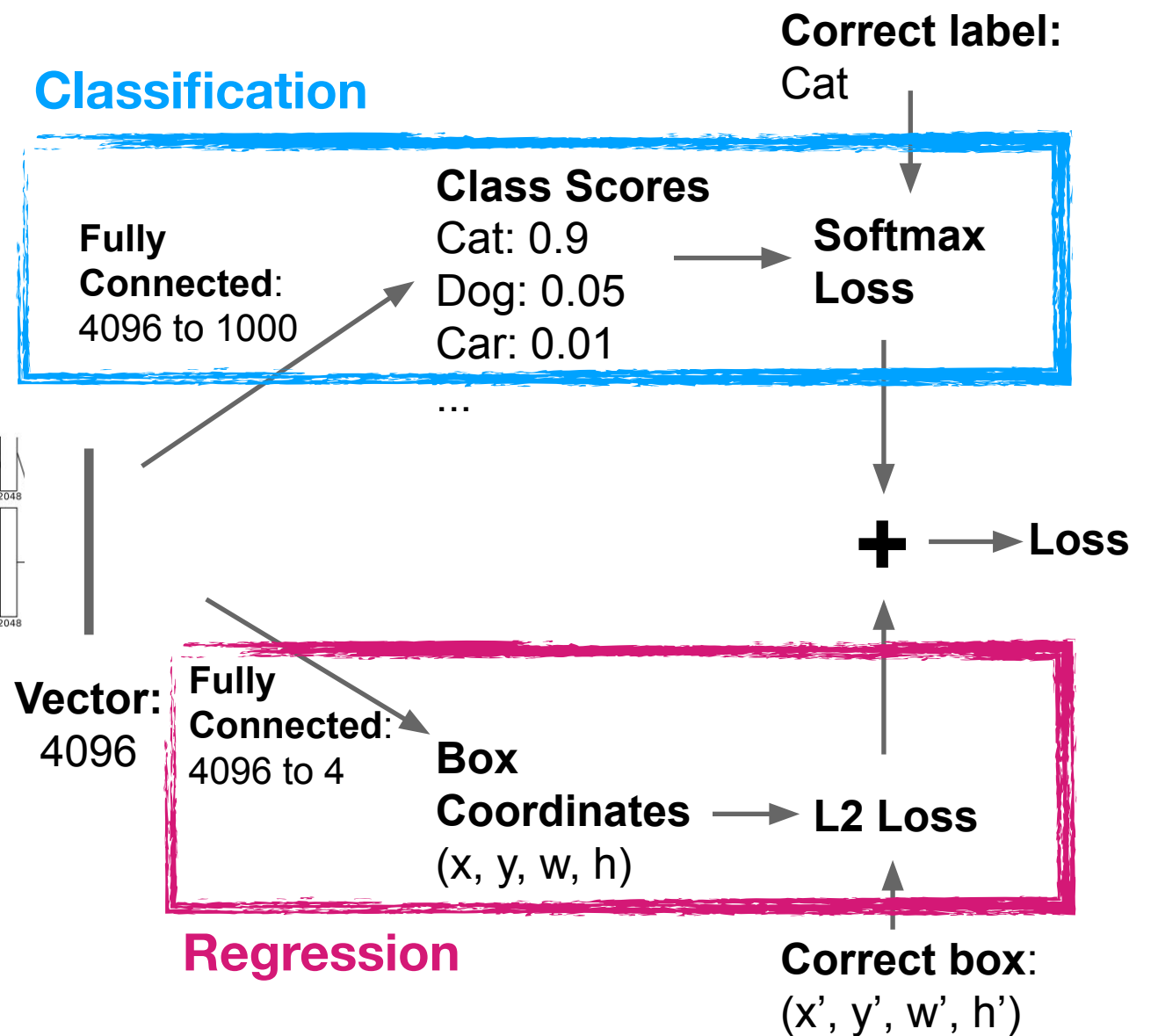
Idea: Separate network branches



[This image is CC0 public domain](#)



Classification

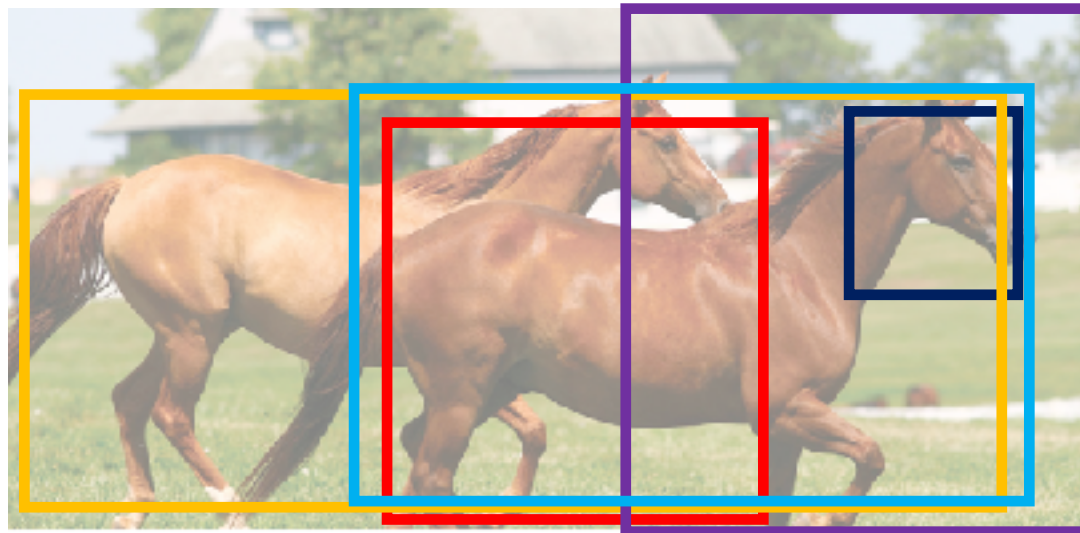




**Problem: Number of
objects and their size
not known a priori**

Why is detection hard(er)?

- Precise localization



Why is detection hard(er)?

- Much larger impact of pose



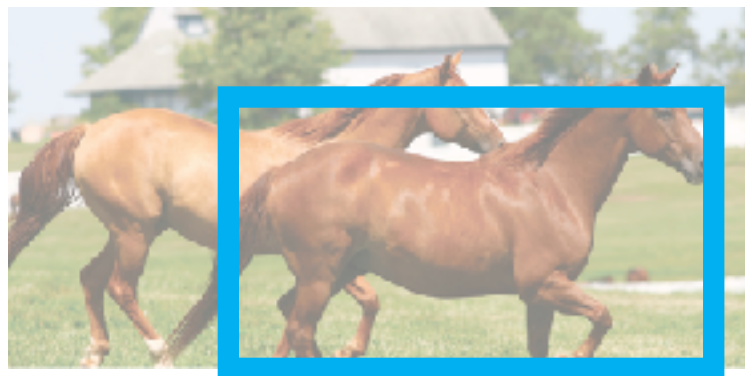
Why is detection hard(er)?

- Occlusion makes localization difficult



Why is detection hard(er)?

- Counting



Why is detection hard(er)?

- Small objects



Datasets



- Face detection
- One category: face
- Frontal faces
- Fairly rigid, unoccluded

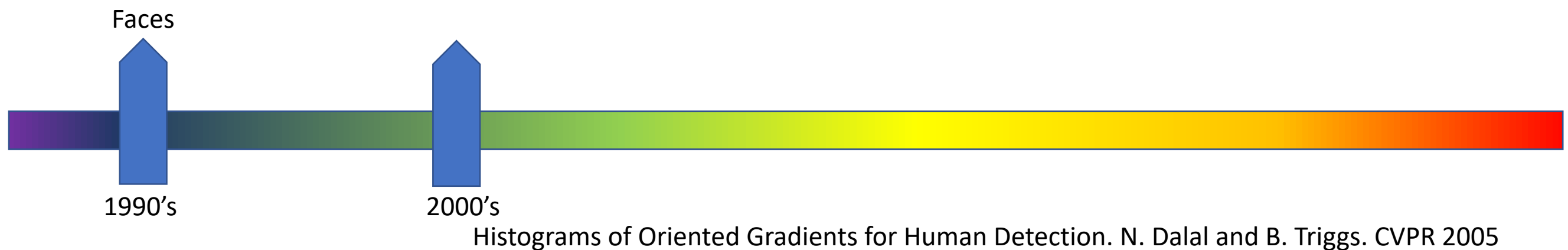


1990's

Human Face Detection in Visual Scenes. H. Rowley, S. Baluja, T. Kanade. 1995.

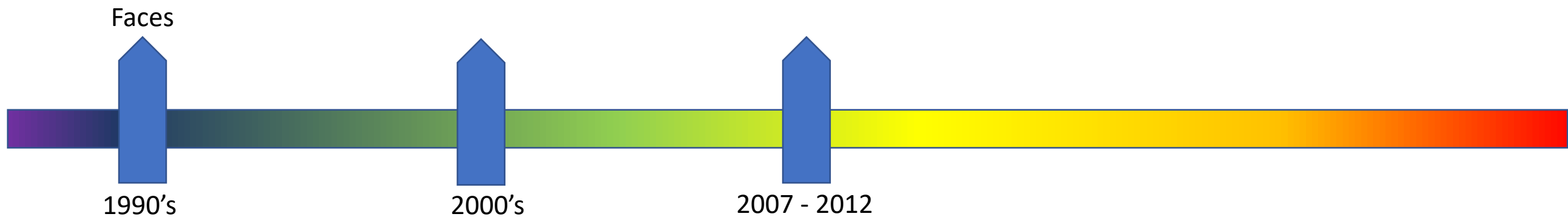
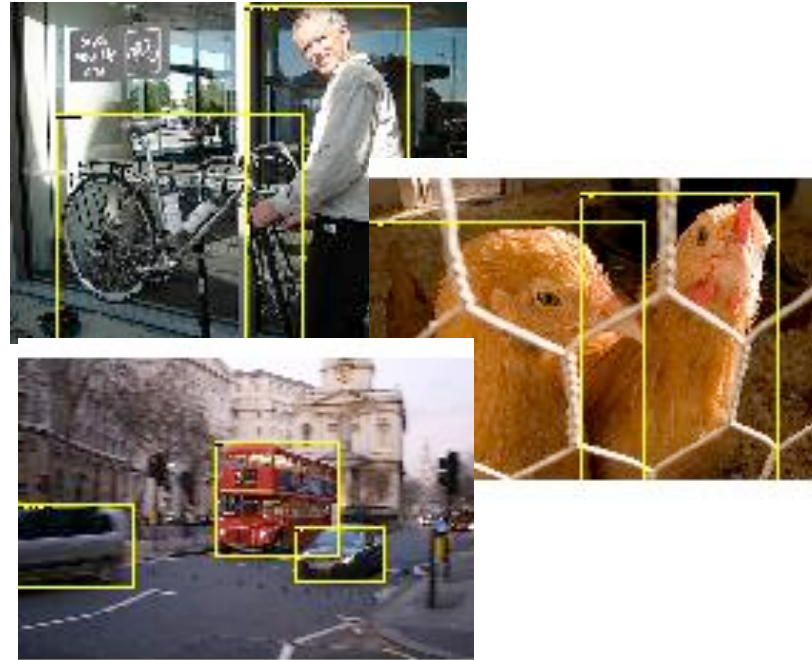
Pedestrians

- One category: pedestrians
- Slight pose variations and small distortions
- Partial occlusions



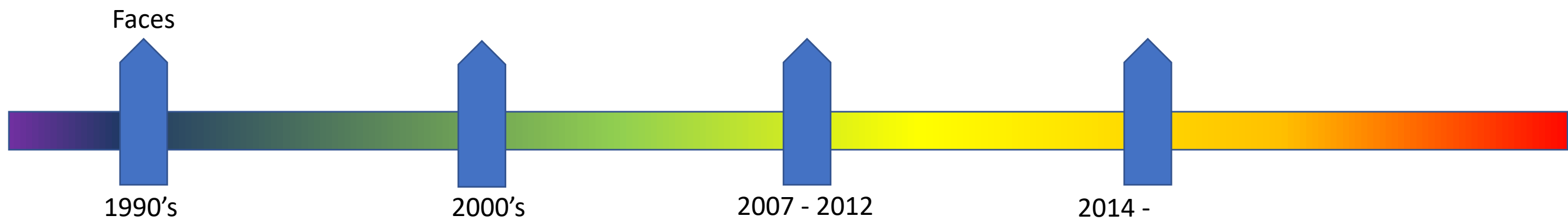
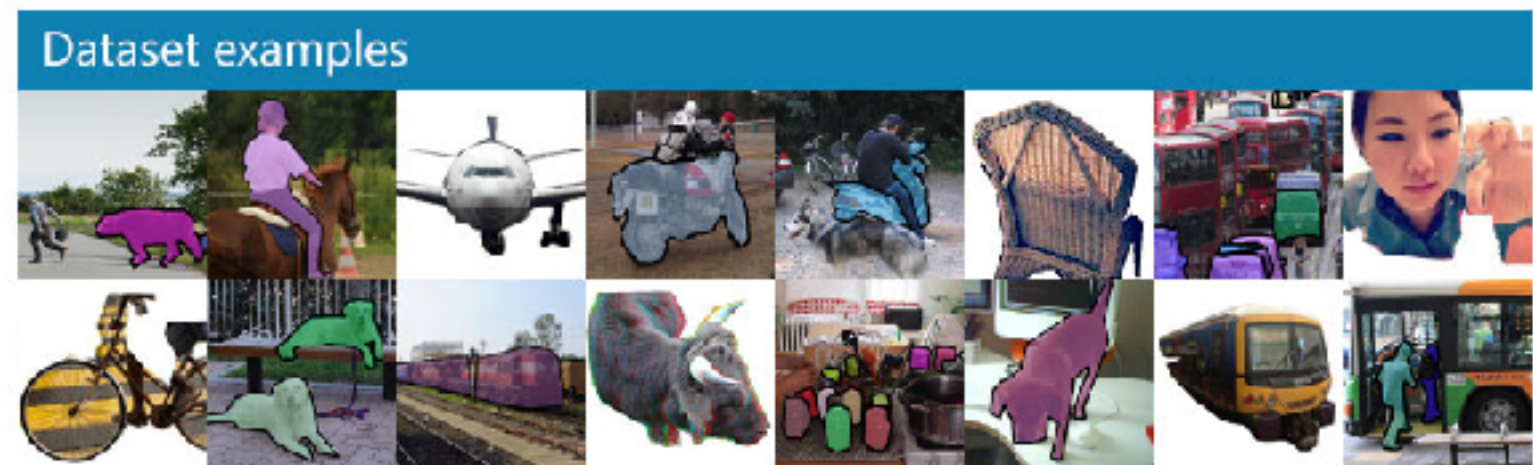
PASCAL VOC

- 20 categories
- 10K images
- Large pose variations, heavy occlusions
- Generic scenes
- Cleaned up performance metric

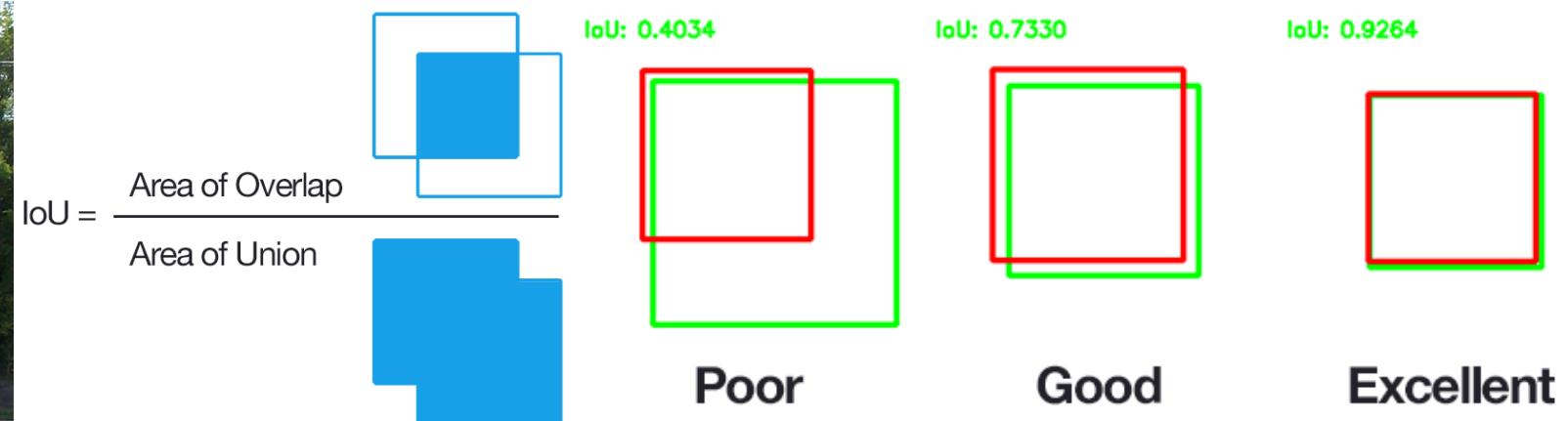
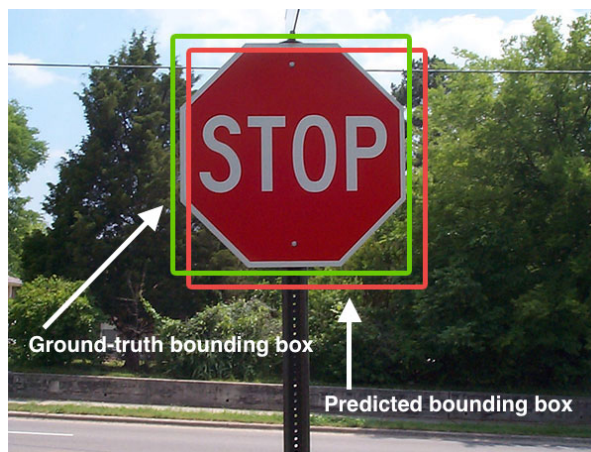


Coco

- 80 diverse categories
- 100K images
- Heavy occlusions, many objects per image, large scale variations



Intersection over union (IoU)



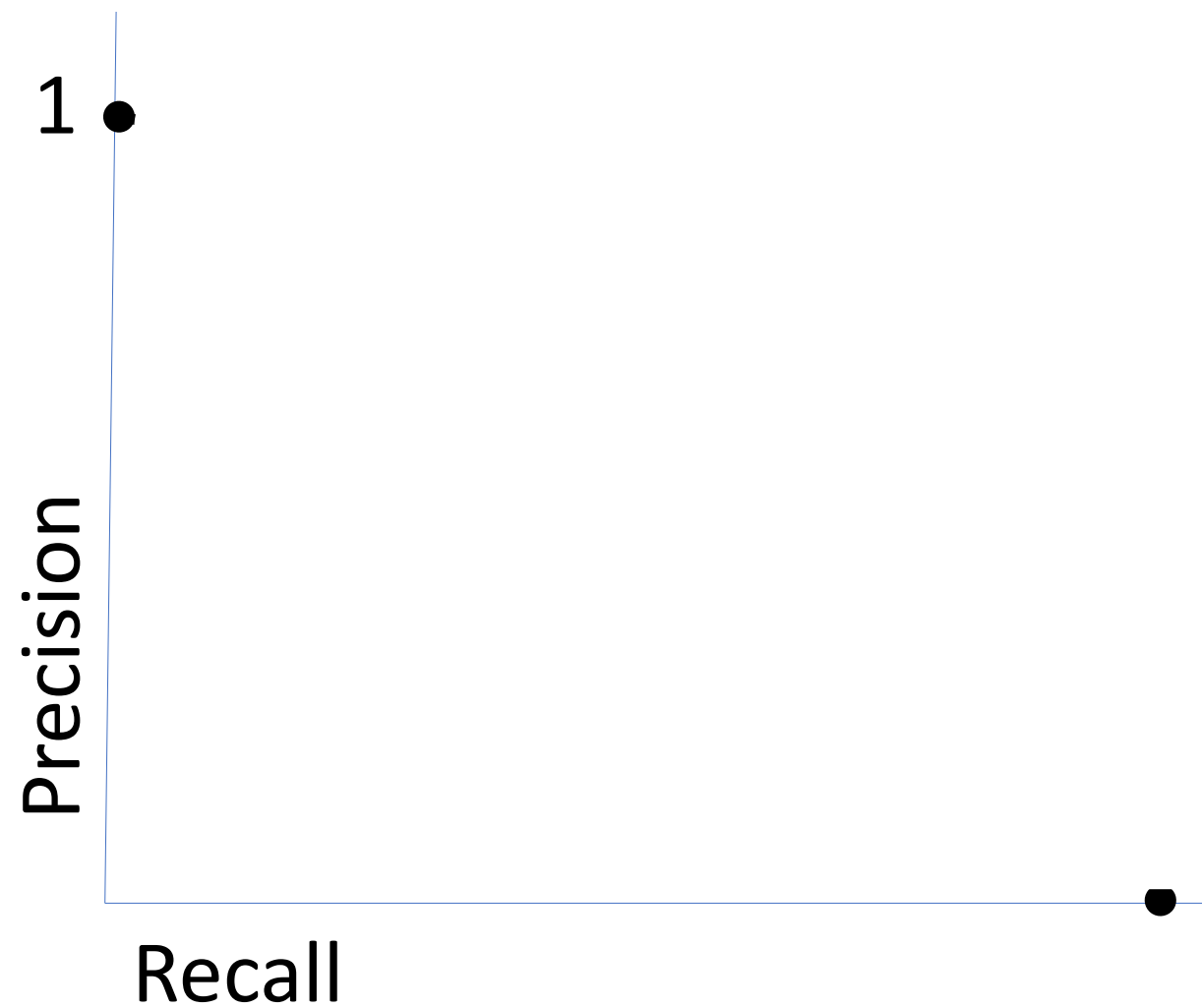
Matching detections to ground truth

- Match detection to most similar ground truth
 - highest IoU
- If $\text{IoU} > 50\%$, mark as correct
- If multiple detections map to same ground truth, mark only one as correct
- Precision = $\text{\#correct detections} / \text{total detections}$
- Recall = $\text{\#ground truth with matched detections} / \text{total ground truth}$

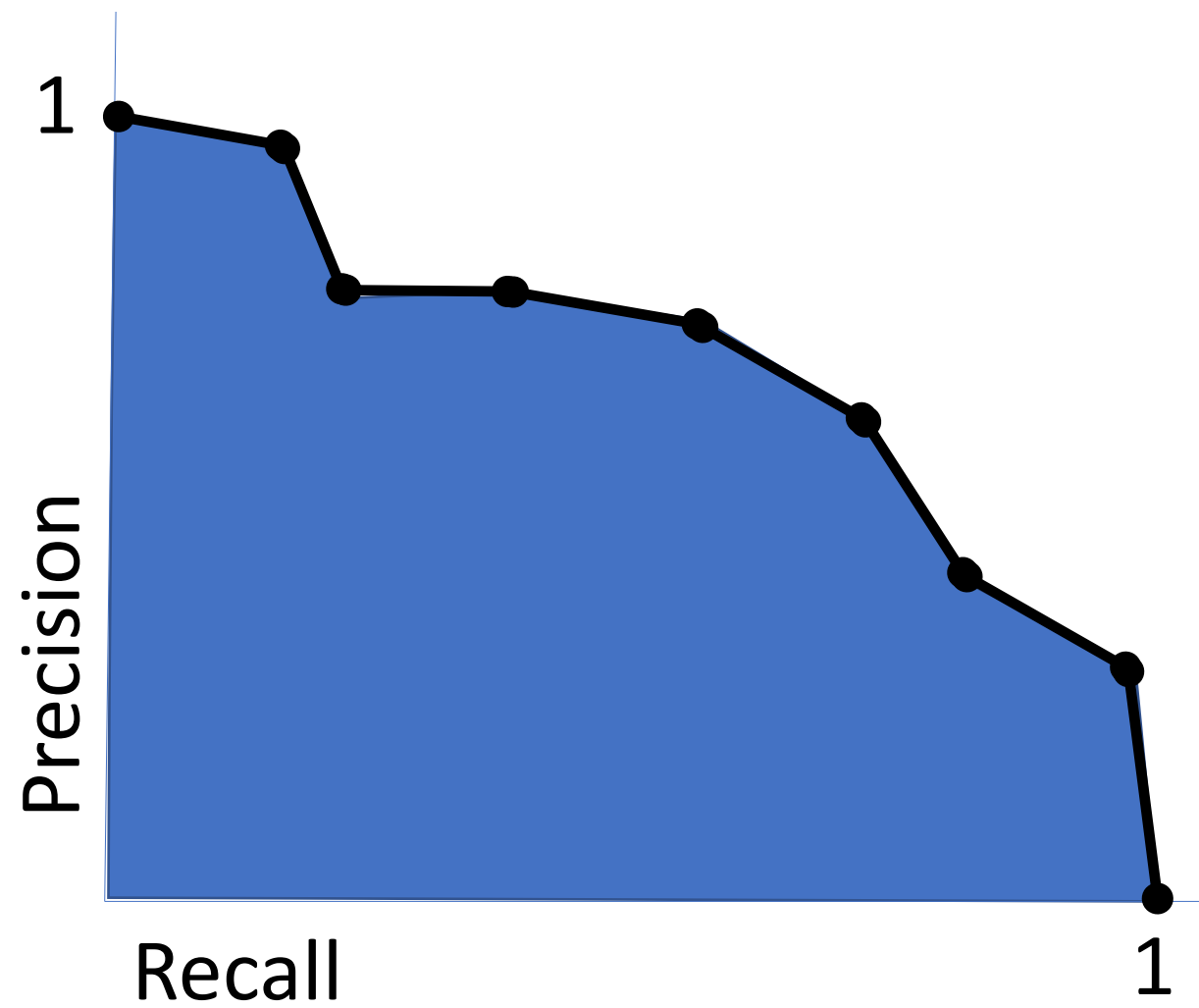
Tradeoff between precision and recall

- ML usually gives scores or probabilities, so threshold
- Too low threshold:
 - too many detections → low precision, high recall
- Too high threshold:
 - too few detections → high precision, low recall
- Right tradeoff depends on application!
 - Detecting cancer cells in tissue: need high recall
 - Detecting edible mushrooms in forest: need high precision

Average precision



Average precision



Average average precision

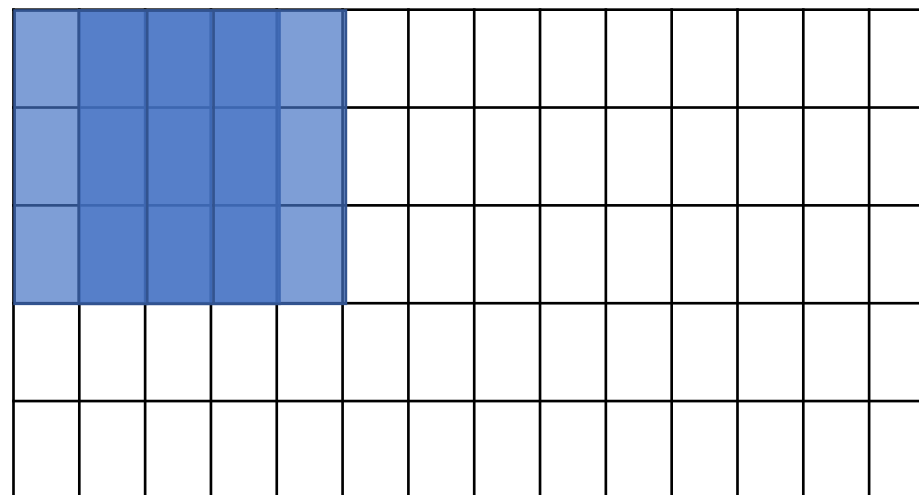
- AP marks detections with overlap $> 50\%$ as correct
- But may need better localization
- Average AP across multiple overlap thresholds
- Confusingly, still called average precision
- Introduced in COCO

Mean and category-wise AP

- Every category evaluated independently
- Typically report mean AP averaged over all categories
- Confusingly called “mean Average Precision”, or “mAP”

Scanning window

- Fix size
 - Can take a few different sizes
- Fixed stride
- Convolution with a filter
 - Classic: compute HOG features over entire image



Dealing with scale



Dealing with scale



Use same
window size,
but run on *image
pyramid*



Issues

- Classifies millions of boxes, so must be very fast
- Needs ultra-fine sampling of scales and object sizes, can still miss outlier sizes



Scanning window results on PASCAL

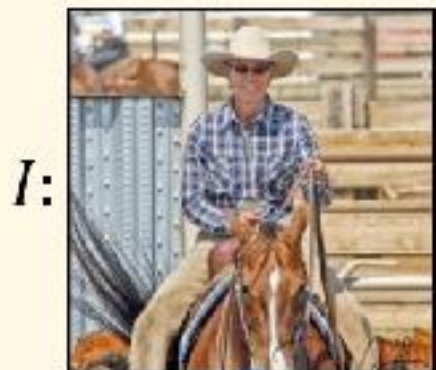
	VOC 2007	VOC 2010
DPM v5 (Girshick et al. 2011)	33.7%	29.6%

Reference systems

R-CNN (Girshick et al, 2014)

Region-based Convolutional Neural Network

Per-image computation



Selective search,
Edge Boxes,
MCG, ...

1

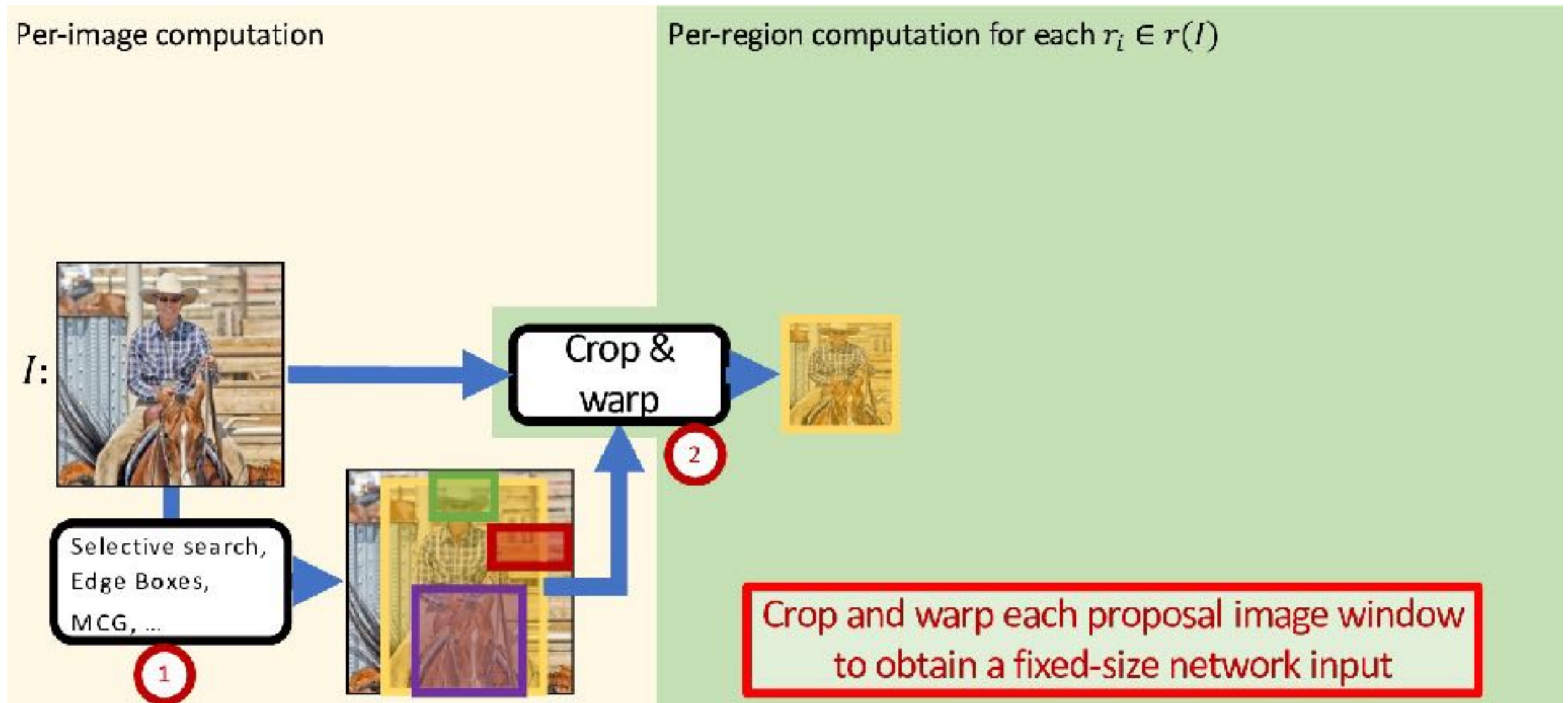


Use an off-the-shelf region/object/detection
proposal algorithm (~2k proposals per image)

Girshick, Donahue, Darrell and Malik: Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR, 2014.

7
6

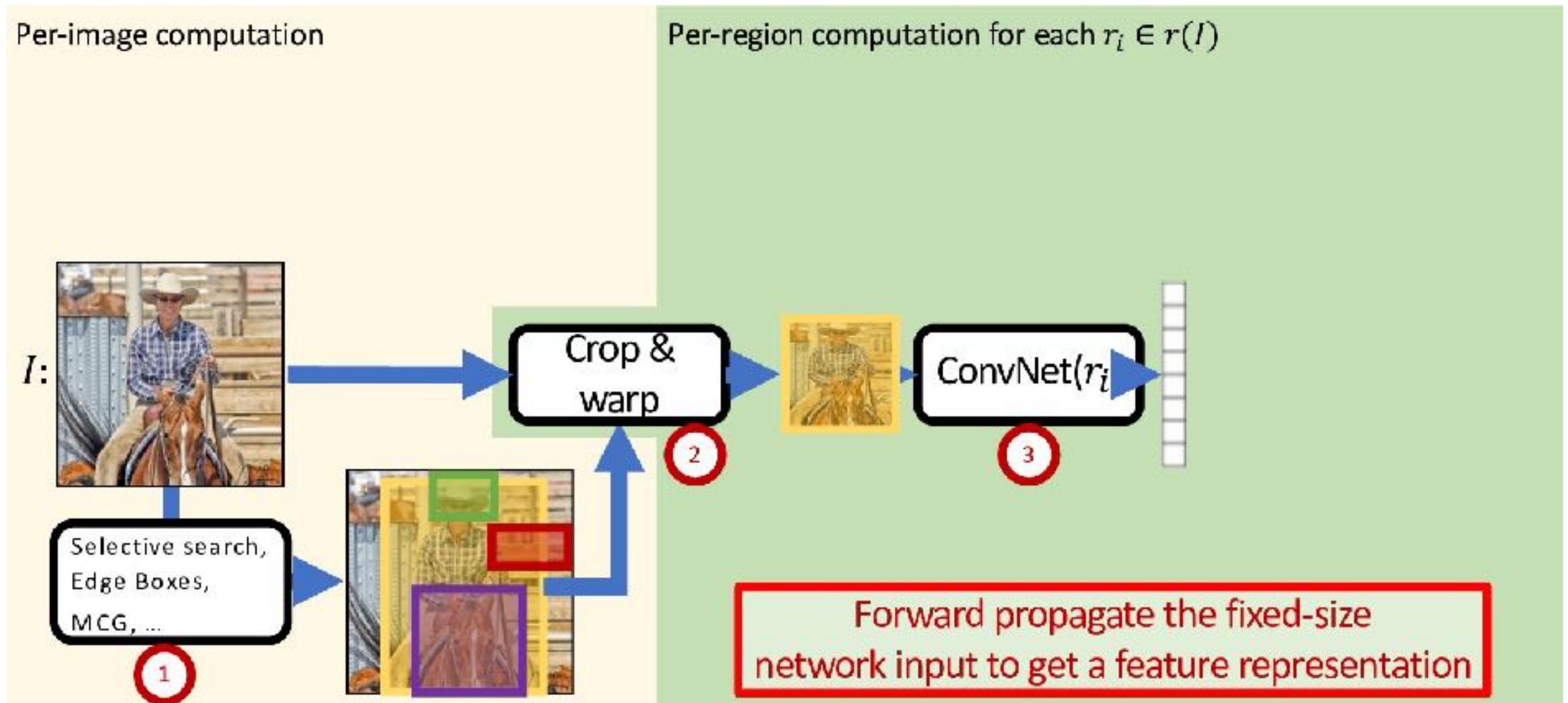
R-CNN (Girshick et al, 2014)



Girshick, Donahue, Darrell and Malik: Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR, 2014.

7
6

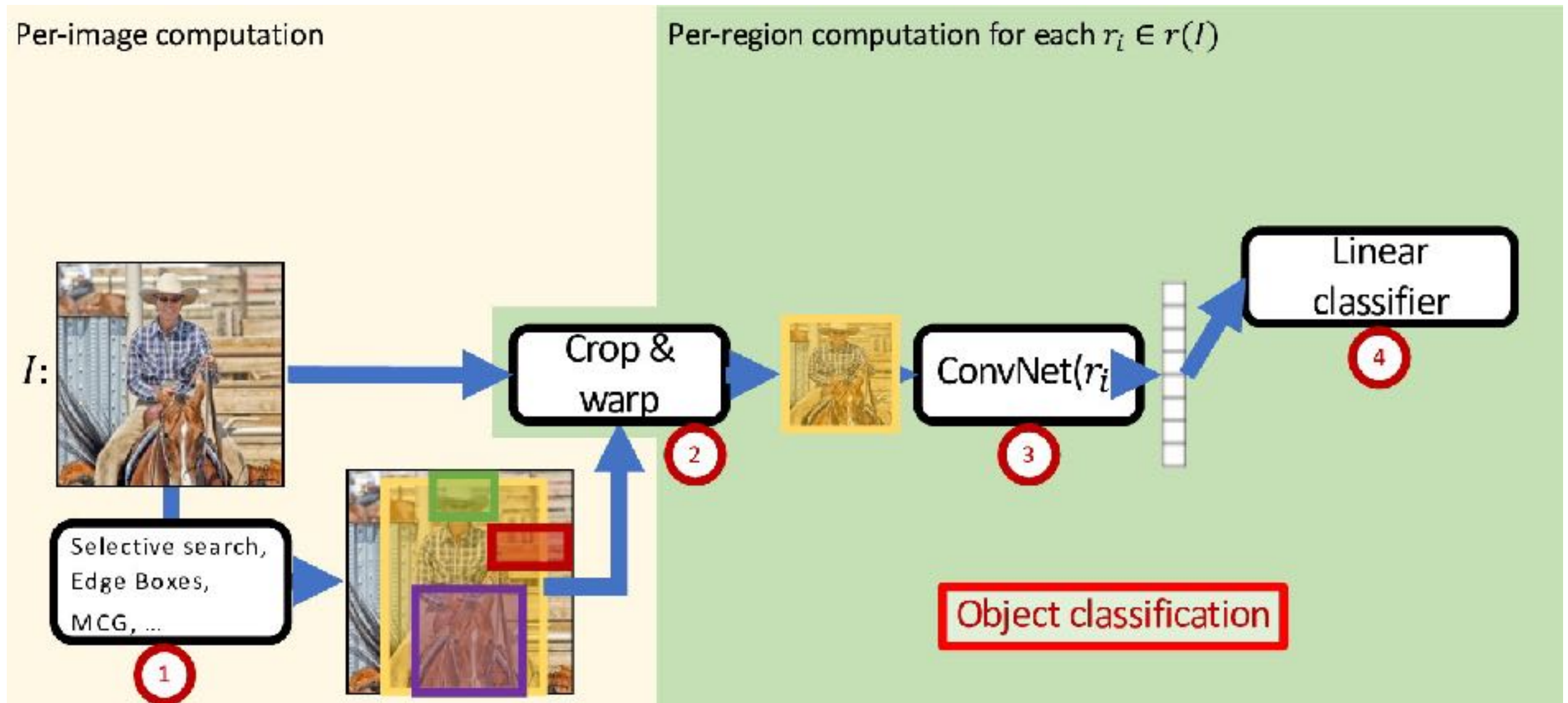
R-CNN (Girshick et al, 2014)



Girshick, Donahue, Darrell and Malik: Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR, 2014.

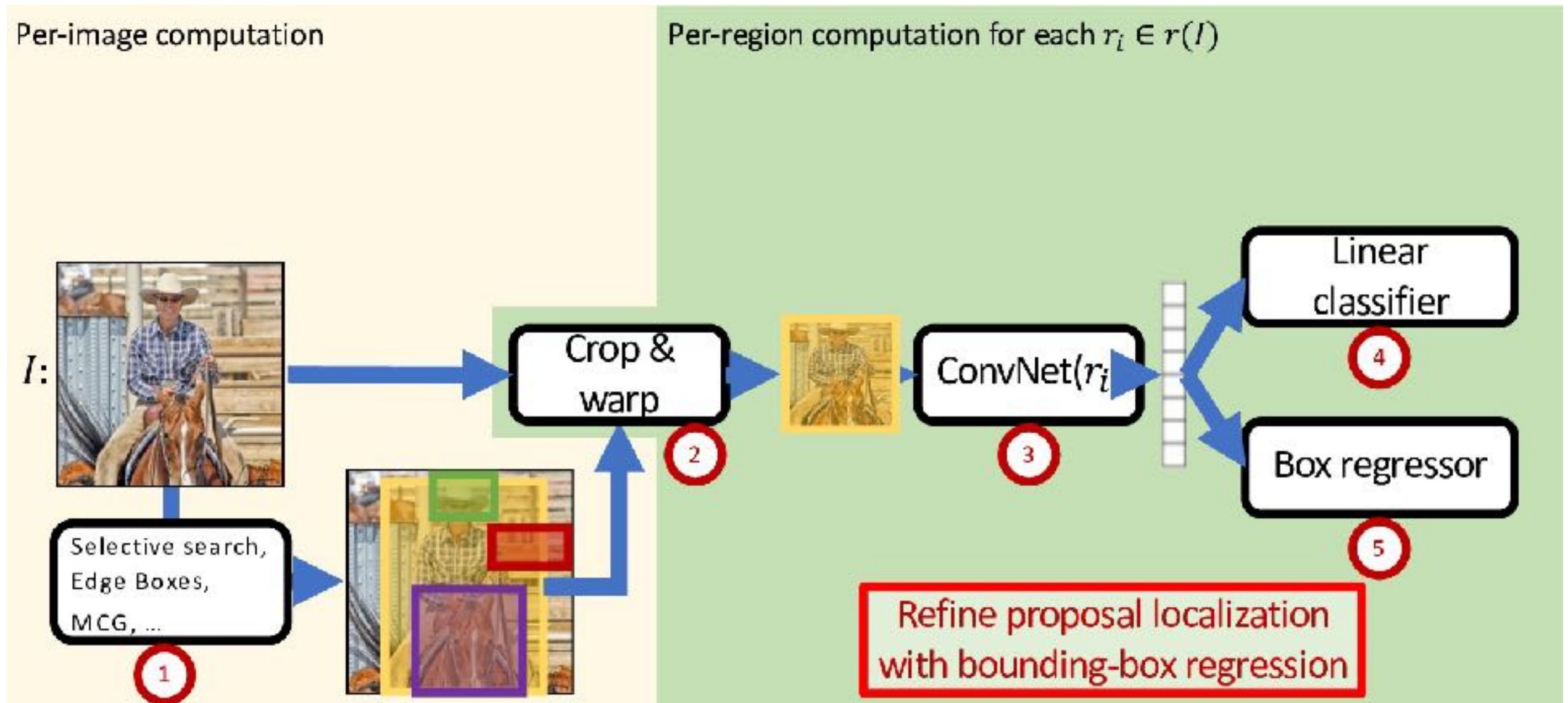
7
6

R-CNN (Girshick et al, 2014)



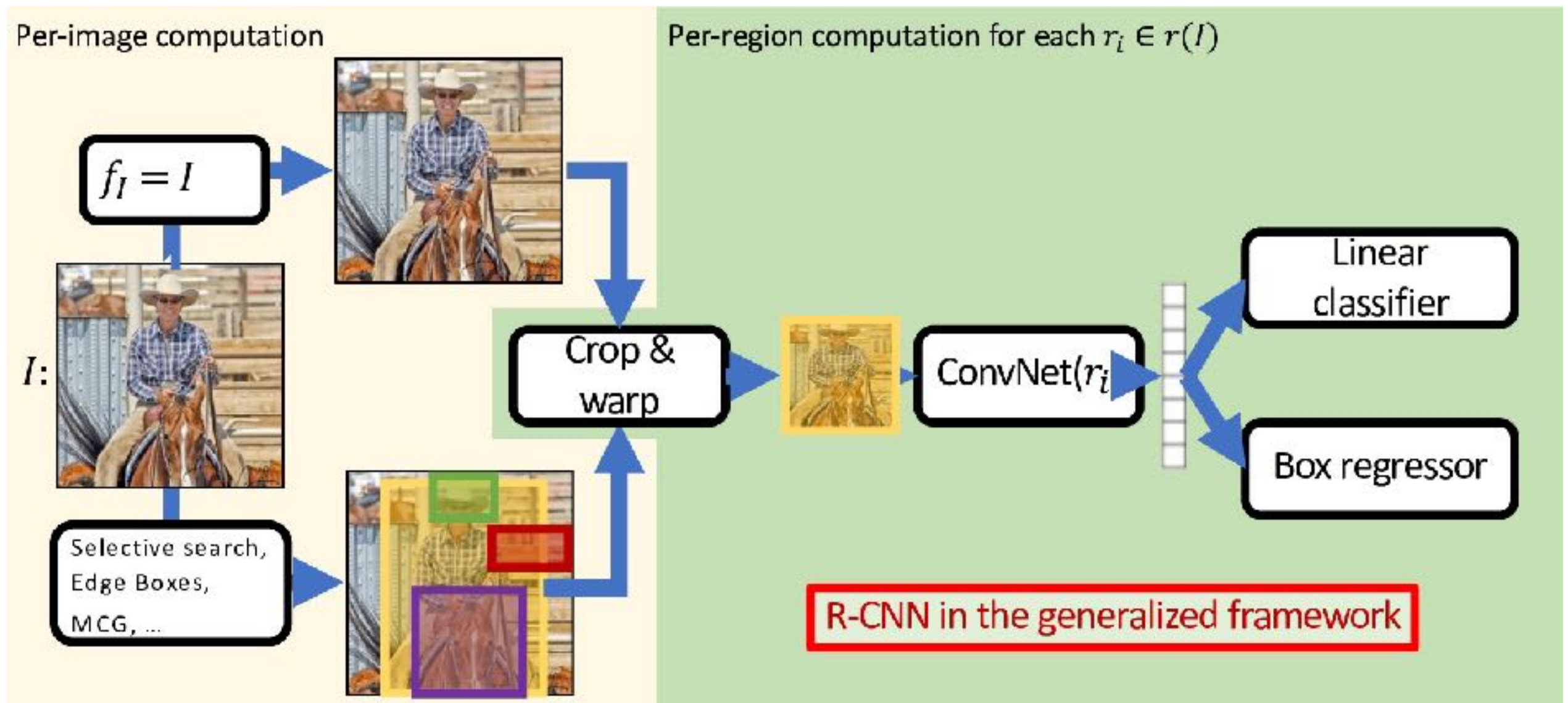
Girshick, Donahue, Darrell and Malik: Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR, 2014.

R-CNN (Girshick et al, 2014)



Girshick, Donahue, Darrell and Malik: Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR, 2014.

R-CNN



Girshick, Donahue, Darrell and Malik: Rich feature hierarchies for accurate object detection and semantic segmentation. CVPR, 2014.

Weaknesses of R-CNN

- Heavy per-region computation (e.g., 2000 full network evaluations)
- No computation/feature sharing
- Slow region proposal method adds to runtime
- Generic region proposal techniques have limited recall

R-CNN results on PASCAL

	VOC 2007	VOC 2010
DPM v5 (Girshick et al. 2011)	33.7%	29.6%
UVA sel. search (Uijlings et al. 2013)		35.1%
Regionlets (Wang et al. 2013)	41.7%	39.7%
SegDPM (Fidler et al. 2013)		40.4%
R-CNN	54.2%	50.2%
R-CNN + bbox regression	58.5%	53.7%

Slide credit : Ross
Girshick

Training R-CNN

- Train convolutional network on ImageNet classification
- *Finetune* on detection
 - Classification problem!
 - Proposals with $\text{IoU} > 50\%$ are positives
 - Sample fixed proportion of positives in each batch because of imbalance

Other details - Non-max suppression

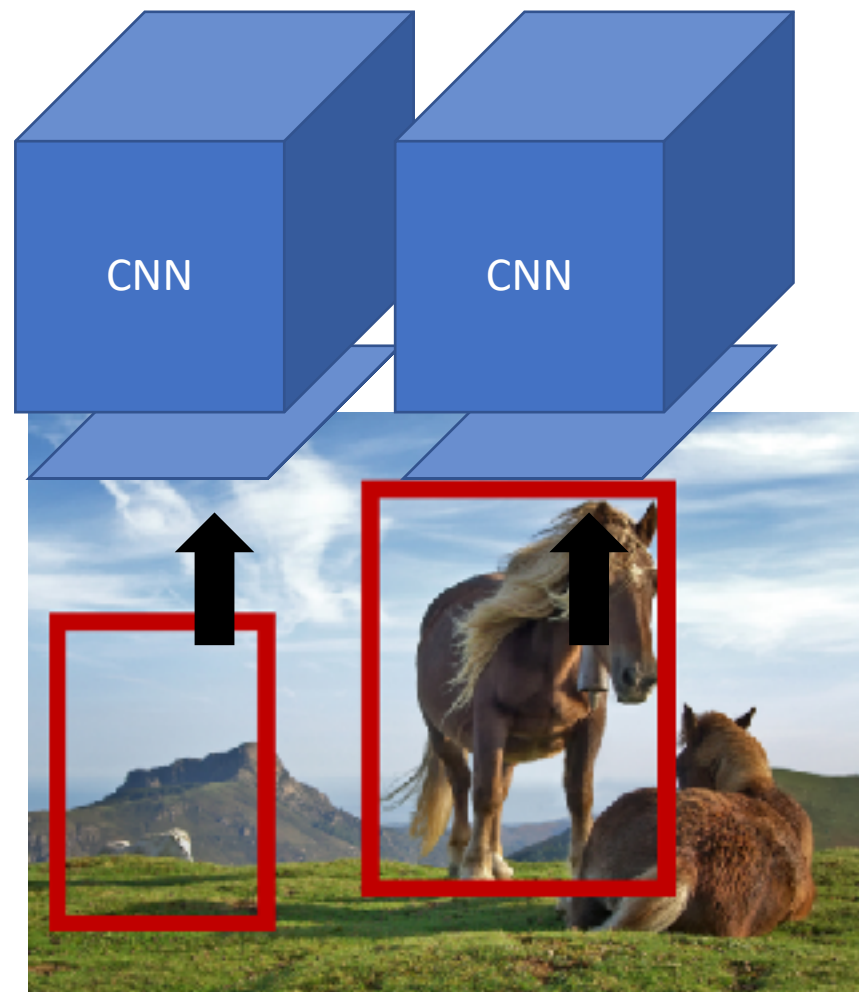


How do we deal with multiple detections on the same object?

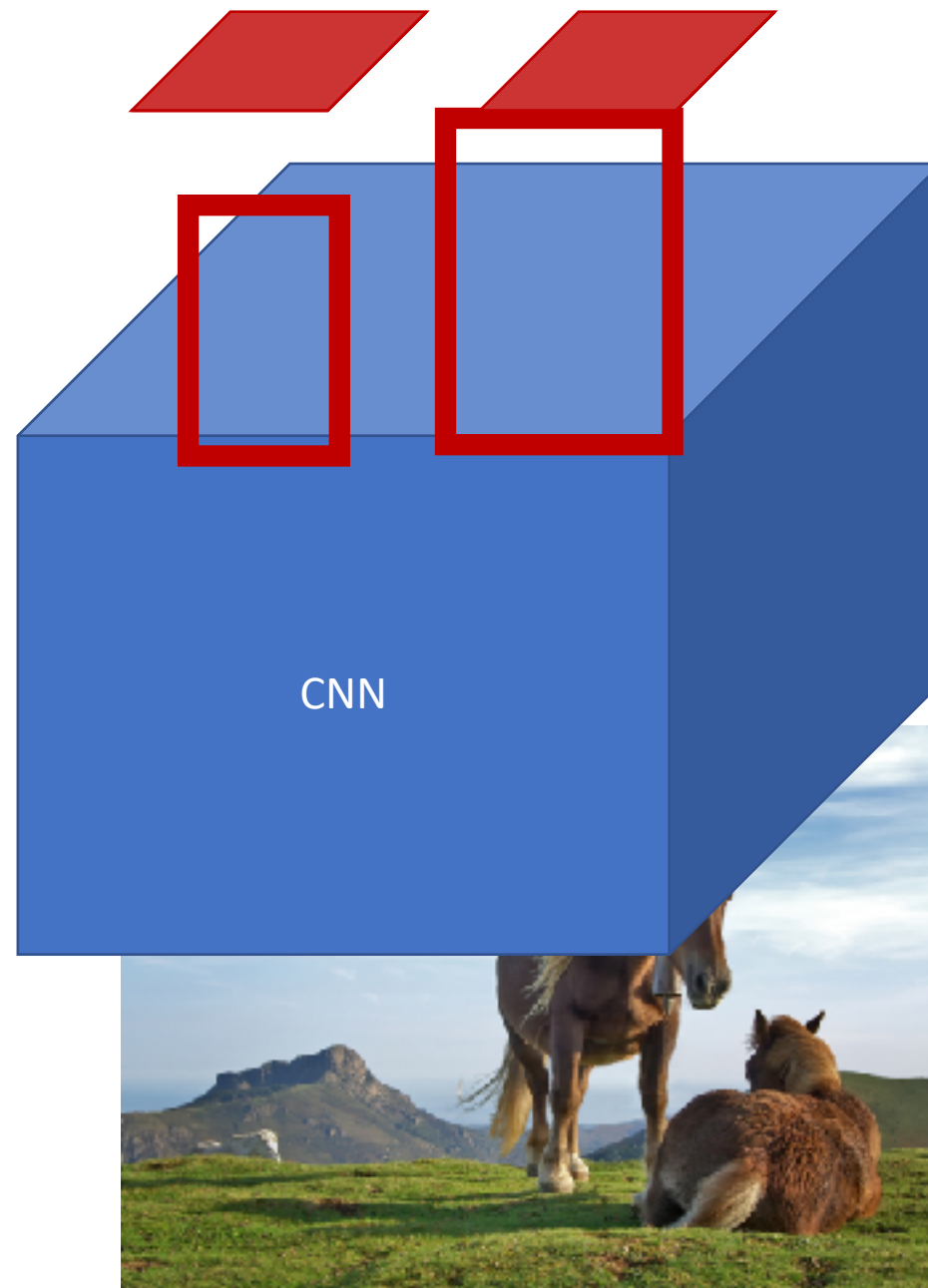
Other details - Non-max suppression

- Go down the list of detections starting from highest scoring
- Eliminate any detection that overlaps highly with a higher scoring detection
- Separate, heuristic step

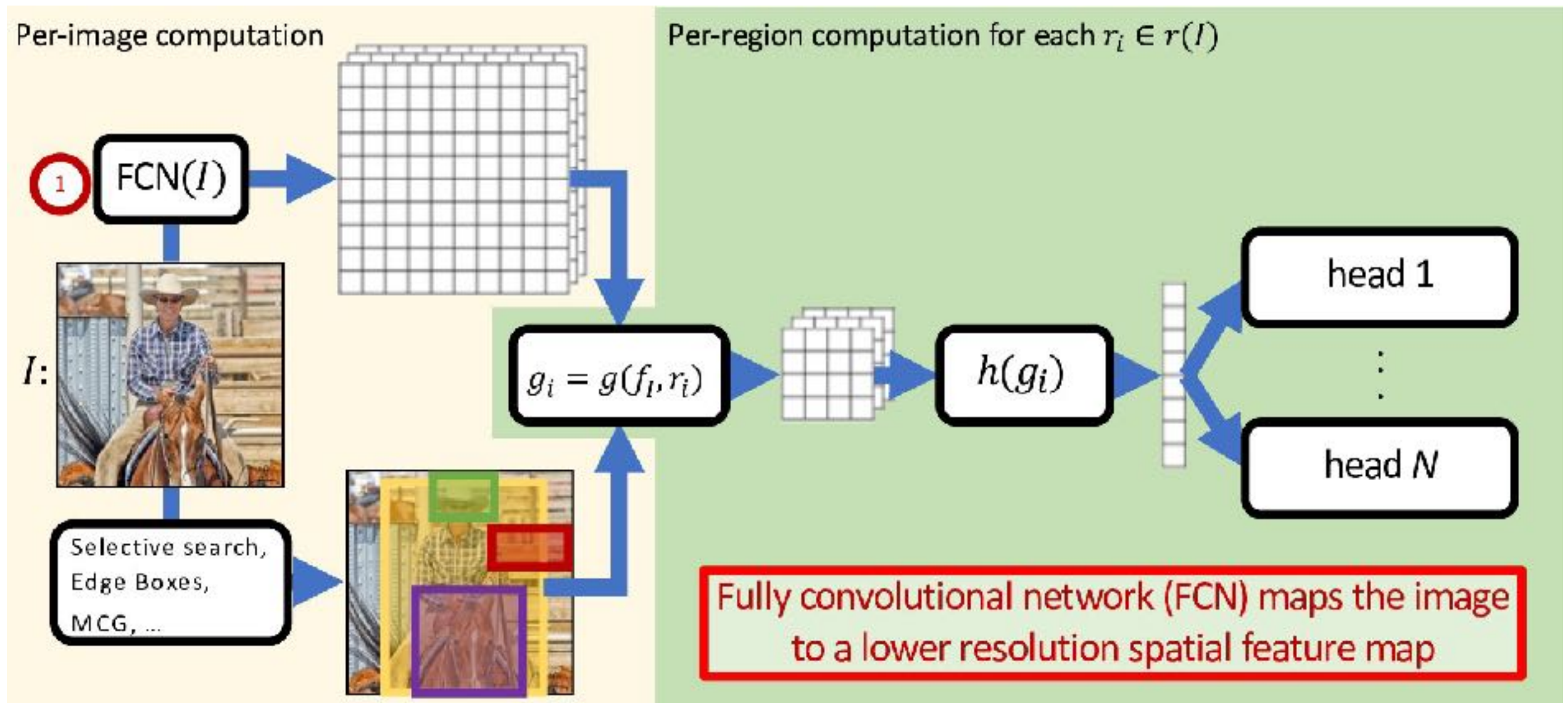
Speeding up R-CNN



Speeding up R-CNN

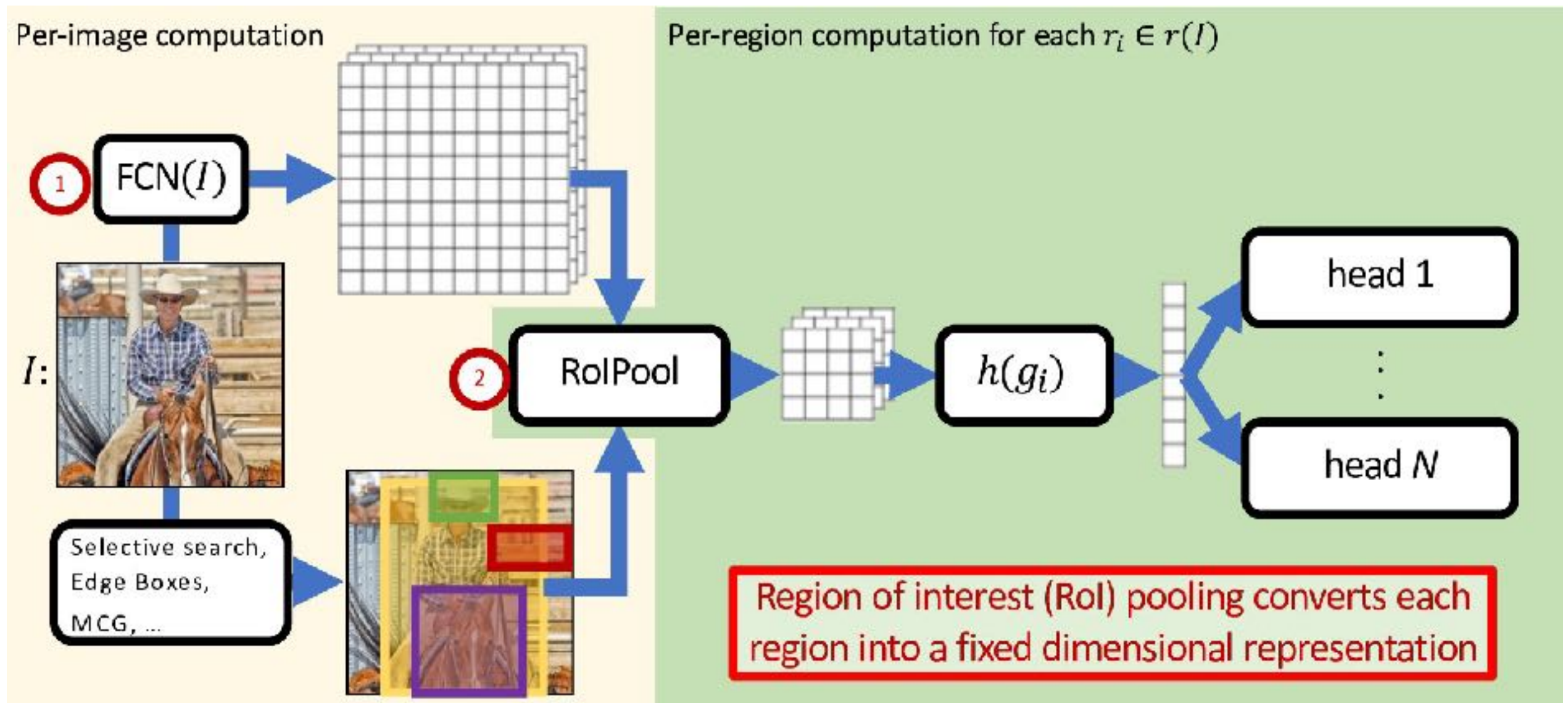


Fast R-CNN (Girshick et al. 2015)



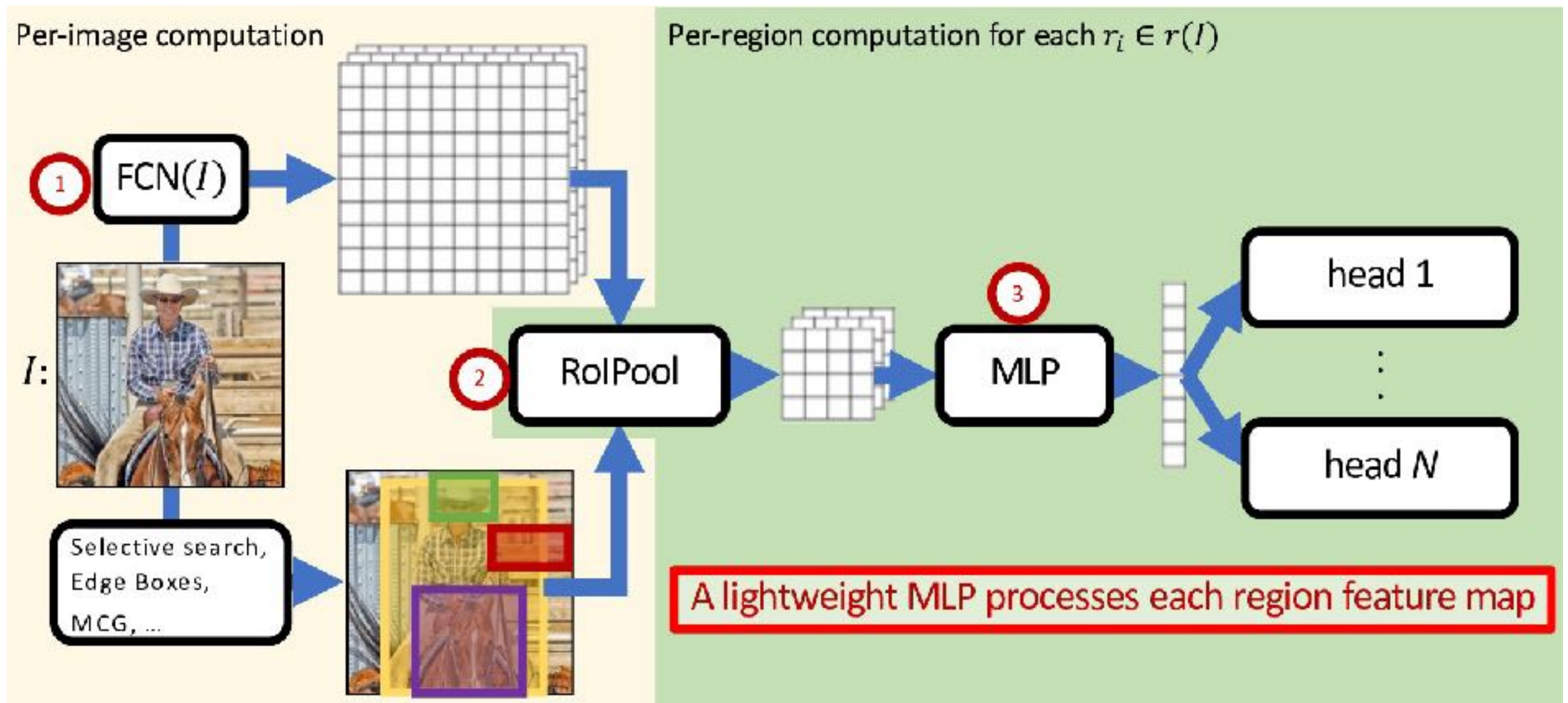
Girshick: Fast R-CNN.
ICCV, 2015.

Fast R-CNN (Girshick et al. 2015)



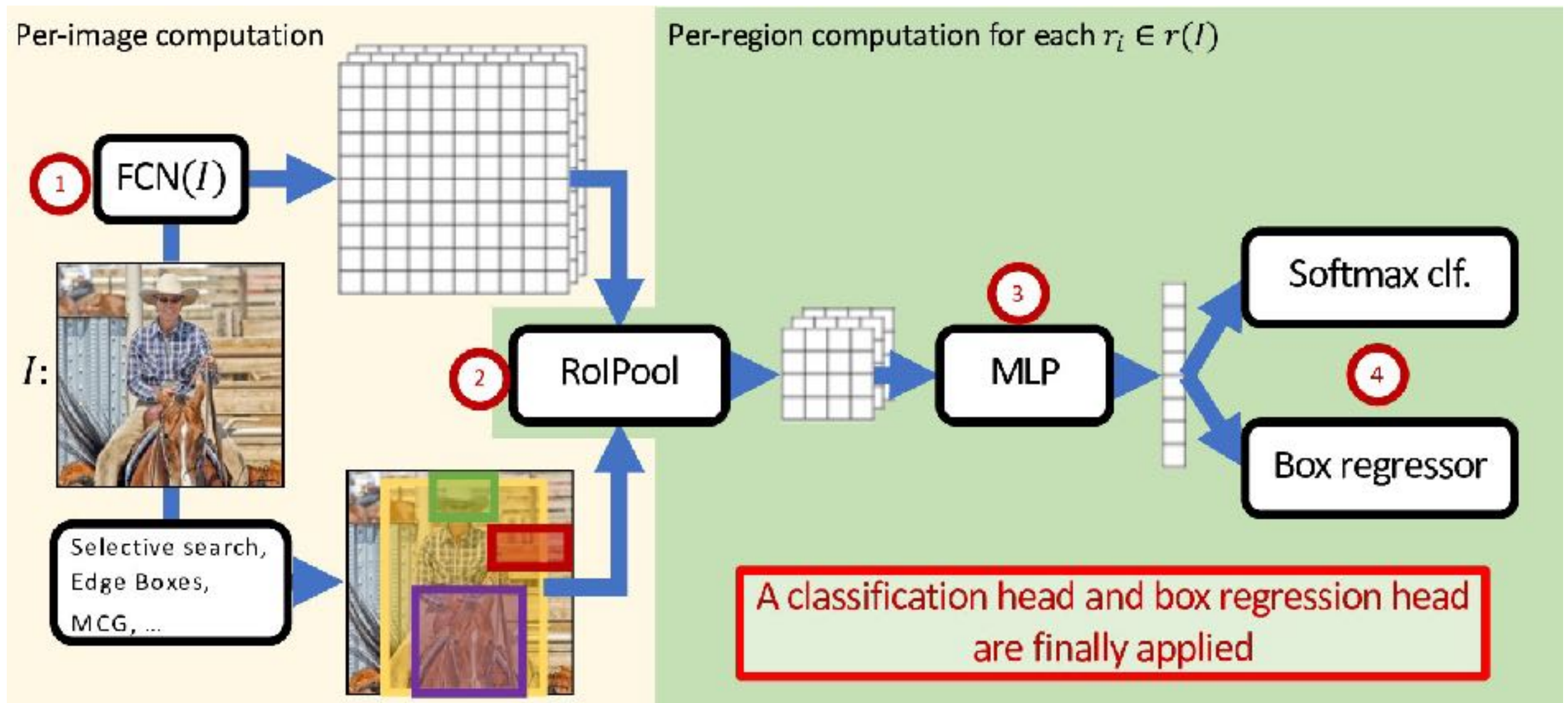
Girshick: Fast R-CNN.
ICCV, 2015.

Fast R-CNN (Girshick et al. 2015)



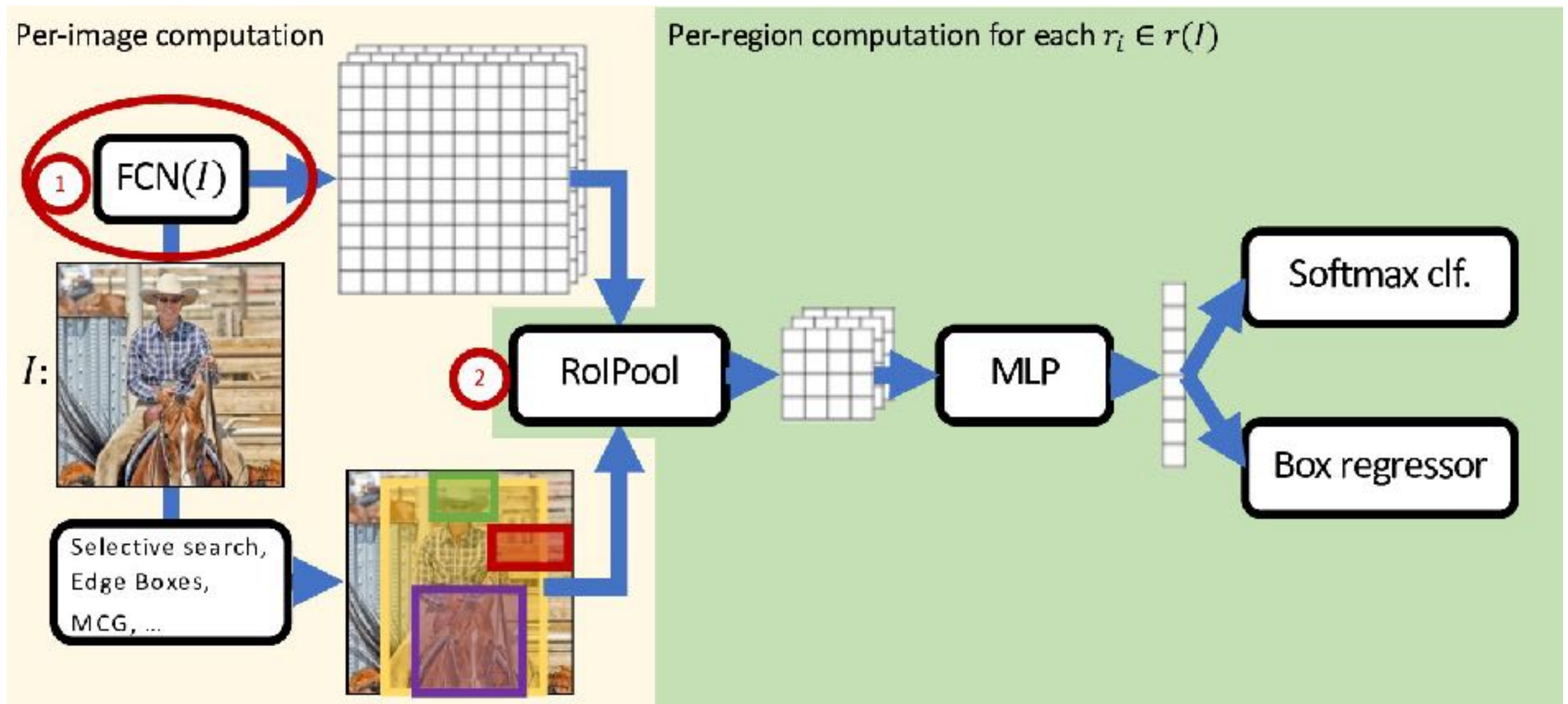
Girshick: Fast R-CNN.
ICCV, 2015.

Fast R-CNN (Girshick et al. 2015)



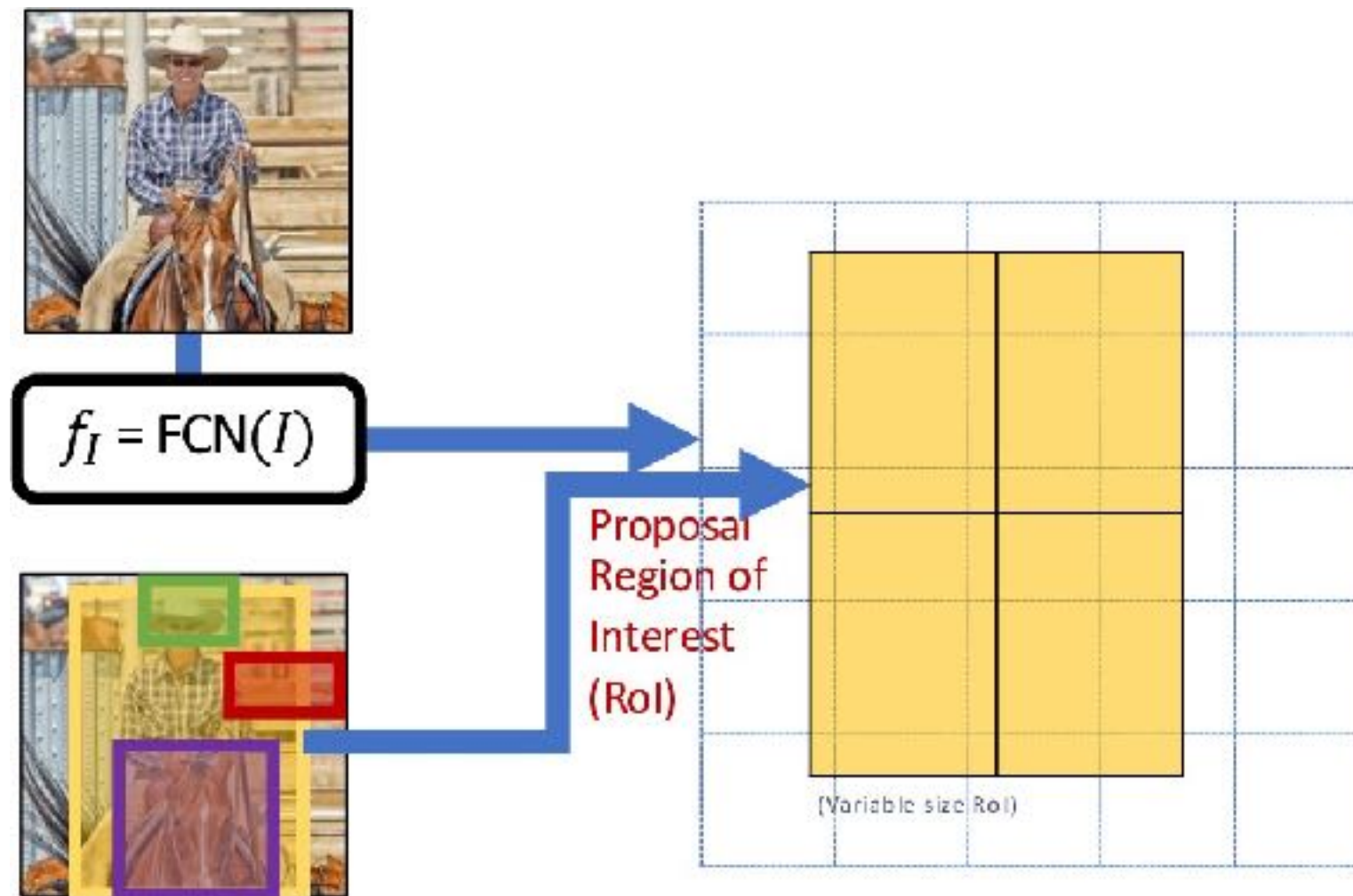
Girshick: Fast R-CNN.
ICCV, 2015.

Fast R-CNN (Girshick et al. 2015)



Girshick: Fast R-CNN.
ICCV, 2015.

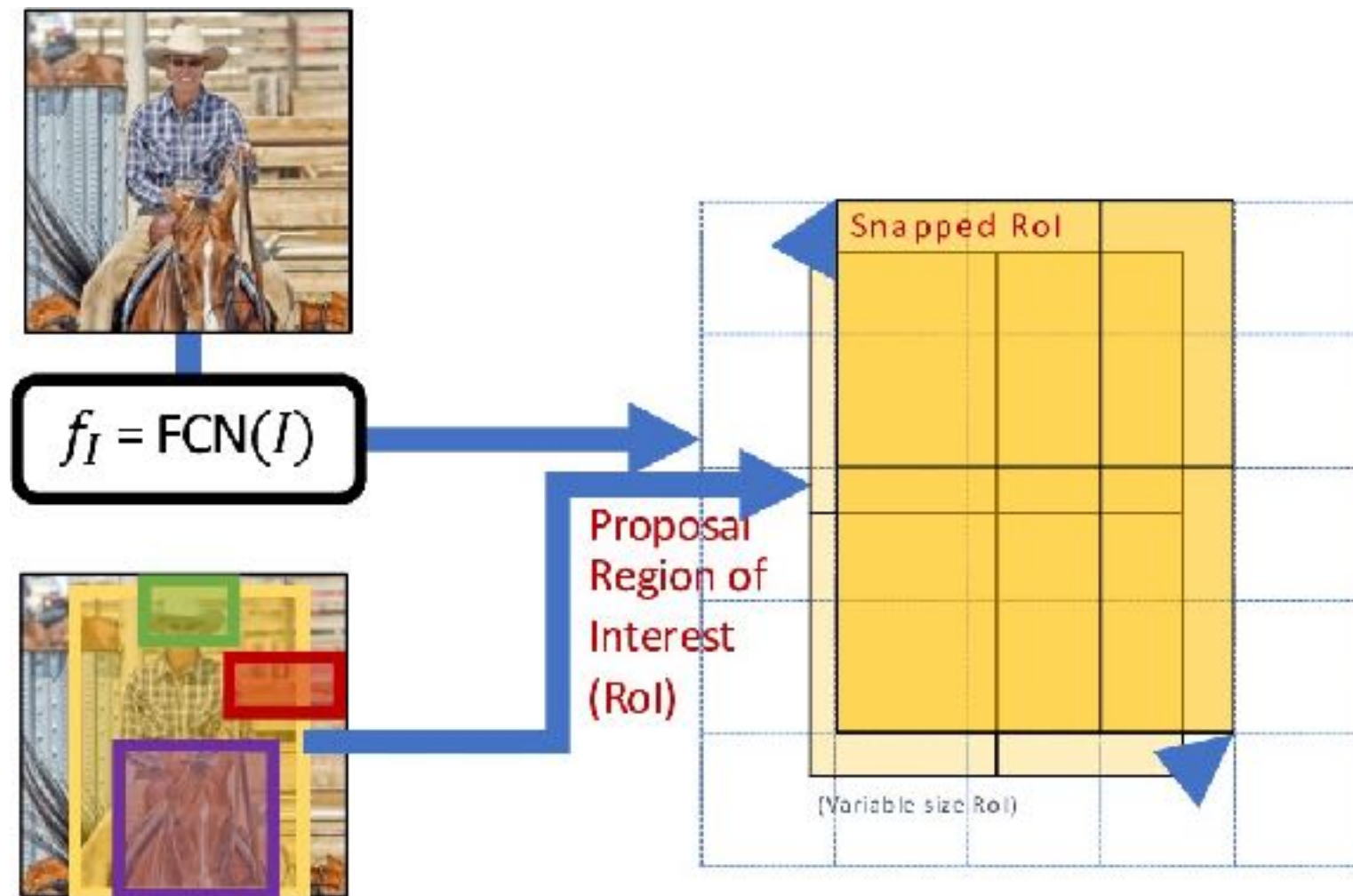
ROI pooling



Key innovation in SPP-net
[He et al. 2014]

Girshick: Fast R-CNN.
ICCV, 2015.

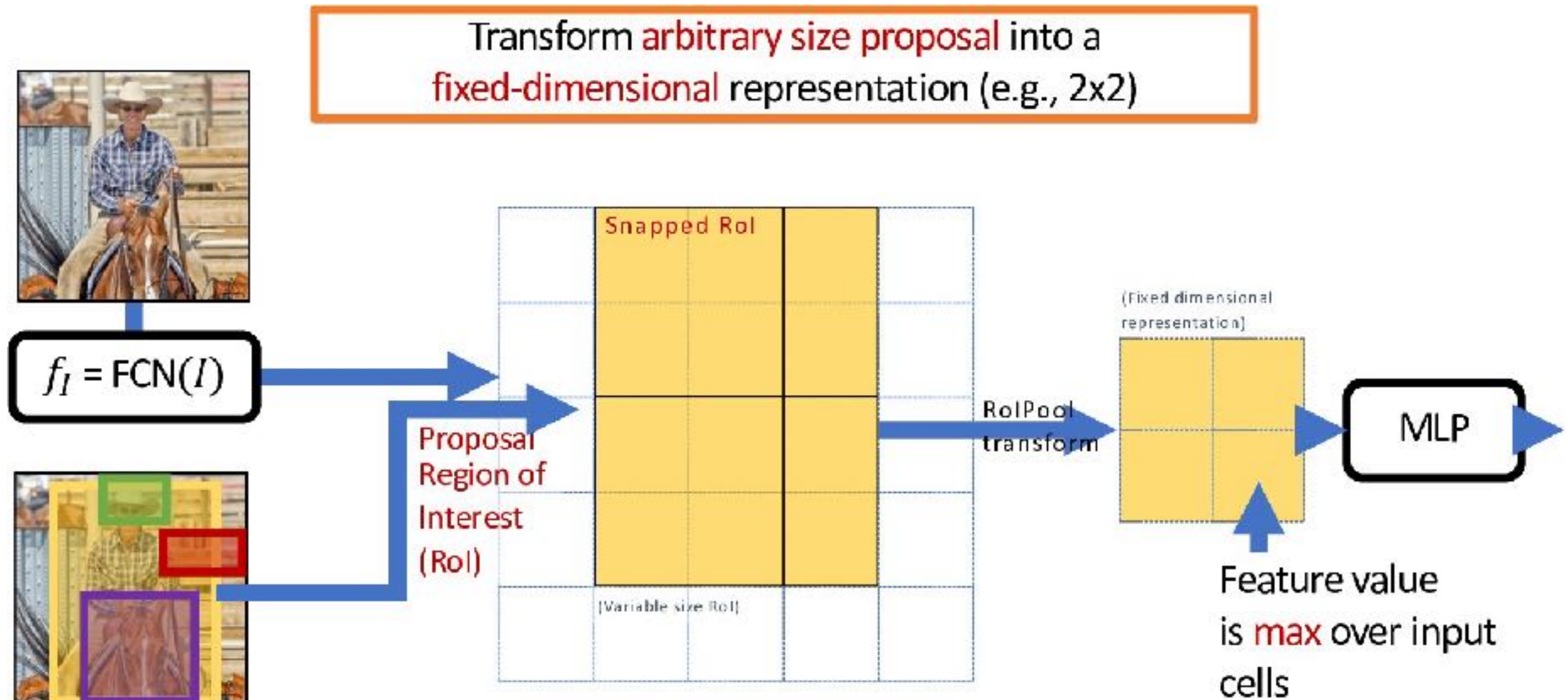
ROI pooling



Key innovation in SPP-net
[He et al. 2014]

Girshick: Fast R-CNN.
ICCV, 2015.

ROI pooling



Girshick: Fast R-CNN.
ICCV, 2015.

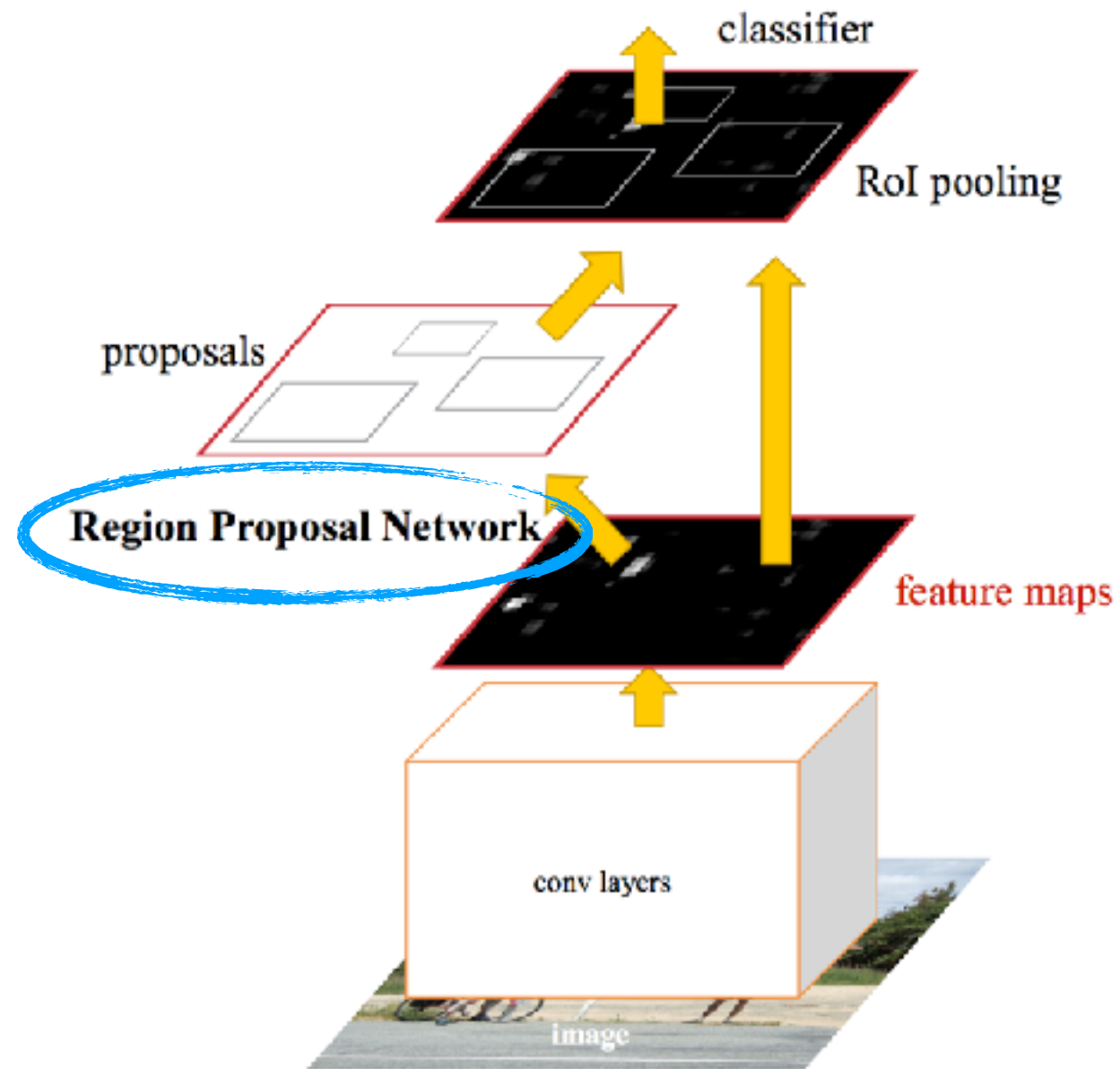
Fast R-CNN

	Fast R-CNN	R-CNN
Train time (h)	9.5	84
Speedup	8.8x	1x
Test time / image	0.32s	47.0s
Speedup	146x	1x
mean AP	66.9	66.0

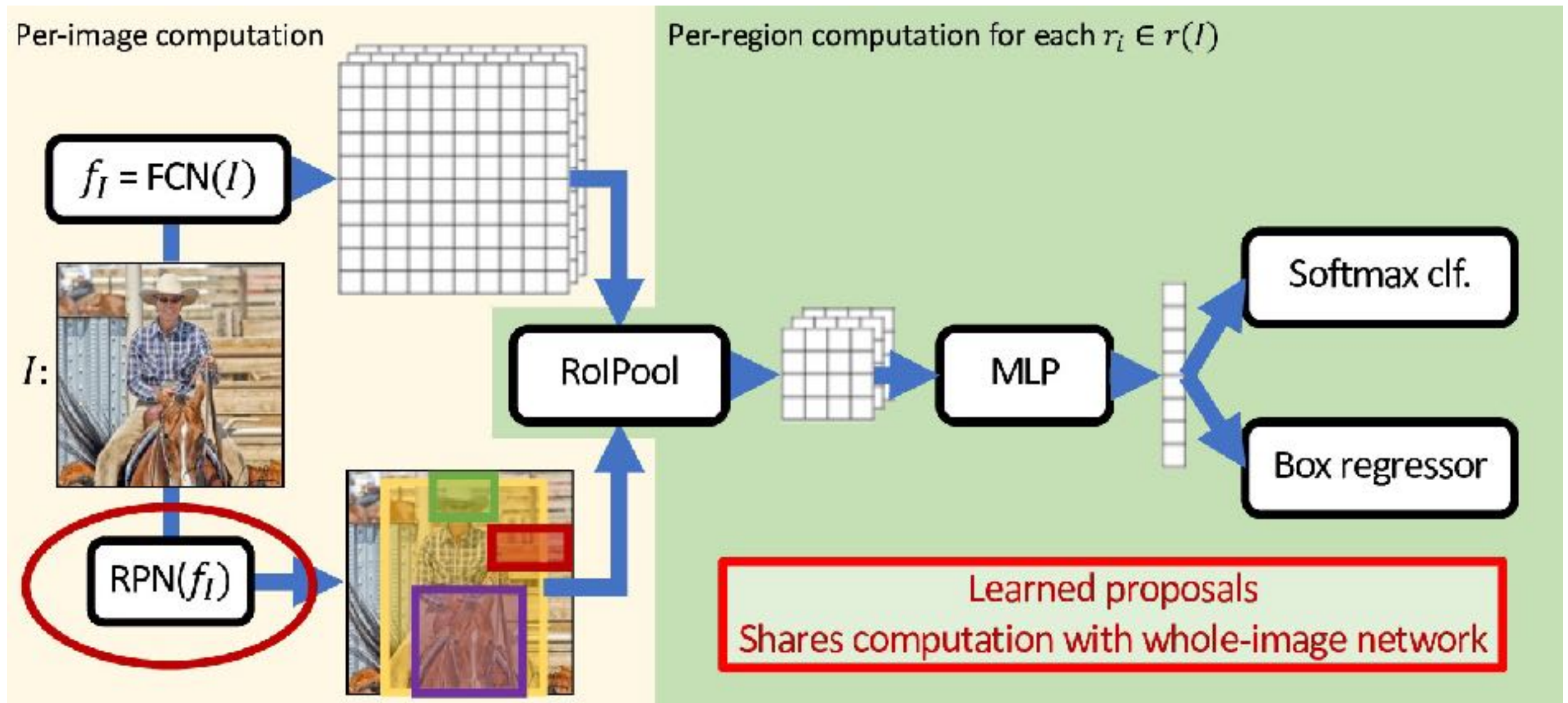
Fast R-CNN

- Bottleneck remaining (not included in time):
 - Object proposal generation
- Slow
 - Requires segmentation
 - $O(1s)$ per image

Faster R-CNN

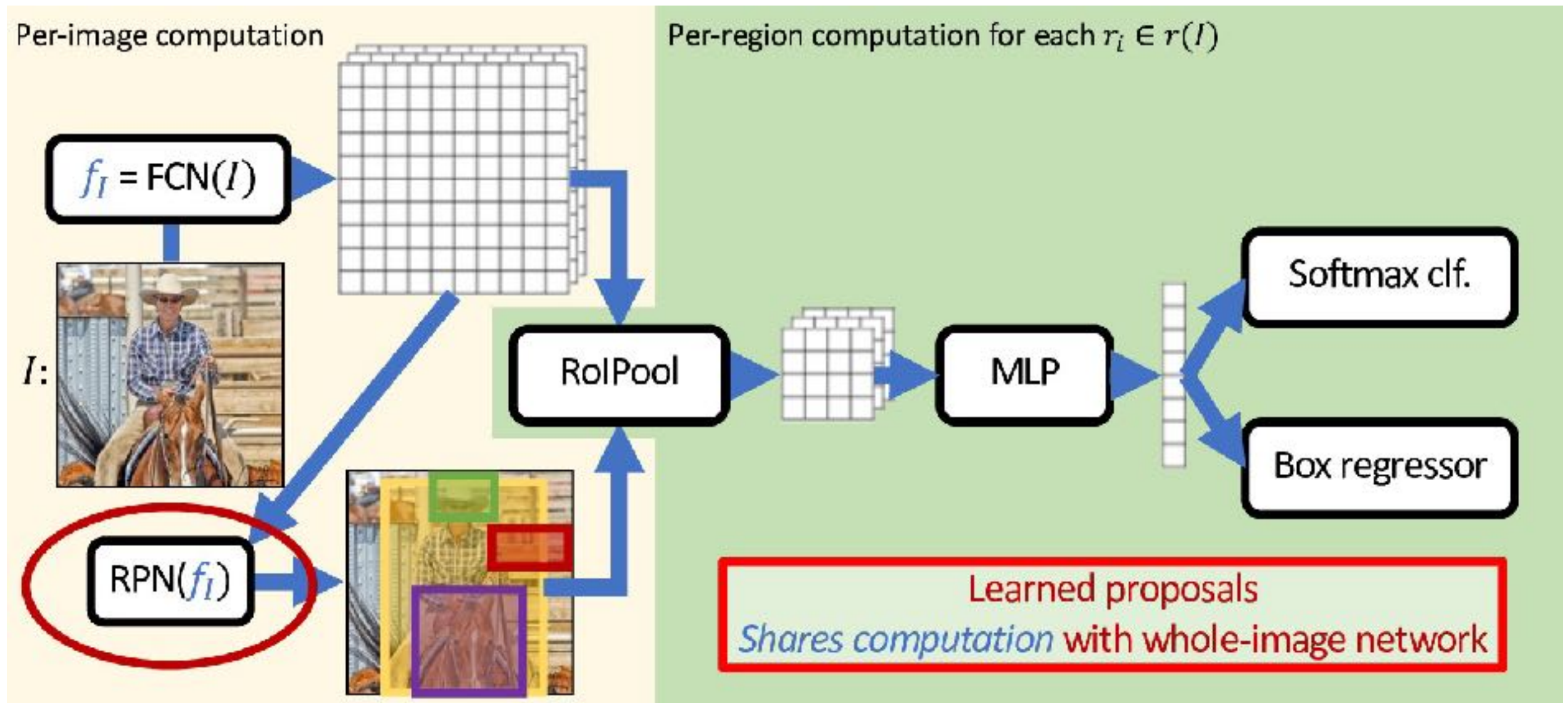


Faster R-CNN (Ren et al. 2015)



Ren, He, Girshick and Sun et al.: Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NIPS, 2015.

Faster R-CNN (Ren et al. 2015)



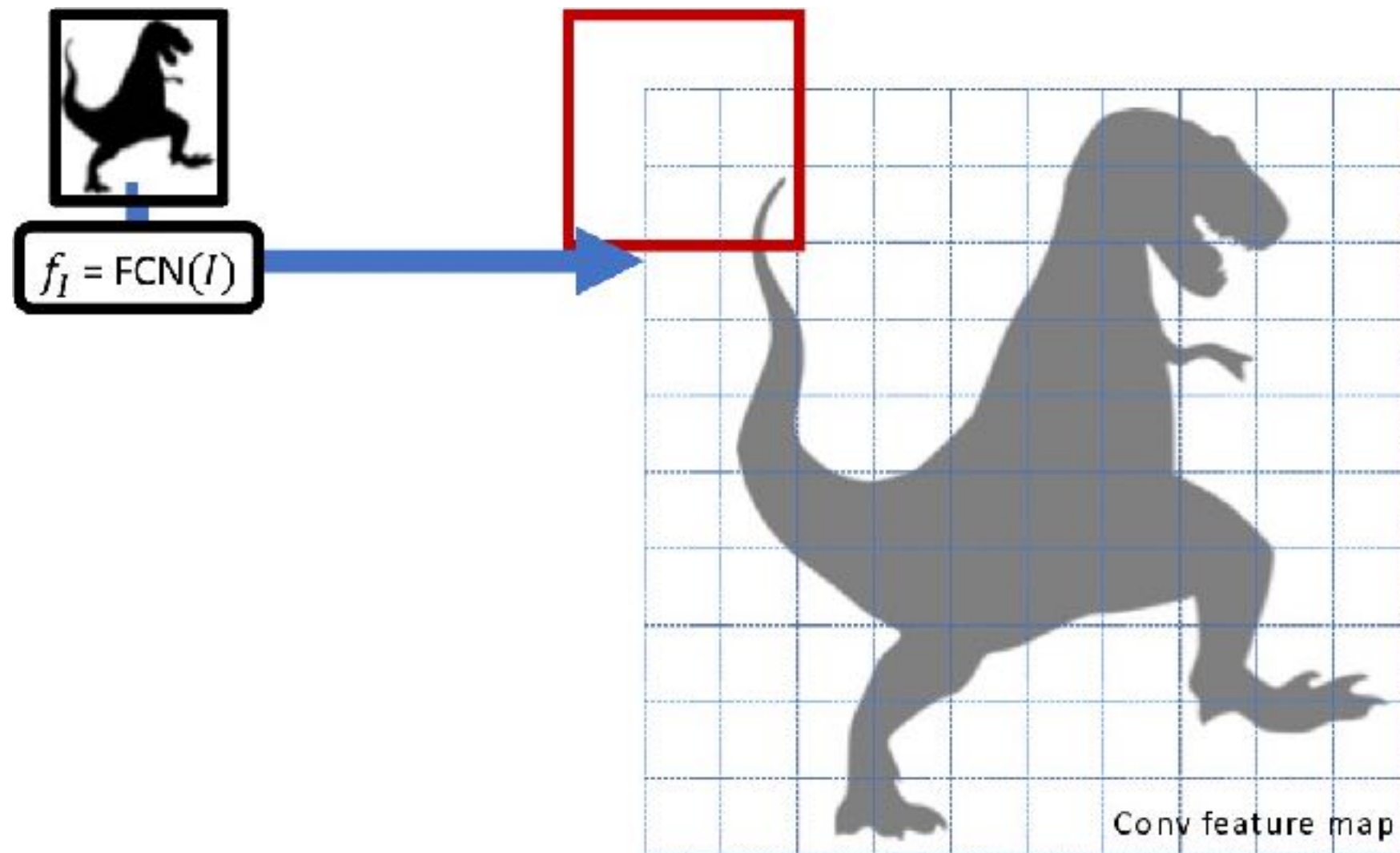
Ren, He, Girshick and Sun et al.: Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NIPS, 2015.

8
0

Faster R-CNN: a 2-stage detector

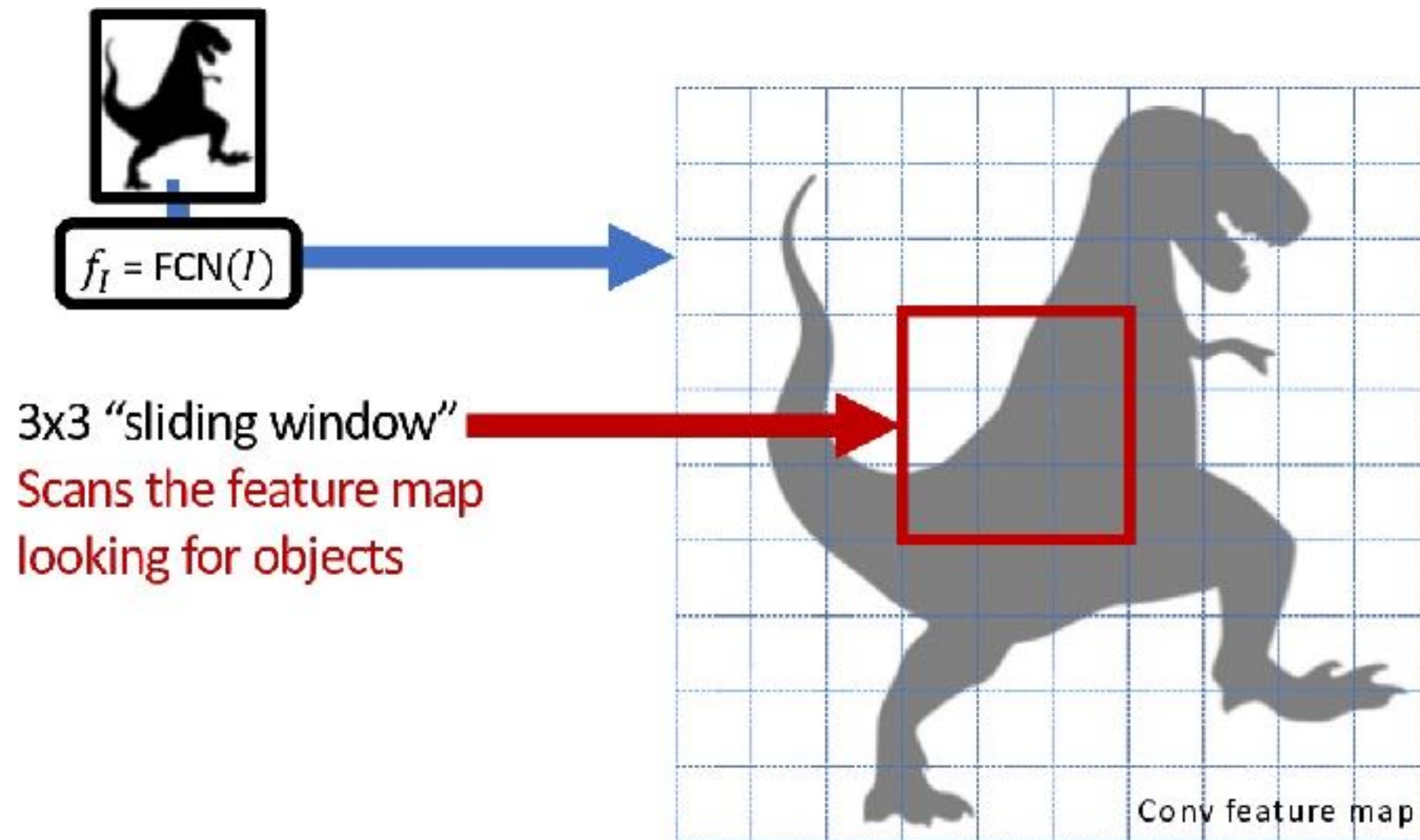
- First stage: Run once per image
 - Backbone network
 - Region proposal network
- Second stage: Run once per region
 - Crop features: RoI pool / align
 - Predict object class
 - Prediction bbox offset

Faster R-CNN (Ren et al. 2015)



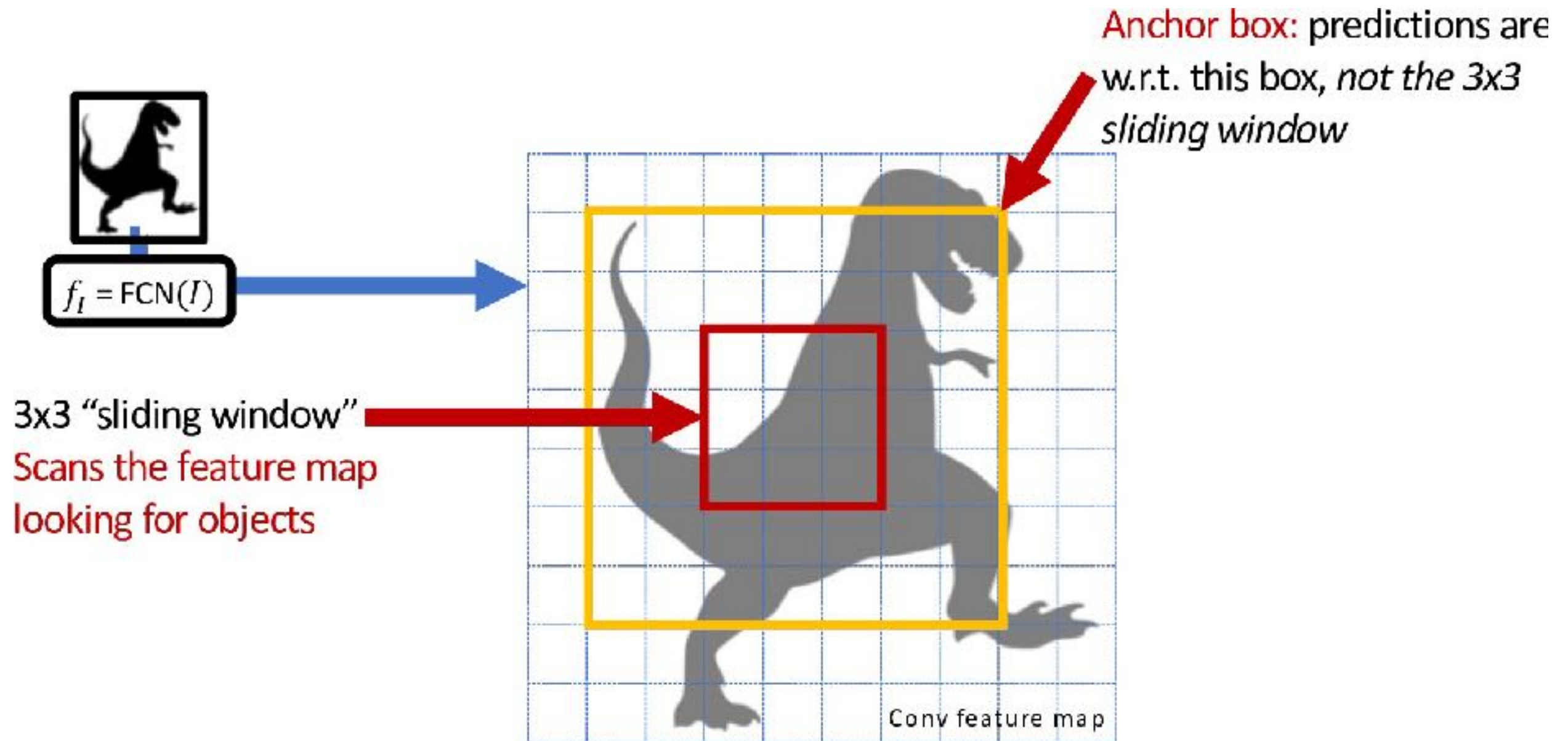
Ren, He, Girshick and Sun et al.: Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NIPS, 2015.

Faster R-CNN (Ren et al. 2015)



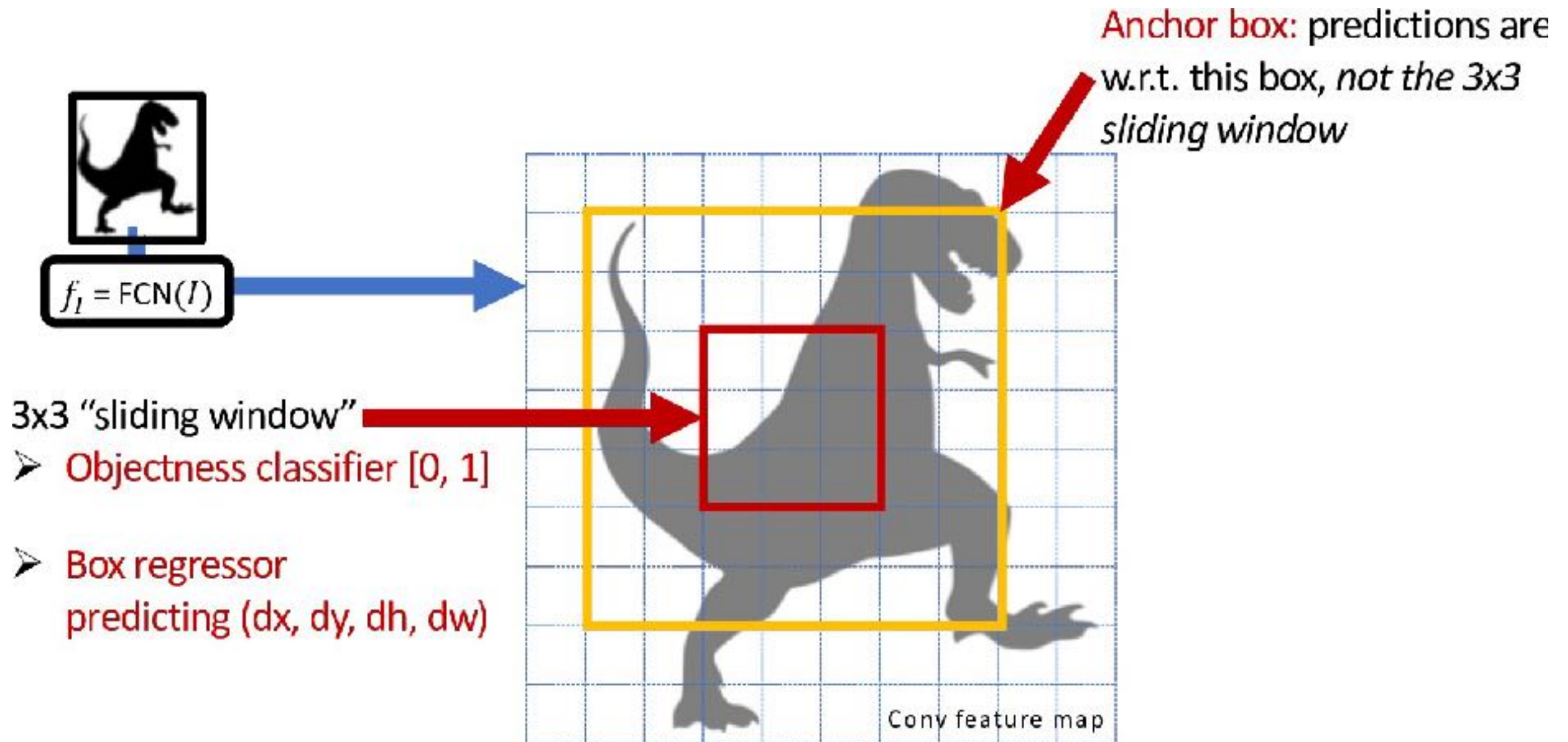
Ren, He, Girshick and Sun et al.: Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NIPS, 2015.

Faster R-CNN (Ren et al. 2015)



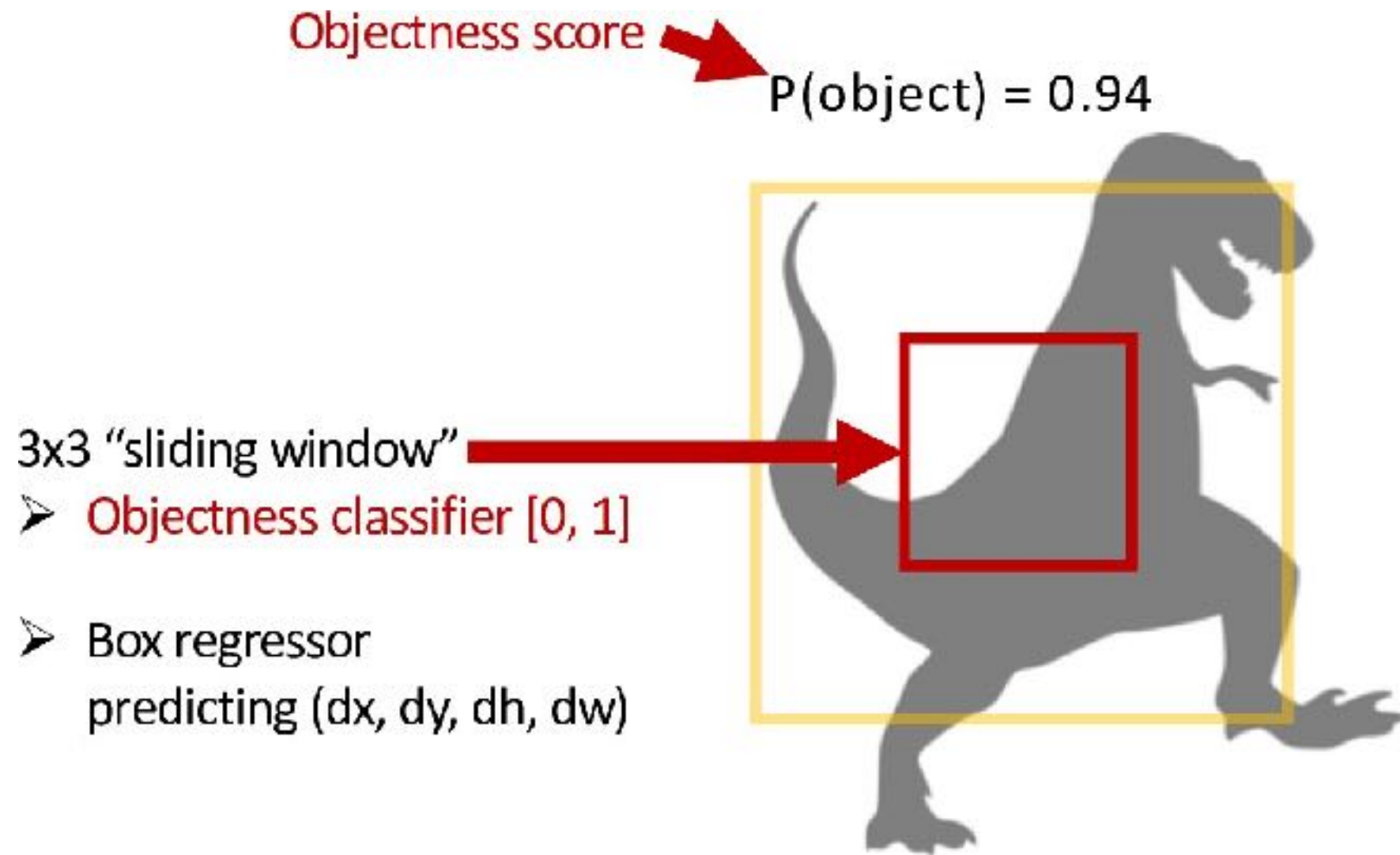
Ren, He, Girshick and Sun et al.: Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NIPS, 2015.

Faster R-CNN (Ren et al. 2015)



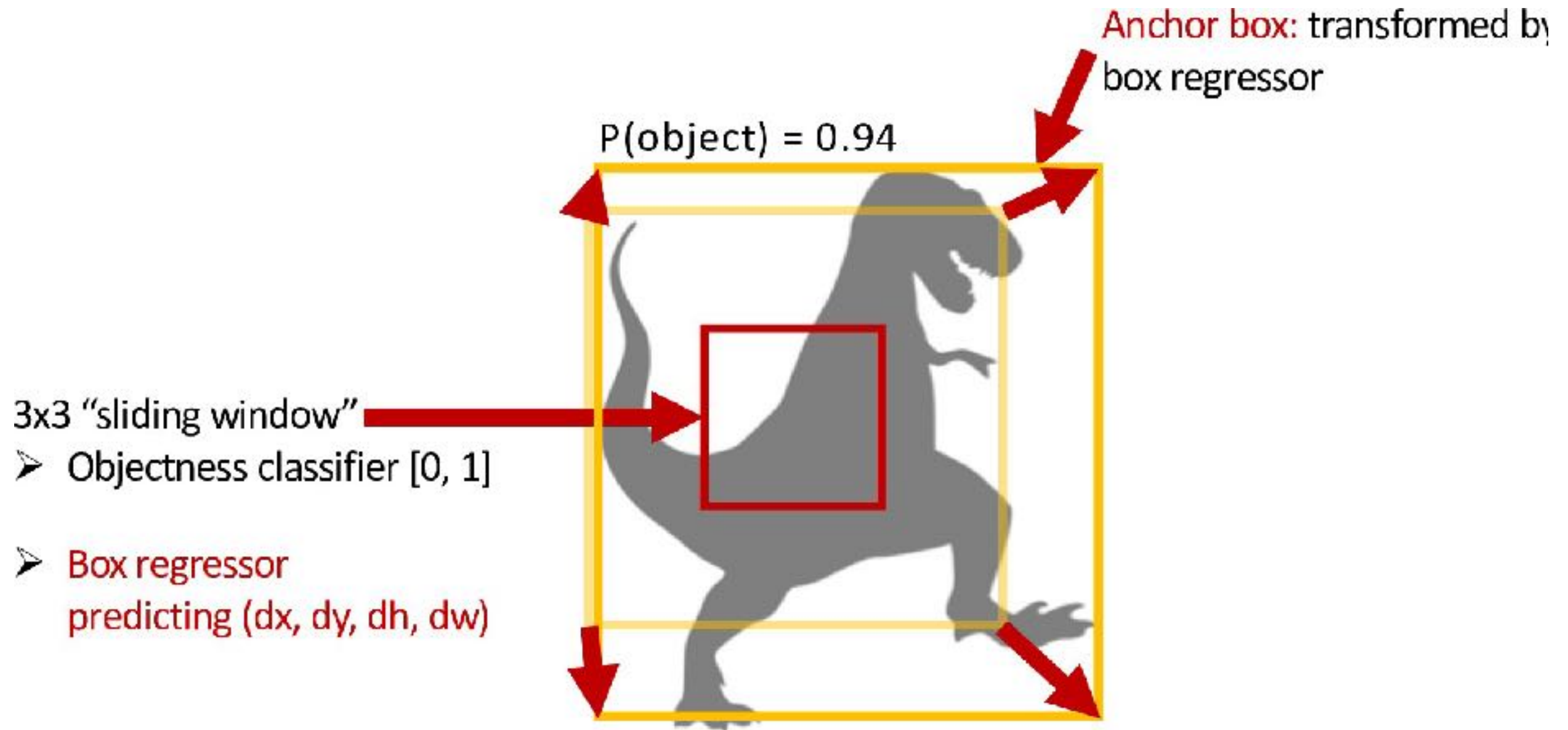
Ren, He, Girshick and Sun et al.: Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NIPS, 2015.

Faster R-CNN (Ren et al. 2015)



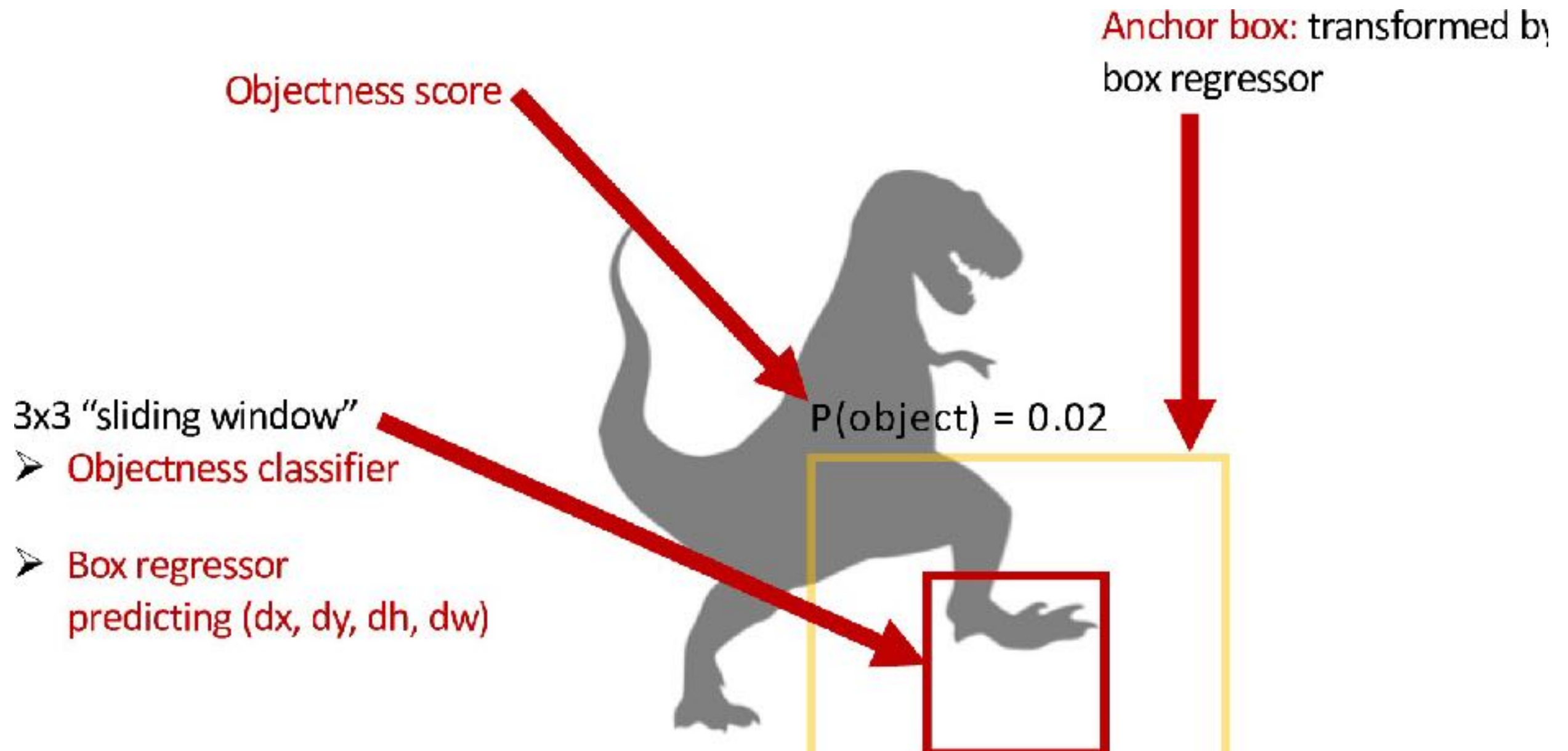
Ren, He, Girshick and Sun et al.: Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NIPS, 2015.

Faster R-CNN (Ren et al. 2015)



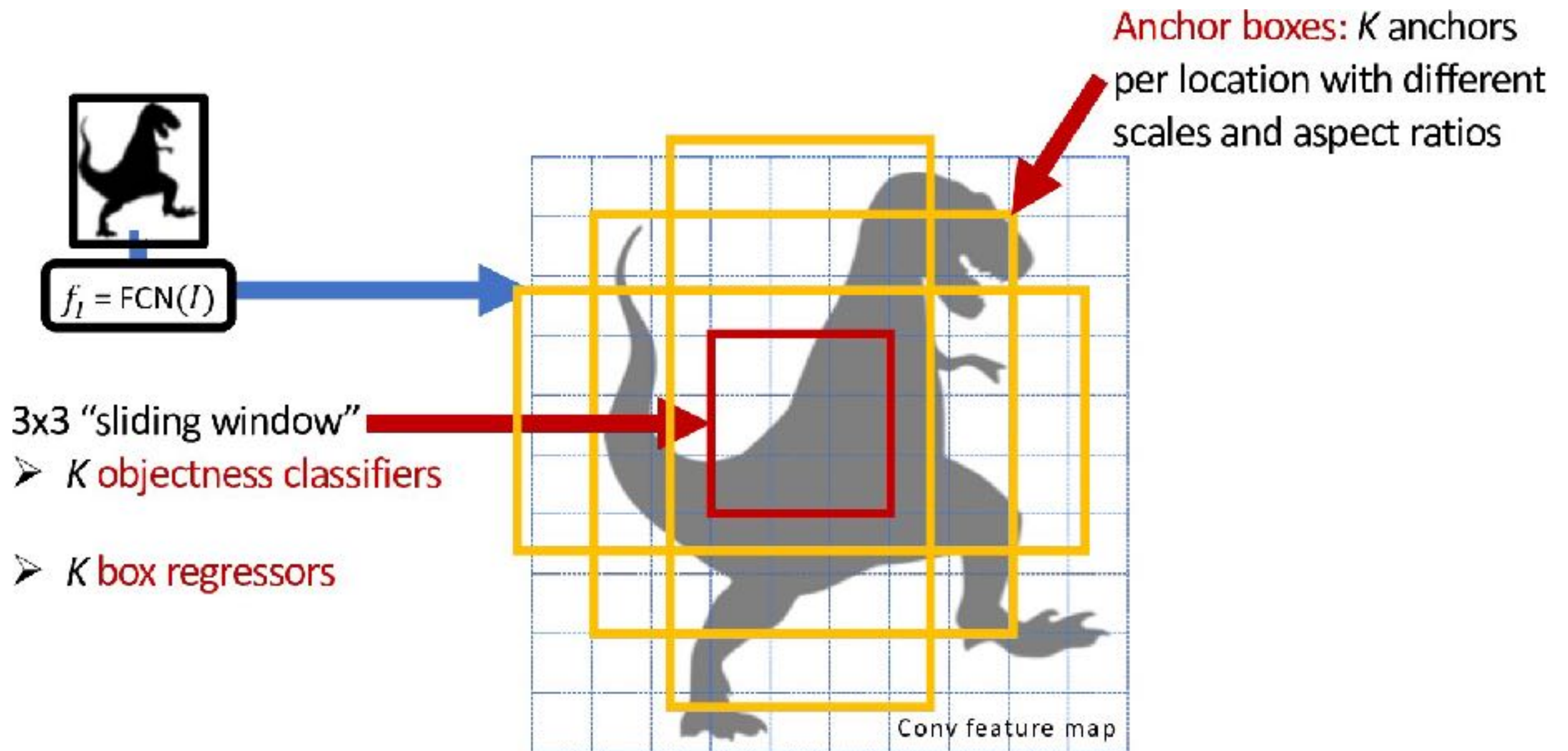
Ren, He, Girshick and Sun et al.: Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NIPS, 2015.

Faster R-CNN (Ren et al. 2015)



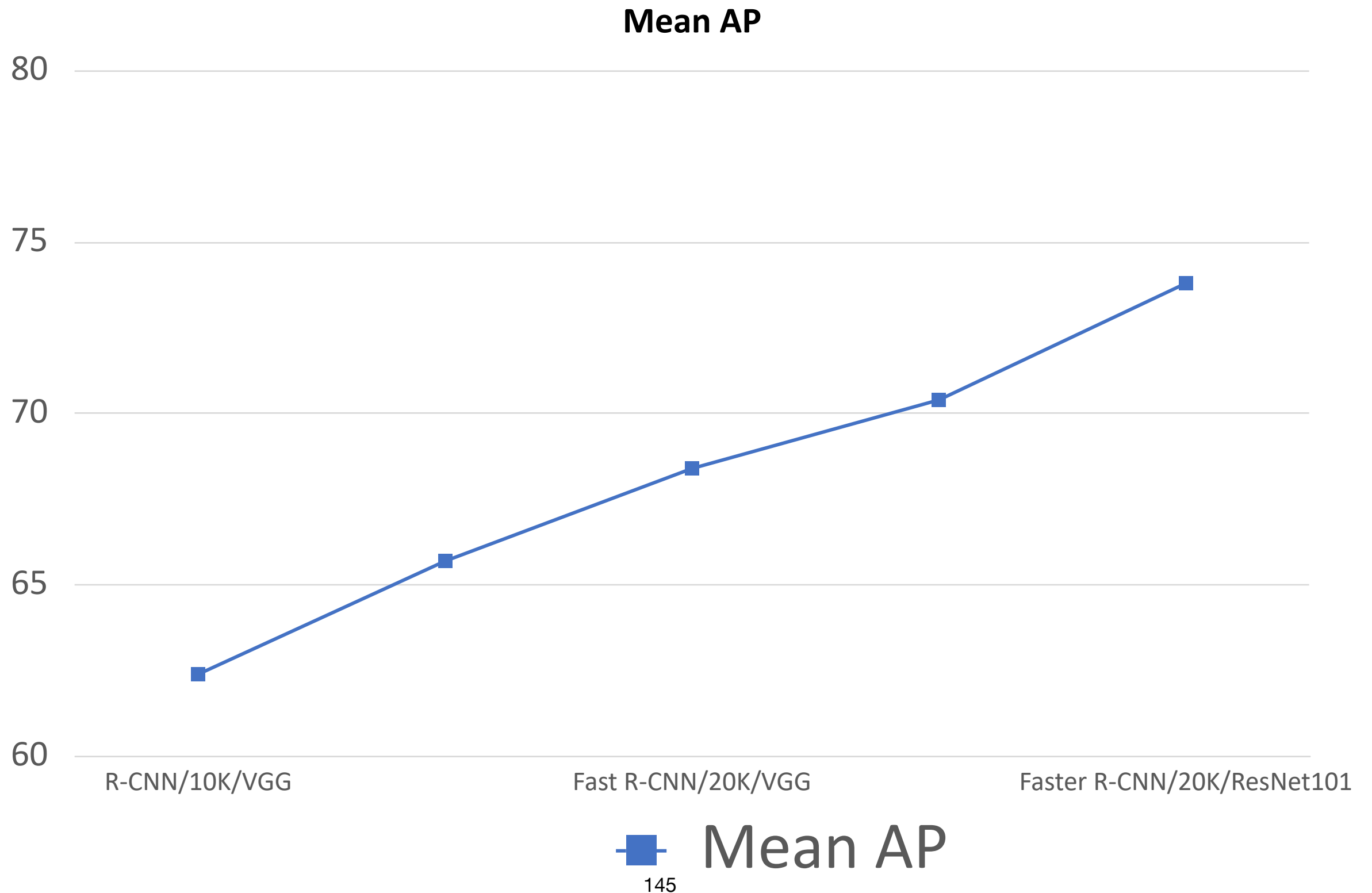
Ren, He, Girshick and Sun et al.: Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NIPS, 2015.

Faster R-CNN (Ren et al. 2015)

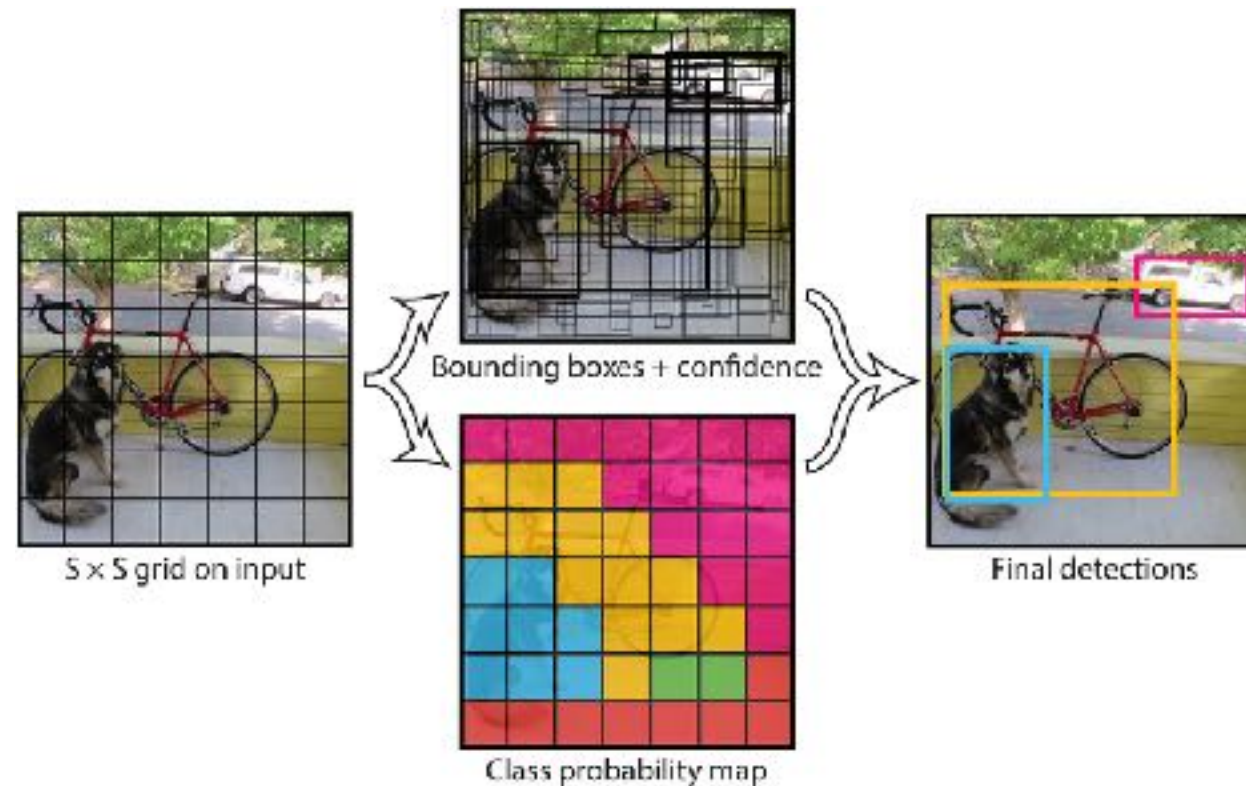


Ren, He, Girshick and Sun et al.: Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. NIPS, 2015.

The R-CNN family of detectors



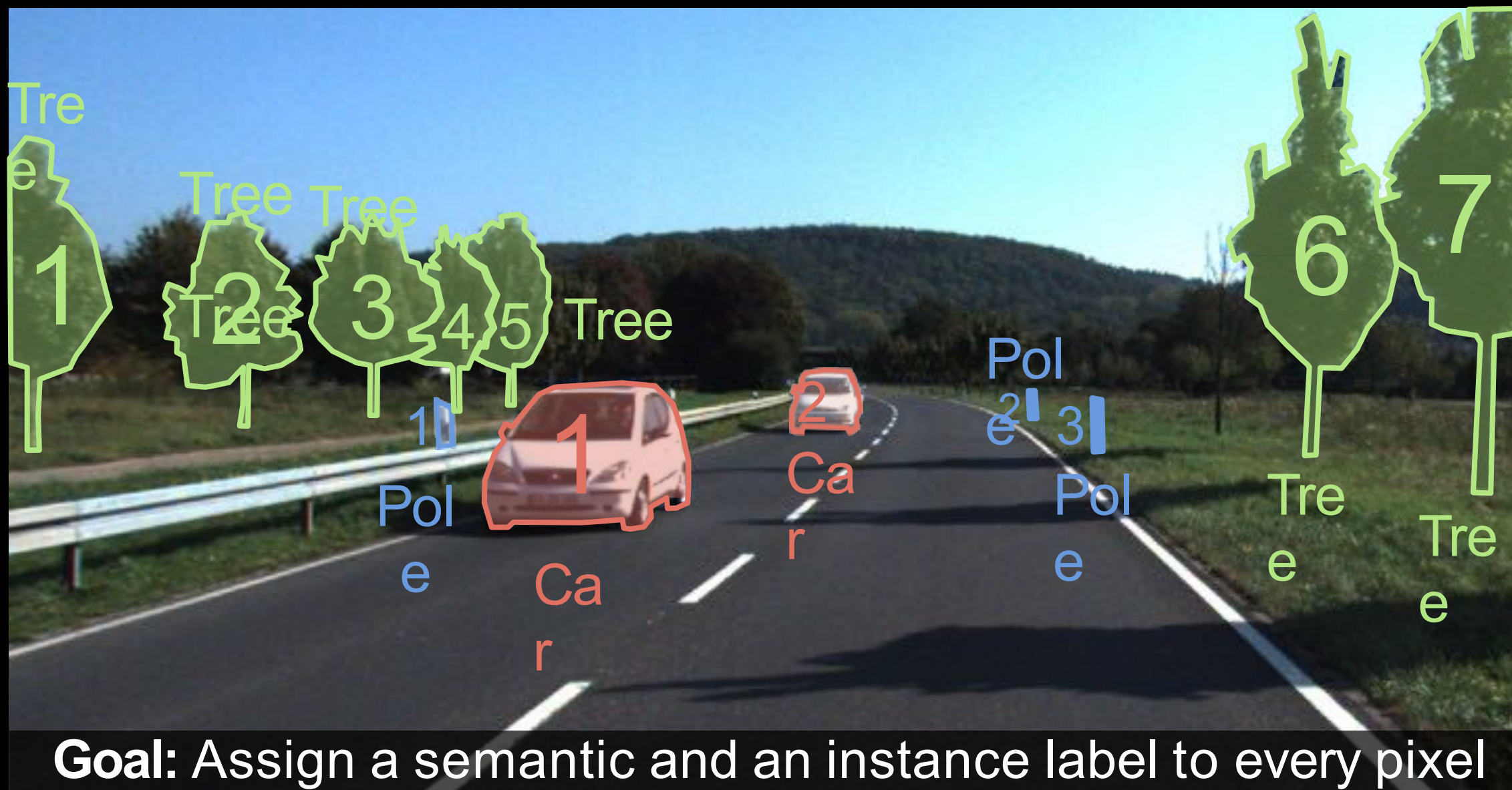
Single Stage Detection



Joseph, Divvala, Girshick and Farhadi:
You Only Look Once: Unified, Real-Time
Object Detection. CVPR, 2016.

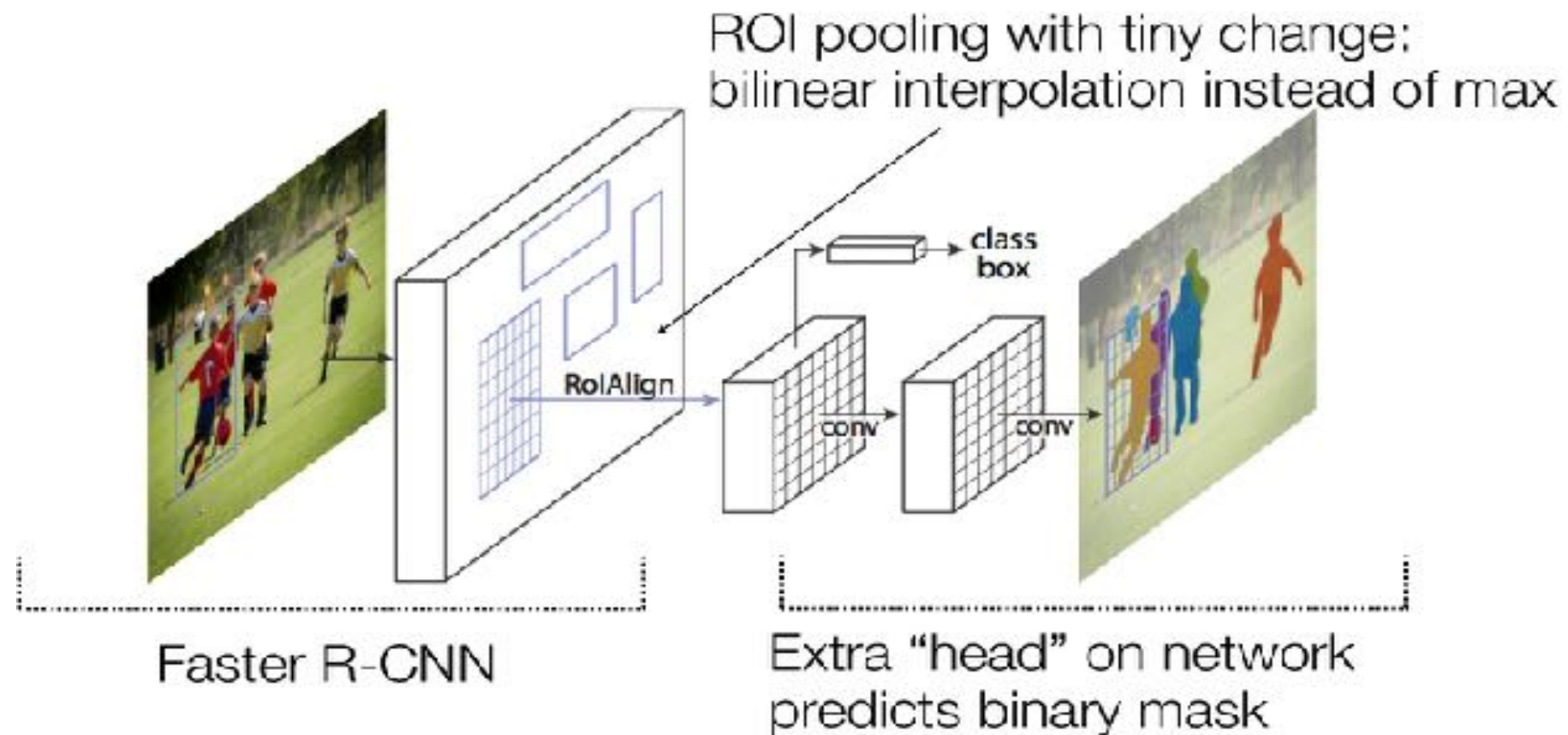
- YOLO and SSD: Alternative to proposal based (two stage) methods
- Predict 2D bounding boxes in a single stage
- Faster, but often worse performance in practice

Instance segmentation



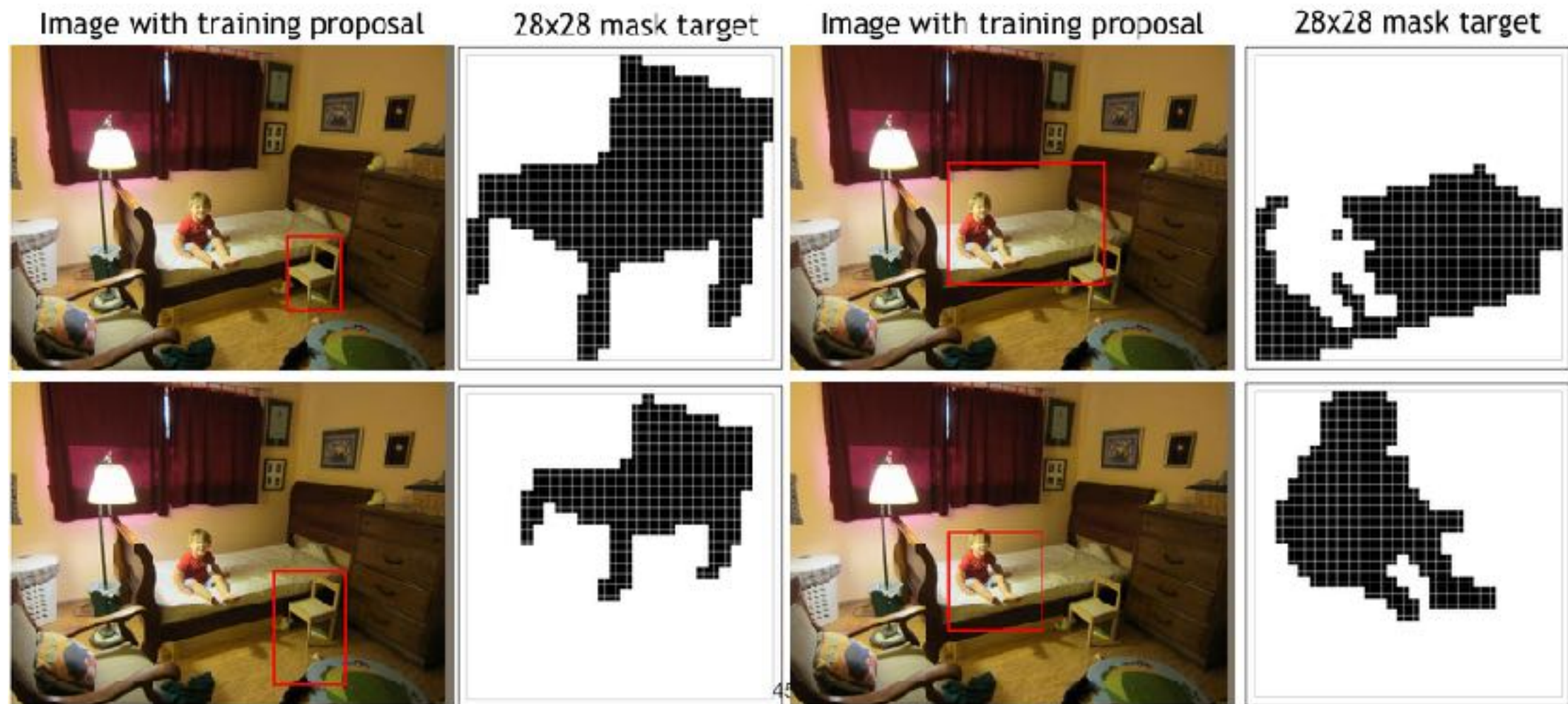
Goal: Assign a semantic and an instance label to every pixel of an object

Mask R-CNN: Faster R-CNN for Instance Segmentation

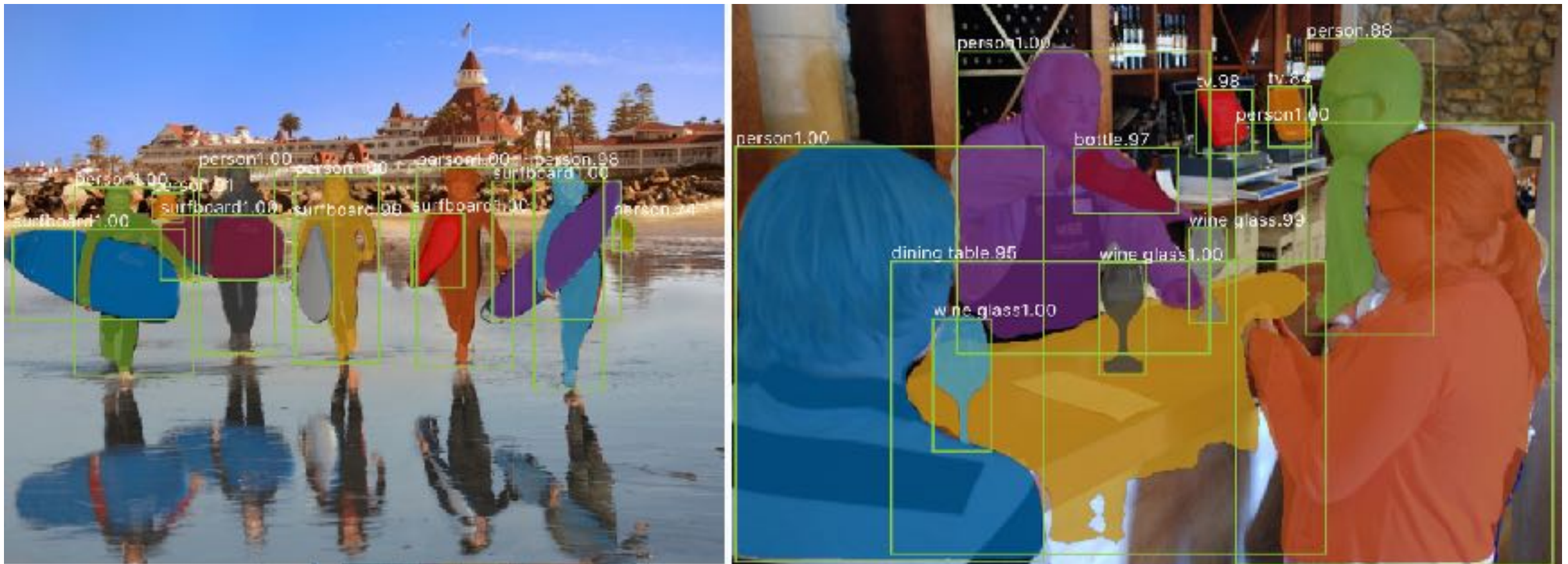


Predict binary instance segmentation (foreground vs. background) per detection

Mask R-CNN: Faster R-CNN for Instance Segmentation



Mask R-CNN: Faster R-CNN for Instance Segmentation



Evaluation: Intersection-over-Union at mask level (not bounding box level)